

COMSATS University, Islamabad Pakistan

PodVerse

A project presented to

COMSATS University, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Rabeea Chughtai **CIIT/SP22-BCS-079/ISB**
Sadia Imran **CIIT/SP22-BCS-087/ISB**

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be a reproduction of some other, we will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Rabeea Chughtai

Sadia Imran

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) Dukan was developed by **RABEEA CHUGHTAI (CIIT/SP22-BCS-079)** and **SADIA IMRAN (CIIT/SP22-BCS-087)** under the supervision of SAADIA MAQBOOL and that in her opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences

---

Supervisor

External Examiner

Head of Department
(Department of Computer Science)

Executive Summary

In today's digital era, small business owners, entrepreneurs, and content creators increasingly rely on podcasts to share their stories, build brand identity, and connect with audiences. However, the process of producing a podcast remains challenging due to high production costs, the need for technical expertise, difficulty in securing credible guests or hosts, and the time-consuming nature of editing and publishing. These challenges create barriers to entry and limit the consistency and quality of podcast content produced by individuals and small teams.

To overcome these challenges, PodVerse is developed—an AI-powered podcast creation platform designed to simplify and modernize the entire podcast production process. PodVerse serves as a centralized and intelligent system that automates core tasks such as recording, editing, and publishing, while also enabling real-time AI-driven interactions between hosts and guests. By integrating technologies like voice cloning, AI-based conversation agents, and real-time language translation, the system eliminates dependency on external hosts or guests and provides creators with tools to produce engaging, professional-quality podcasts effortlessly.

PodVerse is a web-based platform that offers a user-friendly interface, allowing users with minimal technical skills to plan, record, and publish their podcasts seamlessly. Through automation and personalization modules, users can customize voices, set up AI interviewers or guests, and generate audio-visual podcast episodes suitable for multiple platforms.

This intelligent solution ensures that creators can focus more on content quality and storytelling rather than technical complexities. By making podcast creation more accessible, cost-effective, and interactive, PodVerse empowers users to expand their creative reach, strengthen audience engagement, and grow their brands with confidence.

Abstract

PodVerse is a proposed solution that would make it easy for small business owners, entrepreneurs, and content creators to record high-quality podcasts with no technical skills. Podcasting is highly expensive, requires a lot of technical knowledge, and includes other challenges such as finding credible guests and complex production tools. PodVerse offers AI hosts, guests, voice simulations, and customizable multimedia content creation using pre-designed templates. This decreases the production costs and time together. This is an all-in-one, user-friendly solution that allows solo creators to truly be able to take control over the entire process of podcasting from start to end. The project's primary objectives include reducing production time by 50%, user friendly features for recording as well as publishing. It will eliminate entry barriers for even more users so that they might share their content, build their brands, and contribute to more accessible and efficient podcasts.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Saadia Maqbool” .Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are grateful to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us with the values of honesty & hard work.

Rabeea Chughtai

Sadia Imran

Revision History

Name	Date	Reason for Changes	Version

Application Evaluation History

Comments (by committee) ***include the ones given at scope time both in doc and presentation**	**Action Taken**
Scope should be increases, add major modules e.g. a usecase where these podcasts can be used. At the moment the scope is not enough for 2 students with pretrained models/api usage.	• Added Social Media Integration for podcast sharing and engagement. • Introduced Podcast Summary Feature for better accessibility and usability. • Integrated Admin Panel & Analytics for user/content management and insights. • Expanded Audience Engagement with messaging, notifications, and user interactions. • Enhanced AI Video Podcast Creation with avatars, and visual elements
1. Tools' usage and purpose are not clear. 2. Mockups are betrayal. 3. Students' effort is unclear and unjustified with time required. 4. Prompt engineering and voice to text generation are unclear to students. 5. Lack of existing study	• Implementation demo given for speaker creation. • Tool usage explained with updated documentation. • Mockups revised to match implementation.

Supervised by

Ms. Saadia Maqbool

Signature_____

Table of Contents

Abstract **5**
1. Introduction **6**
1.1 Vision Statement 6
1.2 Related System Analysis/Literature Review 6
1.3 Project Deliverables 7
1.4 System Limitations/Constraints 7
1.5 Tools and Technologies 8
1.6 Relevance to Course Modules 8
2. Problem Definition **9**
2.1 Problem Statement 9
2.2 Problem Solution 9
2.3 Objectives 9
2.4 Scope 10
2.5 Modules 10
2.5.1 Module 1: User Authentication & Profile Management 10
2.5.2 Module 2: Voice Cloning 10
2.5.3 Module 3: Knowledge Base Setup 11
2.5.4 Module 4: RAG Retrieval 11
2.5.5 Module 5: Agent Customization 11
2.5.6 Module 6: Podcast Audio Creation 11
2.5.7 Module 7: AI Host 11
2.5.8 Module 8: AI Guest 12
2.5.9 Module 9: Podcast Video Creation & Publishing 12
2.5.10 Module 10: Subscription & Payment 12
2.5.11 Module 11: FAQs Chatbot 12
2.5.12 Module 12: Audience Engagement and Communication 12
2.5.13 Module 13: Admin Panel & Analytics 13
2.6 Project Contribution 13
3. Requirement Analysis **14**
3.1 User classes and characteristics 14
3.2 Requirement Identifying Technique 14
3.2.1 Use Case Diagram 14
3.2.2 Detail Use Cases 18
3.2.3 Event Response Table 65
3.3 Functional Requirements 71
3.3.1 UC-1 Register Account 71
3.3.2 UC-1.2 User Login 75
3.3.3 UC-1.3 Google Authentication 77
3.3.4 UC-1.4 Password Recovery 80
3.3.5 UC-1.5 Logout 83
3.3.6 UC-1.6: Access Sample Podcasts 84
3.3.7 UC-1.7: Start a Podcast Session 85
3.3.8 UC-1.8: View Dashboard 86
3.3.9 UC-1.9: Create and Personalize User Profile 87
3.3.10 UC-1.10: Edit Profile 88
3.3.11 UC-1.11: Follow Creators and View Updates 89
3.3.12 UC-1.12: View Activity Feed of Followed Creators 90
3.3.13 UC-2.1 Voice Cloning Setup 91
3.3.14 UC-3.1 Knowledge Base Setup 93
3.3.15 UC-4.1 AI Agent Customization 95
3.3.16 UC-5.1 Record Podcast Session 97
3.3.17 UC-5.2 Edit, Review, and Export Podcast 99
3.3.18 UC-6.1 AI Host & AI Guest Interaction 106

3.3.19 UC-7.1 Create and Export Video Podcast 108
3.3.20 UC-7.4 Request Podcast Summary 113
3.3.21 UC-7.5 Search for Specific Information in a Podcast Summary 114
3.3.22 UC-7.6 Rate Podcast Session 115
3.3.23 UC-8.1 Subscription Management 116
3.3.24 UC-8.2 Free Content Access Limit 118
3.3.25 UC-8.3 Subscription Expiry Reminder 119
3.3.26 UC-9.1 Ask a Question from FAQs Chatbot 120
3.3.27 UC-9.2 View Suggested FAQs 122
3.3.28 UC-10.1 Receive Notifications for Audience Engagement 123
3.3.29 UC-10.2 User-to-User Messaging and Calls 124
3.3.30 UC-10.3 Engage with Podcasts (Like, Comment, Share) 125
3.3.31 UC-11.1 Manage User Accounts 126
3.3.32 UC-11.2 Review Reported Content 127
3.3.33 UC-11.3 View Platform Analytics 128
3.3.34 UC-11.4 Manage Subscriptions & Payments 129
3.3.35 UC-11.5 Generate User Engagement Reports 130
3.4 Non-Functional Requirements 131
3.4.1 Usability 131
3.4.2 Performance 132
3.4.3 Security 132
3.4.4 Interoperability 132
3.4.5 Integrity 132
3.5 External Interface Requirements 133
3.5.1 User Interfaces Requirements 133
3.5.2 Software interfaces 133
3.5.3 Hardware interfaces 133
3.5.4 Communications interfaces 133
4. Design and Architecture 133
4.1 Architectural Design 133
4.1.1 MVC Architecture 134
4.2 Design Models 136
4.2.1 Activity Diagrams 136
4.2.2 Class Diagram 150
4.2.3 Sequence Diagrams 151
4.2.4 State Transition Diagrams 160
4.3 Data Design 162
4.3.1 Data Dictionary 162
5. Implementation 170
5.1 Algorithm 170
5.2 External APIs/SDKs 173
5.3 User Interface 173
5.3.1 SignIn 173
5.3.2 Library 174
5.3.3 Dashboard 174
5.3.4 Samples 175
5.3.5 Speaker Customization 175
5.3.6 Default Voices 176
5.3.7 Voice Customization 176
5.3.8 Recording Studio 177
5.3.9 Editing Panel 177
177
6. Testing and Evaluation 178
6.1 Unit Testing 178

6.2 Functional Testing 182
6.3 Business Rules Testing 185
6.4 Integration Testing 187
7. Conclusion and Future Work 190
7.1 Conclusion 190
7.2 Future Work 190
8. References 191
9. Plagiarism Report 192

1. Introduction

PodVerse is an AI-powered podcast creation platform developed to simplify and modernize the podcast production journey for small business owners, entrepreneurs, and content creators. By eliminating technical barriers such as high production costs, complex editing processes, and challenges in finding credible guests, PodVerse enables creators to produce engaging, high-quality podcasts effortlessly. The system integrates advanced technologies like voice cloning, real-time AI interactions, and customizable avatars, providing users with a centralized solution for planning, recording, editing, and publishing their podcast content. With a user-friendly interface and modules tailored for automation and personalization, PodVerse ensures seamless experience, empowering creators to build consistent content and strengthen audience engagement.

1.1 Vision Statement

For small business owners, entrepreneurs, and content creators, **the** PodVerse platform is a podcast creation tool **that** automates key processes like voice cloning, editing, and publishing. **Unlike** traditional podcast production methods that require technical skills and high costs, **our product** enables users to create audio and video podcasts easily. By using innovative AI-driven solutions, PodVerse eliminates barriers to entry for podcast creators, enabling them to produce content efficiently. Designed for ease of use, the platform caters to users with minimal technical knowledge while offering advanced options for creators, ensuring a seamless and effective podcasting experience.

1.2 Related System Analysis/Literature Review

Table 1: Related System Analysis

Application Name	Weakness	Proposed Project Solution
Podcastle	Limited to editing, no guest coordination.	PodVerse simulates AI guests for seamless content creation.
Descript	Lacks AI hosts and customizable 3D avatars.	PodVerse includes AI hosts/guests and customizable avatars for enhanced personalization.

Anchor	Basic recording lacks advanced editing and AI tools.	PodVerse provides advanced editing, voice simulation, and AI-driven content for streamlined production.

1.3 Project Deliverables

The PodVerse project will deliver the following key components:

- **Web-Based Platform**: A functional system for planning, recording, editing, and publishing podcasts.
- **AI Voice System:** Enables creation of custom AI hosts or guests using voice cloning.
- **Real-Time Interaction:** Supports natural, intelligent conversations between users and AI agents.
- **Knowledge Base Integration:** Ensures topic-specific and accurate AI discussions.
- **Editing & Publishing Tools:** Provides simple trimming, transcription, and export options (MP3/WAV).
- **Subscription Management:** Includes tier-based access for free and premium users.
- **Documentation & Demo:** Delivers technical documentation and a working prototype showcasing system capabilities.

1.4 System Limitations/Constraints

LI-1: Requires a stable, high-speed internet connection, limiting use in areas with poor connectivity.
LI-2: Supports basic audio/video podcasting but lacks advanced video production features.
LI-3: Voice simulation may not fully capture emotional nuance, affecting content engagement.
LI-4: Primarily designed for small to medium-sized businesses; larger organizations may need customization.
LI-5: Limited customization options for audio/video content, which may restrict users looking for highly personalized or advanced features in podcast production
LI-6: Relies on AI-generated voices, which may not fully replicate natural human speech patterns, leading to a less authentic listening experience.

1.5 Tools and Technologies

Table 2: Tools and Technologies for Proposed Project

	Tools	Version	Rationale
Tools And Technologies	Visual Studio Code	1.84	IDE
	MS Word	365	Documentation
	MS PowerPoint	365	Presentation
	Figma	116.4.2	Mockups Creation
	Supabase	2.39	Database
	Technology	**Version**	**Rationale**
	ReactJS	18.2.0	Front-End development
	FastAPI	4.19.2	Back-End development
	NodeJS	21.7.0	Front-End development
	Python	3.13.0	Deep Learning Models

1.6 Relevance to Course Modules

The development of PodVerse, an AI-powered podcast creation platform, is closely connected to the key concepts learned throughout the **Bachelor of Science in Computer Science (BSCS)** program.
Courses such as **Programming Fundamentals (CSC103), Object-Oriented Programming (CSC241),** and **Data Structures and Algorithms** have helped in designing the core structure of PodVerse, ensuring efficient logic flow and smooth functionality.
Modules like **Artificial Intelligence (CSC462)** and **Machine Learning (CSC354)** directly contributed to building intelligent and interactive features such as the AI Host, AI Guest, and real-time conversation handling.
Additionally, **Database Systems (CSC270)** supported efficient data organization and management for user accounts, podcast storage, and AI knowledge bases.
Courses such as **Software Engineering Concepts (CSC291)** and **Parallel and Distributed Computing (CSC334)** guided the architectural design, ensuring the system's modularity, scalability, and efficient real-time processing.

2. Problem Definition

2.1 Problem Statement

It is difficult for new journalists not to secure well-known interview guests available for their shows, which restricts the amount of impactful content they can produce and build up in their portfolio. Podcasting companies also face very high production costs as often, a lot of money is spent securing a famous host, and dealing with arranging times and recording is costly. This is impossible for small businesses and entrepreneurs already short on time to consistently produce good podcasts, not knowing sufficient technical skill, thus not growing their brand. Similarly, online content creators on different social platforms, including YouTube, find it hard to scale into podcasting considering the time-consuming nature of podcast production that limits them from boosting their content. Such significant obstacles create barriers to entry and growth for the podcasting industry.

2.2 Problem Solution

The disadvantages faced in podcast production can be addressed by providing a solution that automates the core processes involved, such as recording, editing, and publishing. This would mean that there would never be a dependency on external hosts or guests since customizable voices and tones could be integrated into this system, enabling the creators to produce interesting podcasts independently. The ease of use would make this interface accessible for individuals with little technical skills to allow them to create worthy content without being overwhelmed by the technical hurdles. The system, on its own, would help create a natural flow of conversation within the content, thereby making listening interesting for the target audiences. Also, a solo creator would be able to create avatar videos from audio and publish them on desired platforms. This innovative approach would ultimately make it possible for creators to establish and grow their brands with success, giving them all that they need to share their stories to the world and bring it close to its concerned audiences.

2.3 Objectives

BO-1: Make podcast creation effortless by allowing users to clone voices and host podcasts with AI, removing the hassle of finding real guests and handling technical complexities.

BO-2: Enable users to create quality podcasts without requiring technical skills or extra resources.

BO-3: Enhance accessibility to podcast production tools, making it easier for creators to produce and share content effectively.

BO-4: Support businesses in building their brand and engaging audiences with high-quality podcasts.

2.4 Scope

PodVerse is an all-in-one system that helps business owners, entrepreneurs, content creators, and journalists make podcasts easier and better. PodVerse enables users to concentrate on content creation while utilizing advanced AI functionalities to manage technical duties by automating key aspects of production. The platform has features like voice cloning, editing, and recording podcasts with AI agents that help solo performers overcome common problems like finding reliable guests and coordinating schedules PodVerse provides avatars and pre-designed visual templates, allowing users to produce audio and improved video podcasts. People with varying levels of technical expertise can utilize it because of its intuitive design. User profiles and engagement tools are also included in the platform, enabling users to customize their profiles, follow their favorite creators, and communicate with one another through shares, likes, and comments. An integrated communication platform supports user-to-user messaging and calls, building an active community.
With its user-centered approach, PodVerse ensures high-quality content creation, making podcasting a more feasible and useful medium for brand building and audience interactions.

2.5 Modules

2.5.1 Module 1: User Authentication & Profile Management

This module manages authentication, profile customization, and dashboard access for secure login and content engagement.

FE-1: Account creation via email with verification.
FE-2: Secure login via email/password.
FE-3: Password recovery via email verification.
FE-4: Secure Logout.
FE-5: Creator's dashboard for sample podcasts and AI Agent management.
FE-6: View and edit profiles.
FE-7: Activity feeds with updates from followed creators.

2.5.2 Module 2: Voice Cloning

This module allows users to create and manage voice samples for cloning.

FE-1: Enabling the recording or upload of voice samples for cloning.
FE-2: Allowing users to preview voice samples to check on the quality.
FE-3: Allow users to re-record or re-upload if the sample is not as desired.
FE-4: Allow the availability of a library with voices.

2.5.3 Module 3: Knowledge Base Setup

This module facilitates the setup and management of a knowledge base for retrieval.

FE-1: Allow users to upload data in PDFs, URLs, CSV, or JSON.

FE-2: Data is cleaned, processed, and split into meaningful chunks for easy handling.

FE-3: Tag chunks with metadata and link to original sources.

FE-4: Store chunks in a database and generate vector embeddings for better retrieval.

2.5.4 Module 4: RAG Retrieval

This module enhances the retrieval of accurate and relevant information from the knowledge base.

FE-1: Convert queries into vectors to determine purpose.

FE-2: Match vectors with knowledge base data for accurate responses.

FE-3: Re-rank data by relevance for better answers.

FE-4: Provides fact-based correct answers for podcasts.

FE-5: Optimized for large datasets without loss of performance.

FE-6: Allow feedback to enhance agent response quality.

2.5.5 Module 5: Agent Customization

This module allows users to personalize their AI agents.

FE-1: Users can assign a custom name to their AI agent.

FE-2: Users can choose from various voice options to suit their podcast.

FE-3: Users can select or upload an avatar to visually represent their AI agent.

2.5.6 Module 6: Podcast Audio Creation

This module provides tools for creating and publishing podcasts.

FE-1: Let users start dialogues with a customized AI agent.

FE-2: Conversations recorded for user and AI agent.

FE-3: Users allowed minimal editing to trim and splice to rearrange things.

FE-4: Allow users the ability to preview edited podcasts for flow and polish.

FE-5: Export podcasts in MP3/WAV.

FE-6: Save recordings for future use.

2.5.7 Module 7: AI Host

This module provides an AI host to manage the flow of podcasts.

FE-1: AI Host presents the flow of podcasts, introduces the episodes and closes with a summary.

FE-2: Tracks podcast time and gives reminders for the schedule.

FE-3: Facilitates smooth transitions between guests or topics.

FE-4: Provides summary prompts to summarize key points for clarity and engagement.

2.5.8 Module 8: AI Guest

This module provides an AI guest to enhance podcast discussions.

FE-1: AI Guest responds to real-time queries using dynamic responses.
FE-2: AI Guest shares appropriate stories to add depth to the discussion.
FE-3: AI Guest gives back comments/reactions to facilitate engagement.

2.5.9 Module 9: Podcast Video Creation & Publishing

This module enables the creation of video content for podcasts, and AI-generated text summaries.

FE-1: Users select avatars for each of the speakers.
FE-2: The system syncs based on the audio of the podcast.
FE-3: Users can choose from the pre-defined templates and visuals for the video.
FE-4: Pre-designed split-screen video template with two avatars.
FE-5: Exports the final video in MP4 format or publish the final video.
FE-6: Allow creators to rate podcast sessions.
FE-7: Provide written feedback on conversation skills, voice clarity, and AI accuracy.

2.5.10 Module 10: Subscription & Payment

This module manages subscription and payment processes.

FE-1: Let users access some free, limited content.
FE-2: After the limit, users are prompted to subscribe for continued access.
FE-3: Secure payment gateway integration supporting credit/debit cards.
FE-4: User can control subscriptions, see payment history, change methods.
FE-5: Give reminders before expiration with options of auto-renewal.

2.5.11 Module 11: FAQs Chatbot

This module provides a chatbot to answer frequently asked questions.

FE-1: Users can ask about podcasting, AI customization, or troubleshooting.
FE-2: The chatbot retrieves relevant answers from the FAQs and support materials.
FE-3: Provides concise, direct responses for quick problem resolution.

2.5.12 Module 12: Audience Engagement and Communication

This module enables audience interaction, providing a communication platform, and podcast feedback to enhance creator engagement.

FE-1: Notifications for creators about audience interactions.
FE-2: Highlighting top creators and trending content.
FE-3: Integrated communication platform for user-to-user messaging and calls.
FE-4: Generate and display a text-based summary of podcast episodes.

2.5.13 Module 13: Admin Panel & Analytics

This module provides administrative functionalities, including user management, content moderation, analytics, and system configuration.

FE-1: Allow admin to view, search, deactivate, reactivate, or delete user accounts

FE-2: Enable admin to review reported podcasts and comments.

FE-3: Display live statistics on active users, total podcasts created, trending content, user engagement heatmaps, and demographics.

FE-4: Allow admin to track active subscriptions, process refunds, and manage payment disputes.

FE-5: Provide insights into user engagement, podcast performance, and trending topics.

2.6 Project Contribution

PodVerse introduces several innovations that simplify and modernize podcast creation through AI and automation:

- **AI-Driven Podcasting:** Enables fully automated conversations using AI hosts and guests, eliminating the need for real participants.
- **Voice Cloning:** Allows users to create customized and realistic voices for hosts or guests.
- **Editing Platform:** Provides basic yet efficient tools for audio cleanup, and trimming to simplify podcast production and improve overall quality.
- **Knowledge Base Integration:** Lets AI access user-uploaded files to provide factual, brand-specific, and context-aware responses.
- **Supabase-Powered Cloud Storage:** Ensures secure, scalable, and real-time data management for users and podcast content.

These contributions make PodVerse an innovative, user-friendly, and cost-effective solution that empowers creators to produce professional-quality podcasts without technical expertise.

3. Requirement Analysis

3.1 User classes and characteristics

Table 3: User Classes

User class	Description
Admin	The admin oversees platform management, ensuring compliance and smooth operations. They can manage user accounts, moderate flagged content, track subscriptions, and analyze revenue reports. Admins access a dedicated dashboard displaying real-time statistics, engagement trends, and system insights. The admin ensures PodVerse remains a secure and well-regulated environment.
Creator	The Creator is a user who records, edits, and publishes podcasts. Each Creator has their own **channel**, where they can post episodes, manage content, and customize their AI hosts/guests. They can track performance analytics such as views, shares, engagement, and listener demographics through the **Channel Insights Dashboard**. Creators can also utilize AI-powered tools like voice cloning and language translation to enhance their podcasts.
Viewer	The Viewer is a user who consumes podcast content but **does not have a channel**. They can follow Creators, engage through likes, comments, and shares, and receive notifications about new episodes. Viewers can explore trending podcasts, search for topics of interest, and provide feedback on episodes. While they do not create content, they contribute to community engagement and content discovery.

3.2 Requirement Identifying Technique

This section describes the techniques that identify PodVerse's requirements in such a way that it maps a way for PodVerse Final Document better understanding of its functional and non-functional requirements.

3.2.1 Use Case Diagram

Below is the case diagram of the Creator (Figure 2) Viewer (Figure 3) Admin (Figure 4).

3.2.1.1 Creator

Below is the use case diagram of Actor-Creator

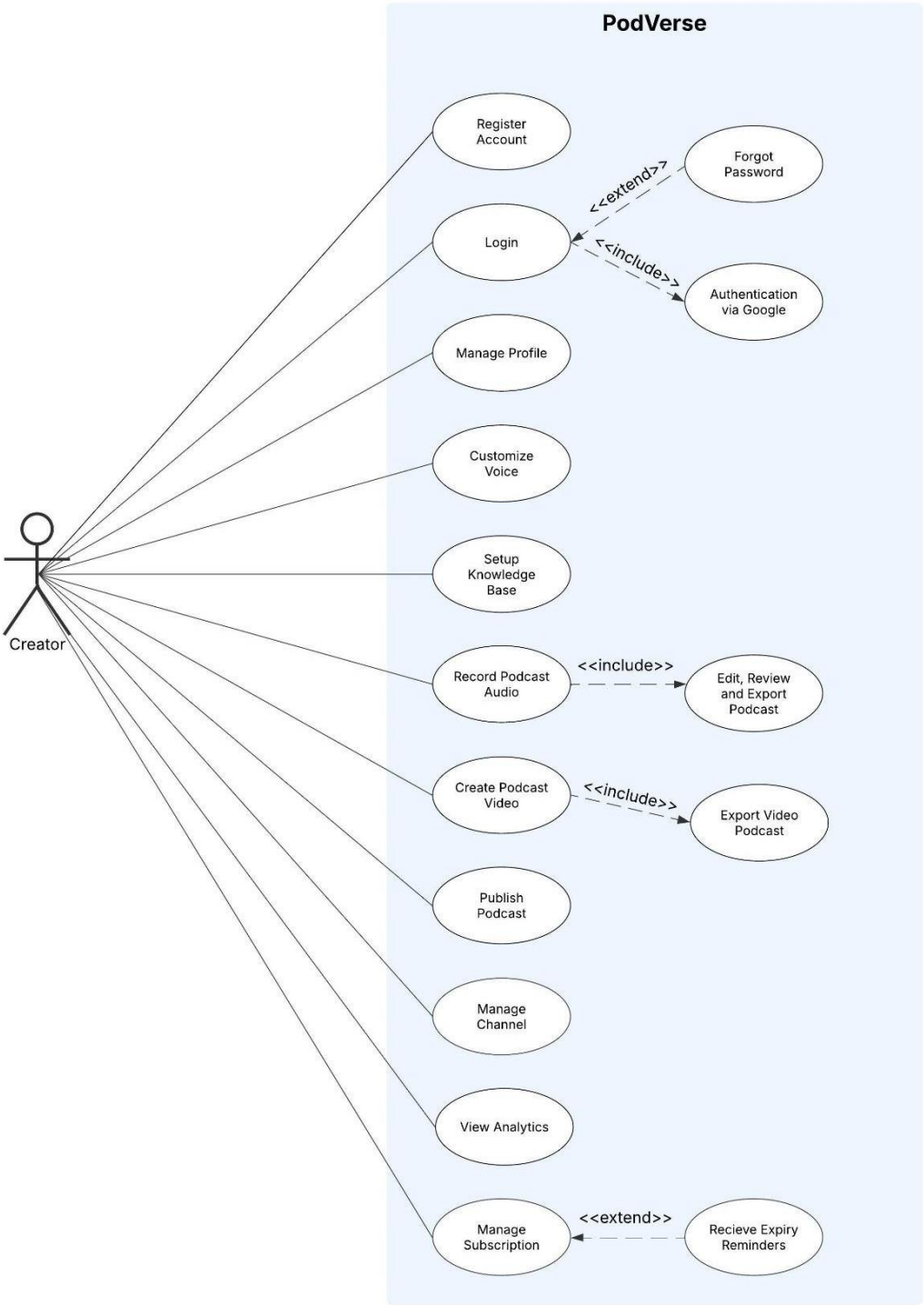


Figure 1: Use Case Diagram of Creator

3.2.1.2 Viewer

Below is the use case diagram of Actor-Viewer

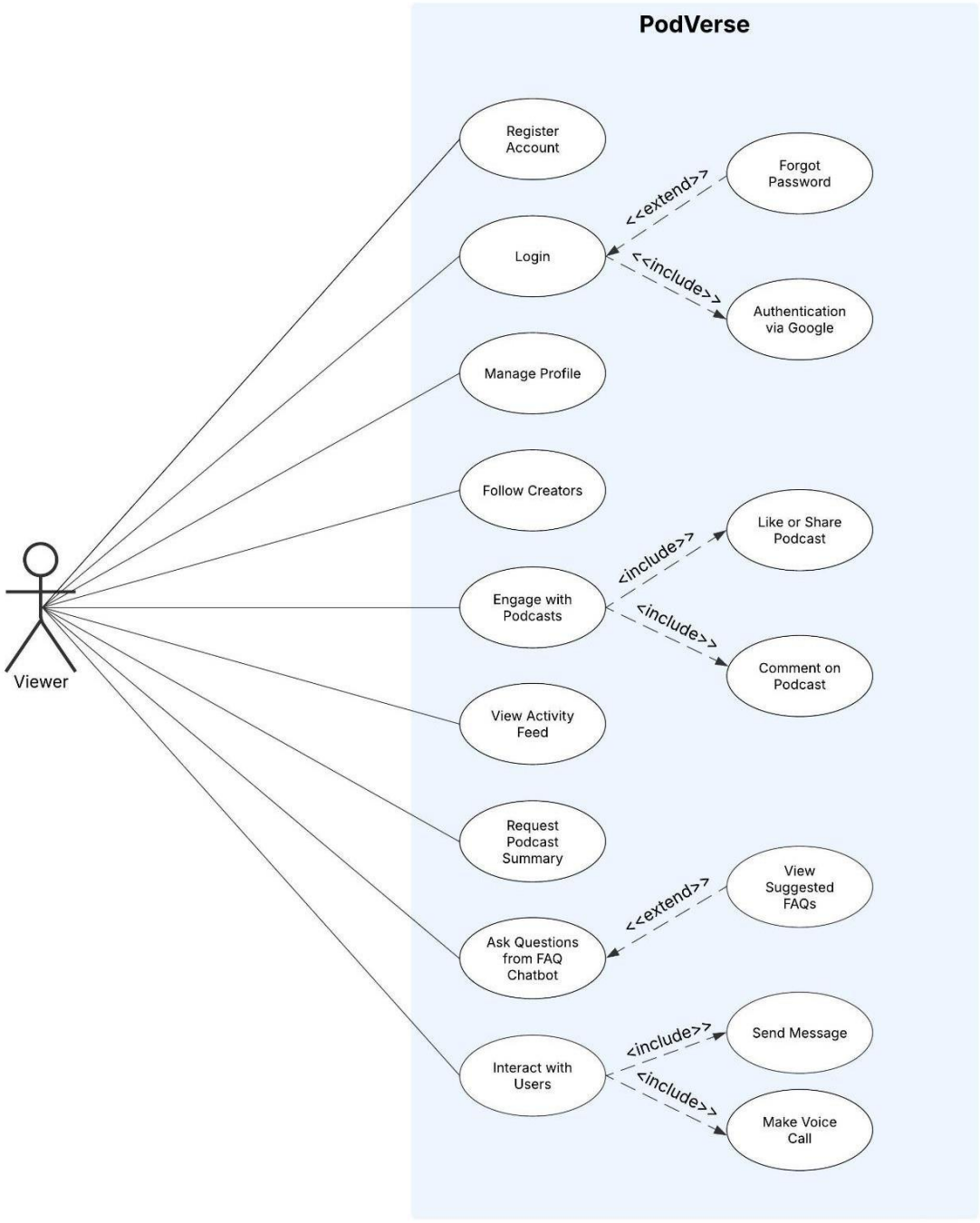


Figure 2: Use Case Diagram of Viewer

3.2.1.3 Admin

Below is the use case diagram of Actor-Admin

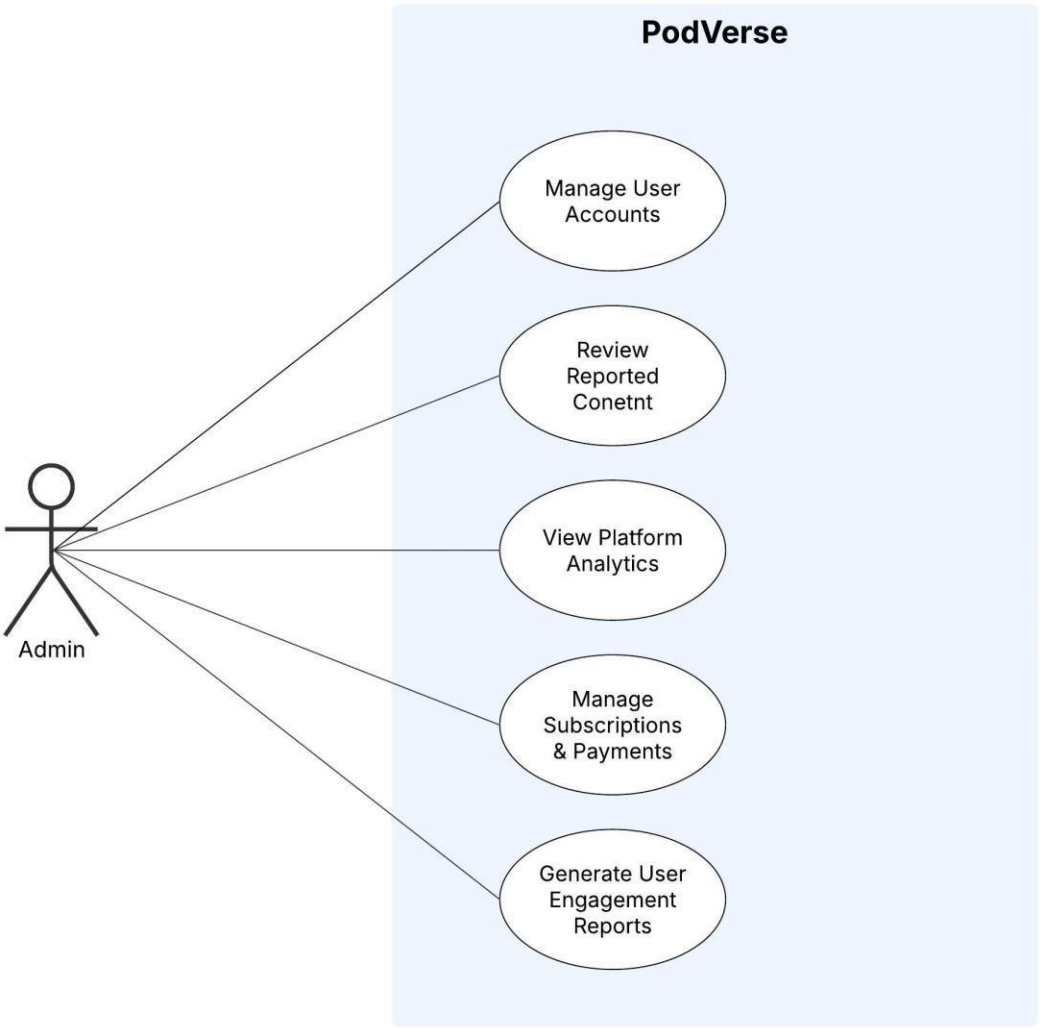


Figure 3: Use Case Diagram of Admin

3.2.2 Detail Use Cases

3.2.2.1 Module 1: User Access & Profile Management

UC-1.1: Register Account

In this case, a user registers via email/password or social login. The system ensures email uniqueness and password strength, requiring email verification for email-based sign-ups, while social logins bypass this step for seamless access.

Table 4: UC-1.1 Register Account

Actors:	User
Trigger:	User selects the **"Sign Up"** option from the login page.
Preconditions:	**PRE-1:** User has a valid email. **PRE-2:** User has an internet connection.
Postconditions:	**POST-1:** User account is created and stored. **POST-2:** A verification email is sent. **POST-3:** User can log in after verifying email.
Normal Flow:	**1.0 Register Account** 1. User selects the "Sign Up" option. 2. User enters email & password. (See 1.0. E1, 1.0. E2) 3. System checks uniqueness & password strength. 4. If valid, system creates the account and sends a verification email. (See 1.0. E3) 5. User verifies the email. 6. User is redirected to the dashboard.

Alternative Flows:	**1.1 Register via Social Login** 1. User selects Google/Facebook login instead of email/password. 2. System retrieves user details & skips email verification.
Exceptions:	**EX-1: 1.0. E1: Email already in use** System prompts the user to enter a new one. 2a. If the user cancels, the system terminates registration. 2b. If a user enters a new email, use case resumes at step 2. **EX-2: 1.0. E2: Weak password entered** System prompts users to meet security criteria. **EX-3: 1.0. E3: Verification link expired** System allows resending a new link.
Business Rules:	**BR-1:** Email must be unique. **BR-2:** Password must meet security criteria. **BR-3:** Email verification is mandatory before login.
Assumptions:	1. User has a working email. 2. OAuth login is configured properly.

UC-1.2: User Login

A registered user logs in using email/password or social login (Google/Facebook). The system verifies credentials and grants access to the dashboard. If authentication fails, appropriate error messages are shown, and multiple failed attempts may trigger a temporary lock.

Table 5: UC-1.2 User Login

Actors:	User
Trigger:	User selects the **"Log In"** option from the login page.

Preconditions:	**PRE-1:** User must have a registered account. **PRE-2:** If using email/password, account must be verified.
Postconditions:	**POST-1:** User session is created. **POST-2:** User is redirected to the dashboard.
Normal Flow:	**1.0 User Login** 1. User enters email & password. (see 1.0.E1, 1.0.E2) 2. System validates credentials. 3. If valid, system creates a session and logs the user in.
Alternative Flows:	**1.1 Social Login** 1. User logs in via Google/Facebook. 2. System verifies identity and logs user in directly.
Exceptions:	**EX-1: 1.0. E1: Incorrect password** System prompts for a retry. **EX-2: 1.0. E2: Unverified email** System notifies the user to verify the email. **EX-3: 1.0. E3: Account not found** System suggests creating a new account.
Business Rules:	**BR-1:** Only verified accounts can log in. **BR-2:** Multiple failed login attempts should trigger temporary lockout.
Assumptions:	1. Users know their credentials. 2. System is connected to authentication services.

UC-1.3: Google Authentication

A user logs into PodVerse using Google authentication for a seamless and secure login. The system verifies credentials via Google and grants access, eliminating the need for a separate password.

Table 6: UC-1.3 Google Authentication

Actors:	User, Google Authentication System
Trigger:	User selects the "Sign in with Google" option on the login page.
Preconditions:	**PRE-1:** User must have a valid Google account. **PRE-2:** System must support Google OAuth authentication.
Postconditions:	**POST-1:** User is authenticated and logged into PodVerse. **POST-2:** Google account is linked for future logins.
Normal Flow:	**1.0 Login with Google** 1. User selects "Sign in with Google". 2. System redirects the user to Google's authentication page. 3. User enters Google credentials. (see 1.0.E1) 4. Google requests permission for authentication. 5. User grants permission. 6. System retrieves user details and logs them in.
Alternative Flows:	**1.1 Auto-login for linked accounts** 1. If the user has previously linked to their Google account, they are logged in directly.
Exceptions:	**EX-1: 1.0.E1: Google authentication fails** System displays an error and prompts the user to retry. **EX-2: 1.0.E2: User denies permission** System cancels login and redirects to the login page.

Business Rules:	**BR-1:** Only valid Google accounts can be used for authentication. **BR-2:** If the user has not registered before, Google login creates a new account with retrieved details**.**
Assumptions:	1. User has an active internet connection. 2. Google authentication services are operational.

UC-1.4: Password Recovery

In this case, a user resets their password using the "Forgot Password" option. The system verifies the entered email and sends a password reset link. The user clicks the link, enters a new password, and the system updates their credentials. If the email is unregistered or the reset link has expired, appropriate error messages are displayed.

Table 7: UC-1.4 Password Recovery

Actors:	User
Trigger:	User selects the **"Forgot Password"** option from the login page.
Preconditions:	**PRE-1:** User must have a registered email.
Postconditions:	**POST-1:** A password reset link is sent to the user. **POST-2:** User sets a new password.
Normal Flow:	**1.0 Password Recovery** 1. User clicks "Forgot Password". 2. User enters a registered email. (See 1.0. E1) 3. System sends a password reset link. 4. User clicks the link and enters a new password. (See 1.0. E2) 5. System updates the password and confirms the change.

Alternative Flows:	**1.1 Request New Reset Link** 1. User requests a new reset link if the previous one expired.
Exceptions:	**EX-1: 1.0. E1: Email not registered** 1. System displays an error message. 2a. If the user cancels, the process terminates. 2b. If a user re-enters an email, resume at step 2. **EX-2: 1.0. E2: Reset link expired** 1. System prompts the user to request a new one.
Business Rules:	**BR-1:** Reset links should expire within 30 minutes. **BR-2:** New password must meet security criteria.
Assumptions:	1. Users can access their email. 2. Reset links work only for registered users.

UC-1.5: Logout

In this case, a User can securely log out from their active session in the application. The User will initiate the logout process, which will end their active session and redirect them to the login page.

Table 8: UC-1.5 Logout

Actors:	User
Trigger:	User selects the log out option on dashboard page.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** User must be on Dashboard Page.

Postconditions:	**POST-1:** User is logged out of the application.
Normal Flow:	1.0 Initiate Logout Process 1. User selects the logout option on the dashboard page. (see 1.1, 1.2) 2. System displays a confirmation message, “Are you sure you want to logout?” 3. User confirms their intention to logout by selecting “OK”. (See 1.0. E1) 4. System logs the user out and redirects them to login page.
Alternative Flows:	1.1 Cancel logout process 1. User selects the option to cancel during the logout confirmation step. (see 1.0) 2. System does not proceed with the logout and use case terminates. 1.2 No active internet connection 1. Users try to log out without an active internet connection. (see 1.0) 2. System displays a message: "Internet connection is required to log out." 3. System does not proceed with the logout and use case terminates.
Exceptions:	**EX-1: 1.0. E1 Server or system error during logout** 1. System encounters an unexpected error, such as a server issue, during the logout process. 2a. If the user attempts to log out again, the system retries the logout process. 2b. Else if the user decides not to retry and cancels, the use case terminates.
Business Rules:	**N/A**
Assumptions:	1. An active internet connection is available to the user.

UC-1.6: Access Sample Podcasts
A user browses sample podcasts for inspiration before starting their own podcast.

Table 9: UC-1.6 Access Sample Podcasts

Actors:	User
Trigger:	User selects **"Sample podcasts"** from the dashboard panel.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** User has an internet connection.
Postconditions:	**POST-1:** Sample Podcasts are displayed.
Normal Flow:	1. Users navigate Sample Podcasts. 2. System loads featured sample podcasts. 3. User plays, pauses, or skips through podcasts.
Alternative Flows:	**AF-1:** User searches for specific sample podcasts using a filter.
Exceptions:	**EX-1:** No Sample podcasts are found.
Business Rules:	**BR-1:** Only verified users can access sample podcasts.
Assumptions:	1. User has an active internet connection.

UC-1.7: Start a Podcast Session
A user wants to start a podcast session with either an AI guest or AI host.

Table 10: UC-1.7 Start a Podcast Session

Actors:	Creator
Trigger:	Creator logs in and selects **"Start New Session".**
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** User must have access to AI Guest or AI Host.
Postconditions:	**POST-1:** User can now specify the settings for podcast session firstly agent customization.
Normal Flow:	1. Users navigate to Start a New Session. 2. Users choose to continue either with AI Host or AI Guest. 3. User can now set the Agent customization.
Alternative Flows:	If there is an Incomplete Session. 1. Incomplete Session is opened. 2. Resume from the previous saved session.
Exceptions:	**EX-1:** Credits are finished.
Business Rules:	**BR-1:** Only verified users can access sample podcasts.
Assumptions:	2. User has an active internet connection. 3. User has credits to create a podcast session.

UC-1.8: View Dashboard
A user wants to go to Dashboard screen or home screen.

Table 11: UC-1.8 View Dashboard

Actors:	User
Trigger:	User selects the home option on dashboard screen.
Preconditions:	**PRE-1:** User must be logged in.
Postconditions:	**POST-1:** User sees a list of relevant posts based on their profile and the people they have followed.
Normal Flow:	1. User selects the "**home**" option. 2. System retrieves and displays a list of podcasts based on: a. User's own podcasts. b. Posts from users they follow. c. Public posts (depending on platform privacy settings).
Alternative Flows:	N/A
Exceptions:	**EX-1:** No internet connection • System shows an error message. • System tries to reload.
Business Rules:	**BR-1:** Appropriate content moderation policies must be implemented.
Assumptions:	1. Users can locate the home option from nav-bar on Dashboard screen.

UC-1.9: Create and Personalize User Profile

In this case, a user creates and customizes their profile, including setting a display name, uploading a profile picture, and adding personal details.

Table 12: UC-1.9 Create and Personalize User Profile

Actors:	User
Trigger:	User navigates to the profile section for the first time.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** System must allow users to update profile information.
Postconditions:	**POST-1:** The user profile is created and saved. **POST-2:** User can update their profile at any time.
Normal Flow:	**1.0 Create and Personalize User Profile** 1. User navigates to "My Profile". 2. System prompts the user to enter profile details (display name, bio, profile picture). 3. User enters the required details. (see EX-1) 4. System saves and updates the user profile. 5. The profile is displayed on the user's page.
Alternative Flows:	**1.1 Modify Profile Details** 1. User revisits "My Profile" to update details. 2. System allows changes to profile picture, bio, and personal settings
Exceptions:	**EX-1: Invalid file format for profile picture** 1. System rejects the upload and prompts the user to select a supported format.
Business Rules:	**BR-1:** Users must provide a unique display name. **BR-2:** Profile updates should be saved immediately.

Assumptions:	1. Users will regularly update their profile to reflect their activity.

UC-1.10: Edit Profile
In this case, a user wants to modify their profile details. They can update their username, profile picture, or password. Once the changes are made, the system ensures the updated information reflects in the user's profile.

Table 13: UC-1.10 Edit Profile

Actors:	User
Trigger:	User selects edit profile option on dashboard page.
Preconditions:	**PRE-1:** User must be logged in.
Postconditions:	**POST-1:** System updates the User information
Normal Flow:	1.0 Edit User Profile 1. User selects edit profile option in dashboard page. 2. User updates phone number if required. 3. User updates username if required. (See 1.0. E1) 4. User updates picture if required. 5. User updates password if required. (See 1.0. E2) 6. The system updates the user information.

Alternative Flows:	1.1 Password Updates 1. User enters the new password. 2. Users re-enter the password for confirmation. 3. System checks if the two passwords match (see 1.1 E1). 4. If they match, the system updates the password and use case resumes at step 6 of the normal flow.
Exceptions:	**EX-1: 1.0. E1 Username already registered** 1. System notifies the user that the chosen username has already taken. 2a. If the user provides an alternate username, the use case resumes from step 3 of the normal flow. 2b. Else if the user decides to exit the registration process, the use case terminates. **EX-2: 1.1. E2 Password Strength Insufficient** 1. System informs user that the password doesn't meet the required criteria. 2a. If the user modifies the password according to guidelines, the use case resumes at step 5 of the normal flow. 2b. Else if the user cancels the password update process, the use case resumes at step 6 of the normal flow. **EX-3: 1.1. E1 Password and Confirm Password Mismatch** 1. System notifies User that the entered passwords don't match. 2a. If a user re-enters matching passwords, use case resumes at step 6 of the normal flow. 2b. Else if the user cancels the password update process, the use case resumes at step 6 of the normal flow.
Business Rules:	**BR-1:** Only one account can be registered for a given username. **BR-2:** The password must be a strong password with both alphabet and digits
Assumptions:	1. Users know that edit profile will change the previous information. 2. An active internet connection is available to the user.

UC-1.11: Follow Creators and View Updates

In this case, a user follows their favorite podcast creators to receive updates on new episodes and activity.

Table 14: UC-1.11 Follow Creators and View Updates

Actors:	Viewer
Trigger:	User selects the "Follow" button on a creator's profile.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** The creator's profile must be publicly available.
Postconditions:	**POST-1:** The user starts following the creator. **POST-2:** Updates from the creator appear in the user's feed.
Normal Flow:	**1.0 Follow a Creator** 1. User visits a creator's profile. 2. User clicks "Follow". 3. System updates the user's following list. 4. System adds creator's updates to the user's feed.
Alternative Flows:	**1.1 Unfollow a Creator** 1. User visits a followed creator's profile. 2. User clicks "Unfollow". 3. System removes the creator from the following list.
Exceptions:	**EX-1: Creator's account is deleted or deactivated** 1. System removes the creator from all users' following lists.
Business Rules:	**BR-1:** Users can follow or unfollow creators at any time.

Assumptions:	1. Creators regularly update their content.

UC-1.12: View Activity Feed of Followed Creators

In this case, a user views a feed displaying updates from followed creators.

Table 15: UC-1.12 View Activity Feed of Followed Creators

Actors:	Viewer
Trigger:	User navigates to "Activity Feed" on home page.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** User must follow at least one creator.
Postconditions:	**POST-1:** System displays updates from followed creators.
Normal Flow:	**1.0 View Activity Feed** 1. The user selects "Activity Feed". 2. System loads recent updates from followed creators. 3. User browses through new content, comments, and interactions.
Alternative Flows:	**N/A**
Exceptions:	**EX-1: No activity available** 1. System informs the user and suggests following more creators.

Business Rules:	**BR-1:** Activity feed updates should be in real-time.
Assumptions:	1. Creators actively post updates.

3.2.2.2 Module 2: Voice Cloning

UC-2.1: Voice Cloning Setup

In this case, a user records, uploads, or selects a voice sample for AI voice cloning. The system processes the sample, generates a cloned voice, and allows the user to preview and save it. If unsatisfied, the user can re-record, re-upload, or choose from AI-generated voices for customization.

Table 16: UC-2.1 Voice Cloning Setup

Actors:	Creator
Trigger:	User selects the "Voice Cloning" option from the settings or AI customization menu.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** System must support voice recording and AI processing.
Postconditions:	**POST-1:** A voice sample is recorded, uploaded, or selected. **POST-2:** The system processes and saves the voice model.

Normal Flow:	**1.0 Voice Cloning Setup** 1. User selects "Voice Cloning" from settings. 2. System prompts the user to record or upload a voice sample. 3. User records or uploads a sample. (See 1.0. E1, 1.0. E2) 4. System processes the sample for cloning. 5. User previews the generated AI voice. 6. If satisfied, the user confirms and saves the AI-generated voice.
Alternative Flows:	**1.1 Modify Cloned Voice** 1. If the user is not satisfied, they can re-record or re-upload a sample. (See 1.1. E1) 2. User can select a voice from the AI-generated voice library instead of recording their own.
Exceptions:	**EX-1: 1.0. E1: Poor-quality recording** 1. System prompts the user to retry. **EX-2: 1.0. E2: Unsupported file format for upload** 1. System notifies the user and requests a valid format. **EX-3: 1.1. E1: User rejects generated voice.** 1. System allows re-recording, re-uploading, or selection from the AI-generated voice library.
Business Rules:	**BR-1:** Voice cloning should meet quality standards before being saved. **BR-2:** Users should be able to re-record or modify their cloned voice.
Assumptions:	1. User has a functioning microphone if recording a voice sample. 2. The system has AI capabilities to process and generate voices.

3.2.2.3 Module 3: Knowledge Base Setup

UC-3.1: Knowledge Base Setup

In this case, a user uploads structured or unstructured data into the AI system's knowledge base. The system processes, cleans, and organizes the data into meaningful chunks for better AI retrieval. Metadata is attached to ensure accurate responses, and the final processed data is stored in a vector database.

Table 17: UC-3.1 Knowledge Base Setup

Actors:	Creator, System
Trigger:	User selects the "Upload Knowledge Base" option from the settings or AI customization menu.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** User must have valid data sources in supported formats (PDFs, URLs, CSV, JSON). **PRE-3:** System must have available storage for data processing.
Postconditions:	**POST-1:** Uploaded data is processed and stored. **POST-2:** Data is split into meaningful chunks with metadata attached. **POST-3:** System generates vector embeddings for efficient retrieval.
Normal Flow:	**1.0 Knowledge Base Upload** 1. User selects "Upload Knowledge Base". 2. System prompts the user to choose a data source (PDF, URL, CSV, JSON). 3. User uploads the file or enters a URL. (See 1.0. E1) 4. System scans data for formatting issues. 5. System cleans and preprocesses data. 6. Data is split into structured chunks with metadata. 7. System generates vector embeddings for AI retrieval. 8. Processed data is stored in the vector database. 9. System confirms successful upload.

Alternative Flows:	**1.1 Metadata Enhancement** 1. System detects missing metadata. (See 1.1. E1) 2. System suggests default metadata values. 3. User manually updates metadata if necessary.
Exceptions:	**EX-1: 1.0. E1: File format unsupported** 1. System notifies the user and requests a valid format. **EX-2: 1.0. E2: URL is unreachable.** 2. System notifies the user and allows them to enter another URL. **EX-3: 1.1. E1: Missing metadata** 3. System prompts the user to add metadata before proceeding.
Business Rules:	**BR-1:** Only supported file formats (PDF, URL, CSV, JSON) can be processed. **BR-2:** Data must be cleaned and structured before storage. **BR-3:** Metadata is required for AI retrieval accuracy.
Assumptions:	1. User uploads valid and well-formatted data. 2. System can process large datasets efficiently.

3.2.2.4 Module 5: Agent Customization

UC-4.1: AI Agent Customization

In this case, a user customizes their AI agent before starting a podcast session. The system allows users to configure the agent's name, voice, and avatar to match their podcast style.

Table 18: UC-4.1 AI Agent Customization

Actors:	Creator, System

Trigger:	User selects "Customize AI Agent" after starting a new podcast session.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** AI customization feature must be enabled.
Postconditions:	**POST-1:** AI agent is configured with a selected name, voice, and avatar. **POST-2:** The customized agent is saved for future sessions.
Normal Flow:	**1.0 AI Agent Customization** 1. User selects "Customize AI Agent". 2. System prompts the user to enter a name for the AI agent. (See 1.0. E1) 3. User uploads or selects an avatar for the AI agent. (See 1.0. E2) 4. System applies selected settings. 5. User confirms the customization and saves changes.
Alternative Flows:	**1.1 Skip Customization** 1. User chooses to proceed with default AI settings. 2. System loads default AI agent configuration.
Exceptions:	**EX-1: 1.0. E1: AI name not provided.** 1. System prompts the user to enter a name. **EX-2: 1.0. E2: Invalid avatar file format** 1. System requests a valid file format.
Business Rules:	**BR-1:** AI agent customization must be completed before recording begins. **BR-2:** Users can modify AI settings before each session.
Assumptions:	1. System provides a variety of AI voice and avatar options. 2. Users have permission to customize AI settings.

3.2.2.5 Module 6: Podcast Audio Creation

UC-5.1: Record Podcast Session

In this case, a user records a podcast session with a customized AI agent. The system captures real-time audio, records AI-generated responses, and generates live transcripts. Users can pause, stop, or resume recording at any time, ensuring flexibility in content creation.

Table 19: UC-5.1 Record Podcast Session

Actors:	Creator
Trigger:	User clicks "Start Recording" after customizing the AI agent.
Preconditions:	**PRE-1:** User has configured AI agent settings. **PRE-2:** User has selected AI Host or AI Guest. **PRE-3:** User is on the recording page.
Postconditions:	**POST-1:** A recording session is started. **POST-2:** Audio is recorded in real-time. **POST-3:** AI-generated responses are recorded in the same file. **POST-4:** Transcript is generated alongside the recording.
Normal Flow:	**1.0 Record Podcast Session** 1. User clicks Start Recording. 2. System records the user's voice and AI-generated responses in real-time. (See 1.0. E1) 3. System generates a real-time transcript of the conversation. 4. User has the option to pause or stop recording at any time. 5. When the recording is complete, the system displays a confirmation message.
Alternative Flows:	**1.1 Pause and Resume Recording** 1. If the user pauses the recording, the system allows resuming later.

Exceptions:	**EX-1: 1.0. E1: User's microphone access is denied.** 1. System prompts for permissions. **EX-2: 1.0. E2: Storage is full** 1. System warns user and suggests freeing space. **EX-3: 1.0. E3: AI response failure** 1. System retries generating an AI response.
Business Rules:	**BR-1:** AI-generated voice must be synchronized with user input. **BR-2:** The system must auto-save progress at intervals.
Assumptions:	1. User's device has a working microphone. 2. AI voice models are pre-loaded before recording.

UC-5.2: Edit, Review, and Export Podcast

In this case, a user edits a recorded podcast using trimming, splicing, and rearranging tools, previews the final version for quality assurance, and then exports the finalized podcast in a preferred file format.

Table 20: UC-5.2 Edit, Review, and Export Podcast

Actors:	Creator
Trigger:	The user selects "Edit Podcast" after completing a recording session.
Preconditions:	**PRE-1:** A recorded podcast session exists. **PRE-2:** User has completed the recording.
Postconditions:	**POST-1:** The user edits the audio using trimming and splicing options. **POST-2:** The user previews the final podcast before exporting. **POST-3:** The podcast is exported and saved in a preferred format (MP3/WAV).

Normal Flow:	**1.0 Edit, Preview, and Export Podcast** 1. User selects “Edit Podcast”. 2. System loads the audio editing interface. (See 1.0. E1 if loading fails) 3. User edits the podcast by trimming, splicing, and rearranging audio clips. 4. User listens to a preview of the edited podcast. 5. User confirms final version and selects “Export & Save”. 6. System prompts the user to choose a file format (MP3/WAV). 7. System converts and exports the file in the selected format. 8. User confirms saving, and the podcast is stored locally or in cloud storage.
Alternative Flows:	**1.1 Return to Editing** 1. If the user finds errors during preview, they can return to editing before finalizing.
Exceptions:	**EX-1: 1.0.E1: Editing tools fail to load.** 1. System retries loading. **EX-2: 1.0. E2: Audio playback failure during preview** 1. System retries playing the podcast. **EX-3: 1.0. E3: Export failure due to insufficient storage** 1. System prompts the user to free up space before proceeding. **EX-4: 1.0. E4: Unsupported file format selection** 1. System requests the user to select a valid format.
Business Rules:	**BR-1:** Editing should not degrade audio quality. **BR-2:** Users must preview the podcast before saving. **BR-3:** System must auto-save edits periodically to prevent data loss. **BR-4:** Exported files must meet minimum quality standards.
Assumptions:	1. User has basic knowledge of audio editing tools. 2. Users have enough storage to export the file.

3.2.2.6 Module 7: AI Host & Module 8 : AI Guest

UC-6.1: AI Host & AI Guest Interaction

In this use case, a user initiates a podcast session with either an AI Host or AI Guest. The AI Host manages the flow of the podcast, introduces episodes, ensures smooth transitions, and summarizes discussions. The AI Guest responds dynamically to user questions, shares relevant stories, and engages in discussions.

Table 21: UC-6.1 AI Host & AI Guest Interaction

Actors:	Creator, System
Trigger:	User starts a podcast session with AI Host or AI Guest.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** User must have selected AI Host or AI Guest. **PRE-3:** Knowledge base and voice model must be pre-configured.
Postconditions:	**POST-1:** AI Host/Guest interacts with the user in a structured podcast session. **POST-2:** AI-generated responses and podcast flow are recorded. **POST-3:** AI Host provides episode summary upon completion.
Normal Flow:	**1.0 AI Podcast Session** 1. User selects "Start Podcast" and chooses AI Host or AI Guest. 2. System loads pre-configured AI settings, voice model, and knowledge base. 3. AI Host introduces the episode and sets the discussion agenda. (see 1.0. E1) 4. AI Guest engages in conversation by responding to queries and sharing relevant insights. (see 1.0. E2) 5. AI Host facilitates smooth transitions between topics. 6. AI Host provides summary prompts to recap key points. 7. The session concludes, and the recording is saved.

Alternative Flows:	**N/A**
Exceptions:	**EX-1: 1.0. E1: AI Host fails to load settings** 1. System prompts user to reconfigure AI preferences. **EX-2: 1.0. E2: AI Guest fails to generate responses** 1. System retries or defaults to a generic knowledge base.
Business Rules:	**BR-1:** AI Host must ensure structured podcast flow. **BR-2:** AI Guest should generate contextually relevant responses. **BR-3:** AI must provide a coherent and engaging podcast experience.
Assumptions:	1. AI models are trained to deliver engaging and structured conversations. 2. The system has sufficient processing power to generate real-time AI responses.

3.2.2.7 Module 9: Podcast Video Creation & Publishing

UC-7.1: Create and Export Video Podcast

In this case, a creator customizes and finalizes a video podcast by selecting avatars, adding visual elements, previewing the content, and exporting the final video.

Table 22: UC-7.1 Create and Export Video Podcast

Actors:	Creator, System
Trigger:	The creator initiates the video creation process after completing the podcast audio.

Preconditions:	**PRE-1:** The podcast audio is available. **PRE-2:** System-provided avatars and an option to upload a custom image are accessible. **PRE-3:** System-provided visuals and an option to upload a custom visuals are accessible.
Postconditions:	**POST-1:** The video podcast is generated with assigned avatars and visual elements. **POST-2:** The creator previews and approves the final video. **POST-3:** The system exports the video in MP4 format.
Normal Flow:	**1.0 Create and Export Video Podcast** 1. The creator selects "Create Video Podcast". 2. System displays options for avatars and visual elements. 3. Avatar Selection: o Creator selects AI-generated avatars for speakers (see 1.0. E1). o If desired, the creator uploads a custom avatar (see 1.0. E2).
	4. Visual Elements Selection: o Creator selects a background, overlays, and other effects (see 1.0. E3). o If desired, the creator uploads a custom visual (see 1.0. E4). 5. System applies the selected avatars and visuals to the video. 6. Preview & Export: o The creator selects "Preview Video". o System plays the video for review (see 1.0. E5). o The creator confirms and selects "Export". 7. System processes and exports the video in MP4 format.
Alternative Flows:	**1.1 Modify Selections Before Finalizing** 1. The creator decides to change avatars or visuals before exporting. 2. The system allows reselection or re-upload.

Exceptions:	**EX-1: 1.0. E1: User does not select an avatar** 1. System prompts user to choose or upload an avatar. **EX-2: 1.0. E2: Upload failure** 1. System notifies the user and requests a valid format. **EX-3: 1.0. E3: User does not select visuals** 1. System prompts user to choose or upload visuals. **EX-4: 1.0. E4: Upload failure** 1. System notifies the user and requests a valid format. **EX-5: 1.0. E5: Video playback failure** 1. System retries playing.
Business Rules:	**BR-1:** Users must assign an avatar to each speaker before proceeding. **BR-2:** At least one visual element must be selected before exporting. **BR-3:** Users must preview before exporting.
Assumptions:	1. The system provides a library of predefined avatars and visual elements. 2. The system ensures smooth video playback without lag. 3. Users have enough storage for exporting the final video.

UC-7.2: Request Podcast Summary

In this use case, a user requests an AI-generated summary of a podcast episode. The system processes the episode's content, identifies key topics, and generates a structured text-based summary.

. Table 23: UC-7.2 Request Podcast Summary

Actors:	Viewer
Trigger:	User selects "Request Summary" on a podcast episode.
Preconditions:	**PRE-1:** The podcast episode must exist in the system. **PRE-2:** AI-powered summarization feature must be enabled.

Postconditions:	**POST-1:** System processes and displays a text-based summary. **POST-2:** Summary includes main topics and key highlights.
Normal Flow:	**1.0 Request Podcast Summary** 1. User selects a podcast episode. 2. User clicks the "Request Summary" option. 3. System processes the episode's audio content. 4. System extracts key topics and generates a summary. 5. System displays a structured summary with key points.
Alternative Flows:	**1.1 Customize Summary Format** 1. User chooses to generate a detailed or brief summary. 2. System adjusts the output based on user selection.
Exceptions:	**EX-1: 1.0. E1: Podcast episode not found** 1. System notifies the user and suggests selecting another episode. **EX-2: 1.0. E2: AI fails to generate a summary** 1. System retries or provides a generic summary.
Business Rules:	**BR-1:** Summaries must be factually accurate and concise. **BR-2:** AI should ensure key points are highlighted clearly.
Assumptions:	1. Users prefer quick, AI-generated summaries for reference.

UC-7.3: Search for Specific Information in a Podcast Summary

In this use case, a user searches for specific topics or keywords within a podcast summary. The system highlights relevant parts of the summary and provides timestamped references for easy navigation.

Table 24: UC-7.3 Search for Specific Information in a Podcast Summary

Actors:	Viewer

Trigger:	User enters a keyword or topic in the summary search bar.
Preconditions:	**PRE-1:** A podcast summary must be generated and available. **PRE-2:** System must support keyword-based searching.
Postconditions:	**POST-1:** System filters and highlights relevant summary content. **POST-2:** System provides timestamped references to key moments.
Normal Flow:	**1.0 Search for Topics in a Summary** 1. User navigates to a podcast summary. 2. User enters a keyword or topic in the search bar. 3. System scans the summary for relevant content. 4. System highlights matching sections. 5. System provides timestamped references for easy navigation.
Alternative Flows:	**1.1 Filter Search Results** 1. User applies filters (e.g., most relevant, newest content). 2. System adjusts the results accordingly.
Exceptions:	**EX-1: No matching content found.** 1. System suggests related topics or keywords.
Business Rules:	**BR-1:** Search results must be accurate and relevant. **BR-2:** Users should be able to navigate directly to timestamped moments.
Assumptions:	1. Users search summaries to quickly find specific information.

UC-7.4: Rate Podcast Session

In this use case, a creator rates a completed podcast session to provide feedback on its quality. The system records the rating, updates the overall session rating, and ensures that each session is rated only once.

Table 25: UC-7.4 Rate Podcasts Session

Actors:	Creator
Trigger:	The user decides to rate a completed podcast session.
Preconditions:	**PRE-1:** The podcast session must be completed. **PRE-2:** The user must be logged in.
Postconditions:	**POST-1:** The rating is recorded and associated with the podcast session. **POST-2:** The system updates the session's overall rating.
Normal Flow:	**1.0 Rate the session** 1. The user navigates to the completed podcast session. 2. The system displays the rating options (e.g., 1 to 5 stars). 3. The user selects a rating. 4. The system saves the rating and updates the session's rating statistics. 5. A confirmation message is displayed.
Alternative Flows:	**1.1 User Cancels Rating** 1. The user chooses not to rate. 2. The system does not save any rating.
Exceptions:	**EX-1: Session Not Found** If the session is deleted or unavailable, the system notifies the user. **EX-2: Duplicate Rating Attempt** If the user tries to rate the same session twice, the system notifies them that the rating was already submitted.

Business Rules:	**BR-1:** Users can rate only sessions they have hosted. **BR-2:** Ratings should be final once submitted.
Assumptions:	1. Users will rate sessions to provide meaningful feedback.

3.2.2.8 Module 10: Subscription and Payment

UC-8.1: Subscription Management

In this use case, a user manages their subscription, including subscribing, canceling, or renewing their plan. The system processes payments securely and updates the user's access accordingly.

Table 26: UC-8.1 Subscription Management

Actors:	Creator, System
Trigger:	User navigates to the **Subscription & Payments** section.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** System must have an active payment gateway.
Postconditions:	**POST-1:** Subscription is updated based on user actions. **POST-2:** User gains or loses access to premium features accordingly.
Normal Flow:	**1.0 Subscription Management** 1. User navigates to "Subscription & Payments". 2. System displays available subscription plans. 3. User selects a plan and enters payment details. 4. System processes payment and updates the subscription. 5. User receives confirmation notification.

Alternative Flows:	**1.1 Modify Subscription** 1. Users cancel their subscription. (See 1.1. E1) 2. System stops auto-renewal and schedules access termination. 3. User updates payment method. (See 1.1. E2) 4. System verifies the added details and applies changes.
Exceptions:	**EX-1: 1.1. E1: Payment fails.** 1. System prompts user to retry or update payment method. **EX-2: 1.1. E2: Subscription renewal fails.** 1. System notifies the user and requests manual action.
Business Rules:	**BR-1:** Users must have an active subscription for premium features. **BR-2:** Subscription auto-renewal settings should be user-configurable.
Assumptions:	1. System supports secure payment processing.

UC-8.2: Free Content Access Limit
In this case, a user accesses free content until they reach the predetermined limit, after which they are prompted to subscribe.

Table 27: UC-8.2 Free Content Access Limit

Actors:	Creator, System
Trigger:	User access free content (e.g., record limited number of podcasts).
Preconditions:	**PRE-1:** User has not exceeded the free content limit.

Postconditions:	**POST-1:** User either continues listening or is prompted to subscribe.
Normal Flow:	**1.0 Free Content Access Limit** 1. User records a free podcast. 2. System tracks free content usage. (see EX-1) 3. If limit is not reached, user continues listening. 4. If limit is reached, system displays a subscription prompt.
Alternative Flows:	**1.1 Subscription Prompt** 1. User ignores the prompt and exits. 2. User subscribes and gains unlimited access.
Exceptions:	**EX-1: Content playback failure** 1. System retries loading.
Business Rules:	**BR-1:** Free users have a limited number of content views.
Assumptions:	1. System tracks user progress accurately.

UC-8.3: Subscription Expiry Reminder

In this case, the system **notifies users** before their subscription expires and provides renewal options.

Table 28: UC-8.3 Subscription Expiry Reminder

Actors:	Creator, System

Trigger:	System detects an upcoming subscription expiration.
Preconditions:	**PRE-1:** User has an active subscription.
Postconditions:	**POST-1:** User is reminded about the expiration. **POST-2:** User can renew or cancel the subscription.
Normal Flow:	**1.0 Subscription Expiry Reminder** 1. System detects expiration date approaching. 2. System sends reminder notifications. (see EX-1) 3. User chooses to renew, cancel, or ignore. 4. System updates the subscription based on user action.
Alternative Flows:	**1.1 Renewal Process** 1. User renews the subscription before expiry. 2. System processes payment and extends access.
Exceptions:	**EX-1: Payment failure during renewal** 1. System prompts the user to update payment details.
Business Rules:	**BR-1:** Users must receive reminders before expiration.
Assumptions:	1. System has a working notification service.

3.2.2.9 Module 11: FAQs Chatbot

UC-9.1: Ask a Question from FAQs Chatbot

In this use case, a user interacts with the FAQs chatbot to ask questions related to podcasting, AI customization, or troubleshooting. The chatbot searches the FAQs database and provides the most relevant answers.

Table 29: UC-9.1 Ask a Question from FAQs Chatbot

Actors:	User
Trigger:	User selects the FAQs chatbot option from the left panel and enters a query.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** The FAQs database must be available and populated with relevant content.
Postconditions:	**POST-1:** The chatbot retrieves and displays an answer from the FAQs database. **POST-2:** The system logs the interaction to improve future chatbot responses.
Normal Flow:	**1.0 Ask a Question from FAQs Chatbot** 1. User opens the FAQs chatbot from the menu. 2. User types a question related to podcasting, AI customization, or troubleshooting. 3. System processes the query and searches the FAQs database. 4. System retrieves the most relevant response. (see 1.0. E1) 5. Chatbot displays the answer in a structured format. 6. User can either close the chatbot or ask another question.
Alternative Flows:	**1.1 No Exact Match Found** 1. If no exact match is found, the chatbot suggests related FAQs. (see 1.1. E1) 2. User selects the most relevant suggestion.

Exceptions:	**EX-1: 1.0. E1: No data available in FAQs database** 1. System informs the user that no answer is available. 2. System provides an option to contact support. **EX-2: 1.1. E1: User enters an unclear or incomplete query** 1. Chatbot asks for clarification.
Business Rules:	**BR-1:** Chatbot should only display verified responses from the FAQs database. **BR-2:** Chatbot should continuously improve responses based on user interactions.
Assumptions:	1. Users will ask relevant questions. 2. The FAQs database is regularly updated.

UC-9.2: View Suggested FAQs

In this use case, a user browses commonly asked questions suggested by chatbot before query.

Table 30: UC-9.2 View Suggested FAQs

Actors:	User
Trigger:	User opens the FAQs chatbot without entering a query.
Preconditions:	**PRE-1:** The FAQs database must contain commonly asked questions.
Postconditions:	**POST-1:** User is presented with a list of suggested FAQs. **POST-2:** The chatbot learns which FAQs are most frequently accessed.

Normal Flow:	**1.0 View Suggested FAQs** 1. User opens the FAQs chatbot. 2. System displays a list of commonly asked questions. 3. User selects a relevant FAQ. 4. System retrieves and displays the corresponding answer.
Alternative Flows:	**1.1 User Chooses to Enter a Query Instead** 1. User dismisses the suggested FAQs. 2. System allows the user to enter a custom query.
Exceptions:	**EX-1: No frequently asked questions available** 1. System informs the user and provides a manual search option.
Business Rules:	**BR-1:** The chatbot must maintain a list of most-accessed FAQs. **BR-2:** FAQs should be displayed based on popularity and relevance.
Assumptions:	1. Users may prefer browsing FAQs before typing a query.

3.2.2.10 Module 12: Audience Engagement and Communication

UC-10.1: Receive Notifications for Audience Engagement

In this use case, a creator receives notifications when their content is liked, shared, commented on, or followed. The system ensures updates, allowing creators to engage with their audience effectively.

Table 31: UC-10.1 Receive Notifications for Audience Engagement

Actors:	Creator

Trigger:	The audience interacts with the creator's content (like, comment, share, follow).
Preconditions:	**PRE-1:** Creator must have at least one podcast published. **PRE-2:** System must support real-time notifications.
Postconditions:	**POST-1:** Creator is notified of engagement actions. **POST-2:** Notifications are stored in the creator's notification panel.
Normal Flow:	**1.0 Receive Engagement Notifications** 1. The audience interacts with the creator's podcast (like, comment, share, follow). 2. System generates a real-time notification. 3. Creator receives a push or in-app notification. (see EX-1) 4. Creator clicks the notification to view details. 5. System directs the creator to the relevant content.
Alternative Flows:	**1.1 View Notification History** 1. Creator navigates to the "Notifications" section. 2. System displays a history of all past notifications.
Exceptions:	**EX-1: System fails to send real-time notifications.** 1. System retries sending notifications. 2. If still unsuccessful, system logs the failure for later delivery.
Business Rules:	**BR-1:** Notifications should be sent in real-time. **BR-2:** Users can configure notification preferences in settings.
Assumptions:	1. Creators actively check their notifications to engage with their audience.

UC-10.2: User-to-User Messaging and Calls

In this use case, users communicate through in-platform messaging or voice/video calls, fostering collaboration and engagement.

Table 32: UC-10.2 User-to-User Messaging and Calls

.

Actors:	Viewer
Trigger:	User initiates a message or call with another user.
Preconditions:	**PRE-1:** Users must be logged in. **PRE-2:** Messaging and calling features must be enabled.
Postconditions:	**POST-1:** Message or call request is sent and received. **POST-2:** Conversation history is saved.
Normal Flow:	**1.0 Sending a Message** 1. User navigates to the messaging section. 2. User searches for another user and selects them. 3. User types a message and clicks "Send". 4. System delivers the message in real time. 5. Recipient receives a notification and responds. **1.1 Start a Voice/Video Call** 1. User selects a contact from the messaging section. 2. User clicks "Call" to initiate voice or video communication. 3. System sends a call request to the recipient. (See 1.1. E1) 4. If accepted, the call is established. 5. Users communicate and end the call when done.
Alternative Flows:	**1.2 Mute or Block a User** 1. User selects a conversation. 2. User chooses to mute or block the sender. 3. System updates preferences accordingly.

Exceptions:	**EX-1: 1.0. E1: Message fails to send.** 1. System retries sending or notify the user of failure. **EX-2: 1.1. E1: Call request is declined or unanswered.** 1. System informs the caller and allows them to retry.
Business Rules:	**BR-1:** Users can only message or call those who have enabled the feature. **BR-2:** Blocked users cannot send messages or call.
Assumptions:	1. Users actively use messaging and calling for engagement.

UC-10.3: Engage with Podcasts (Like, Comment, Share)

In this use case, a user interacts with podcasts by liking, commenting, or sharing them.

Table 33: UC-10.3 Engage with Podcasts (Like, Comment, Share)

Actors:	Viewer
Trigger:	User selects an engagement option (Like, Comment, Share) on a podcast.
Preconditions:	**PRE-1:** User must be logged in. **PRE-2:** The podcast must be publicly available.
Postconditions:	**POST-1:** Engagement action is recorded in the system. **POST-2:** The creator is notified of engagement.

Normal Flow:	**1.0 Engage with a Podcast** 1. User plays a podcast episode. 2. User selects an engagement action: o Like: System records the like. o Comment: User enters a comment and submits. (see EX-1) o Share: User selects a platform to share the podcast.
Alternative Flows:	**1.1 Delete a Comment** 1. Users navigate their comment on a podcast. 2. User selects "Delete". 3. The system removes the comment.
Exceptions:	**EX-1: Comment contains restricted content.** 1. System flags and removes the comment.
Business Rules:	**BR-1:** Users can engage with public podcasts only. **BR-2:** Comments must adhere to community guidelines.
Assumptions:	1. Users actively engage with content.

3.2.2.11 Module 13: Admin Panel and Analytics

UC-11.1: Manage User Accounts

In this use case, an admin can view search, deactivate, reactivate, or delete user accounts.

Table 34: UC-11.1 Manage User Accounts

Actors:	Admin

Trigger:	Admin navigates to the user management section.
Preconditions:	**PRE-1:** Admin must be authenticated and authorized.
Postconditions:	**POST-1:** User status is updated in the system.
Normal Flow:	**1.0 Admin edits user profile** 1. Admin accesses the user management section. 2. Admin searches for a specific user using filters. 3. System displays user details. 4. Admin selects an action (deactivate, reactivate, delete). 5. System updates user status and logs the action.
Alternative Flows:	**1.1 User doesnot exists** If the user does not exist, the system displays an error message.
Exceptions:	**EX-1:** If admin tries to delete a user with active subscriptions, the system prevents deletion and prompts an action (cancel subscriptions first or transfer ownership).
Business Rules:	**BR-1:** Only authorized admins can modify user accounts.
Assumptions:	1. Admin has the necessary permissions to manage user accounts.

UC-11.2: Review Reported Content

In this use case , admin reviews podcasts and comments flagged for moderation.

Table 35: UC-11.2 Review Reported Content

Actors:	Admin
Trigger:	The admin selects the "Reported Content" section from the admin panel.
Preconditions:	**PRE-1:** The admin must be logged in with moderation privileges. At least one podcast or comment must be flagged.
Postconditions:	**POST-1:** The flagged content is either approved or removed.
Normal Flow:	1. Admin navigates to the "Reported Content" section. 2. Admin reviews a reported podcast or comment. 3. Admin takes one of the following actions: - Approve content (dismiss report) - Remove content - Issue a warning to the user. 4. System updates content status and logs the action.
Alternative Flows:	If no matching user is found, the system displays an appropriate message.
Exceptions:	**EX-1:** If the reported content has already been resolved by another admin, the system notifies the admin.
Business Rules:	**BR-1:** Only admins with moderation rights can review flagged content.

Assumptions:	1. Admin has an active internet connection and the necessary permissions.

UC-11.3: View Platform Analytics

In this case , views live statistics related to user activity and content engagement.

Table 36: UC-11.3 View Platform Analytics

Actors:	Admin
Trigger:	The admin selects the "Analytics Dashboard" option from the admin panel.
Preconditions:	**PRE-1:** The admin must be logged in with analytics access.
Postconditions:	**POST-1:** The admin view real-time analytics and trends.
Normal Flow:	1. Admin navigates to the “Analytics Dashboard” section. 2. System retrieves and displayes metrics , including : o Active users o Total Podcasts created o Trending content o Engagement heatmaps o User demographics 3. Admin applies filters for specific trends.
Alternative Flows:	If the admin selects custom date ranges, the system recalculates and updates the analytics display accordingly

Exceptions:	**EX-1:** If analytics data is unavailable due to a system error, the system displays an error message.
Business Rules:	**BR-1:** Analytics data should be updated in real-time or near real-time.
Assumptions:	1. System logs and metrics are available and up to date.

UC-11.4: Manage Subscriptions & Payments

The admin tracks active subscriptions, processes refunds, and handles payment disputes.

Table 37: UC-11.4 Manage Subscriptions & Payments

Actors:	Admin
Trigger:	The admin selects the "Subscription Management" section from the admin panel.
Preconditions:	**PRE-1:** The admin must be logged in with finance privileges.
Postconditions:	**POST-1:** Subscription status, refunds, or disputes are processed.
Normal Flow:	1. Admin navigates to "Subscription Management." 2. Admin searches for a user or subscription. 3. Admin selects an action: o Track active subscriptions o Issue a refund o Manage payment disputes 4. System updates the subscription or payment status.

Alternative Flows:	If the refund request is pending further approval, the admin marks it for later review.
Exceptions:	**EX-1:** If a refund is not eligible, the system informs the admin.
Business Rules:	**BR-1:** Refund policies must align with platform terms of service.
Assumptions:	1. Payment gateways and transaction history are accessible.

UC-11.5: Generate User Engagement Reports

The admin generates reports on user engagement, podcast performance, and trending topics.

Table 38: UC-11.5 Generate User Engagement Reports

Actors:	Admin
Trigger:	The admin selects the "Generate Reports" option from the analytics panel.
Preconditions:	**PRE-1:** The admin must be logged in with analytics privileges.
Postconditions:	**POST-1:** The requested report is generated and available for download.

Normal Flow:	1. Admin navigates to the "Generate Reports" section. 2. Admin selects report type: o User engagement trends o Podcast performance o Trending topics 3. Admin specifies filters and date range. 4. System processes and generates the report. 5. Admin downloads the report.
Alternative Flows:	If the admin schedules a report for auto-generation, the system sends it via email instead of downloading immediately.
Exceptions:	**EX-1:** If no data matches the filters, the system displays a message.
Business Rules:	**BR-1:** Reports should reflect the latest available data.
Assumptions:	1. Reporting tools are functional, and data is up to date.

3.2.3 Event Response Table

3.2.3.1 ERT-1: User Login

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
1	User logs in	Login credentials (email/password or social login)	User attempts authentication	Invalid credentials (incorrect email/password)	Successfully authenticates and loads user dashboard	Displays an error message prompting the user to try again	User session becomes active, allowing access to the system

3.2.3.2 ERT-2: Profile Update

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
2	User edits profile info	Name, bio, profile picture, contact details	User submits profile changes	Invalid input format (unsupported characters, empty fields)	Successfully updates the database and reflects changes in the profile	Rejects update and prompts the user to correct input	User profile is successfully updated and displayed

3.2.3.3 ERT-3: Voice Cloning

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
3	User uploads voice sample	Audio file (WAV, MP3)	User submits a valid sample	Poor audio quality (background noise, distortion)	Processes the sample, generates an AI voice, and saves it for user use	Requests a new upload with clearer audio	AI-generated voice is successfully available for use in applications

3.2.3.4 ERT-4: Knowledge Base Upload

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
4	User uploads knowledge document	File (PDF, URL, CSV, JSON)	Data processing in progress	File unreadable (corrupted, unsupported format)	Stores and indexes knowledge data, making it searchable	Requests re-upload with a supported format	Knowledge is successfully stored and indexed for future queries

3.2.3.5 ERT-5: Chatbot Query Processing

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
5	User submits a chatbot query	Query text (user question)	Query processing in progress	No relevant data available	Retrieves the best matching answer and displays it to the user	Informs the user that no relevant answer is available	FAQ response is displayed if a match is found, otherwise user is prompted for clarification

3.2.3.6 ERT-6: Terminate Test Section

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
6	Terminate Section When Time Over	Test section duration (time limit)	Timer for the section expires	N/A	The system automatically ends the section, saves responses, and proceeds to the next section	N/A	The next section starts automatically on the screen

3.2.3.7 ERT-7: AI Host Podcast Management

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
7	AI Host starts podcast	Podcast metadata (title, topic, participants)	AI Host initialized	System failure (crash, unavailable resources)	Successfully introduces and manages the podcast session	Retries starting the podcast session automatically	AI-driven session is in progress and being recorded

3.2.3.8 ERT-8: AI Guest Response

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
8	AI Guest receives a question	Query text (spoken or typed question)	AI processes input	No relevant data	Generates an appropriate AI-driven response with context	Provides an alternative response or asks for clarification	AI Guest successfully responds to the user with relevant information

3.2.3.9 ERT-9: Podcast Recording

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
9	User records podcast	Audio stream (live microphone input)	Audio recording active	Microphone issue (hardware failure, permissions denied)	Saves the recorded session successfully	Prompts user to retry or check microphone settings	Podcast recording is ready for editing and post-processing

3.2.3.10 ERT-10: User Edits Podcast Recording

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
10	User edits podcast recording	Audio segments, edit commands (trim, splice)	Editing session active	Corrupt audio file	Applies edits and updates the podcast recording	Rejects changes and prompts user to re-edit	Edited version saved successfully

3.2.3.11 ERT-11: User Exports Podcast Audio/Video

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
11	User exports podcast audio/video	Selected file format (MP3, WAV, MP4)	Export in progress	Format unsupported	Successfully converts and saves file	Prompts user to select a valid format	Podcast ready for publishing

3.2.3.12 RT-12: User Subscribes to Premium Plan

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
12	User subscribes to premium plan	Payment details, user account	Subscription activation pending	Payment failure	Activates premium features and updates billing	Notifies user of failed transaction and prompts retry	Subscription activated or remains inactive

3.2.3.13 ERT-13: Payment Fails

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
13	Payment fails	Transaction details, error message	Payment processing	Invalid card details or insufficient funds	Displays error message and requests retry	Cancels transaction	Transaction incomplete, user remains unsubscribed

3.2.3.14 ERT-14: User Rates a Podcast Session

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
14	User rates a podcast session	Rating value (1-5 stars), podcast ID	Rating submission pending	Invalid rating value	Stores rating and updates average score	Rejects rating and prompts user to try again	Rating recorded successfully

3.2.3.15 ERT-15: System Generates Podcast Summary

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
15	System generates podcast summary	Podcast transcript, AI processing	Summary generation in progress	Incomplete transcript	Generates AI-based summary and stores it	Logs error and requests additional processing	Summary available for users

3.2.3.16 ERT-16: User Interacts with AI-Powered Notifications

ID	Event	Data Element	System State	Exception Condition	System Response		Final State
					Yes	No	
16	User interacts with AI-powered notifications	Notification content, user engagement data	Notification system active	No relevant notifications	Delivers relevant updates and alerts	No updates available	Notification displayed to the user

3.3 Functional Requirements

3.3.1 UC-1 Register Account

3.3.1.1 FR-1: Account.Register

Identifier	FR-1
Title	Account.Register
Requirement	The system shall allow users to create an account using email and password, verifying email uniqueness and enforcing password strength rules.
Source	Sadia
Rationale	Users must register to access the system securely.
Business Rule	BR-1: Each email must be unique.
Dependencies	FR-2, FR-3
Priority	High

3.3.1.2 FR-2: Account.Register.SocialLogin

Identifier	FR-2
Title	Account.Register.SocialLogin
Requirement	The system shall allow users to register using Google login, retrieving necessary user details and skipping email verification.
Source	Sadia
Rationale	Provides seamless account creation without requiring additional credentials.
Business Rule	BR-1: Each email linked with social accounts must be unique.
Dependencies	FR-1
Priority	High

3.3.1.3 FR-3: Account. ValidateEmail

Identifier	FR-3
Title	Account. ValidateEmail
Requirement	When the user enters their email, the system shall verify that it is not already registered.
Source	Sadia
Rationale	To ensure unique user accounts.

Business Rule	BR-1: Each email must be unique.
Dependencies	FR-4
Priority	High

3.3.1.4 FR-4: Account.EmailVerification

Identifier	FR-4
Title	Account.EmailVerification
Requirement	The system shall send an email verification link to the user and restrict access until the email is verified.
Source	Sadia
Rationale	Ensures security and prevents fraudulent account creation.
Business Rule	BR-1: Users must verify their email before accessing the platform.
Dependencies	FR-3
Priority	High

3.3.1.5 FR-5: Account.EmailVerification.Expiration

Identifier	FR-5
Title	Account.EmailVerification.Expiration
Requirement	The system shall set a 24-hour expiration period for email verification links and allow users to request a new one if expired.
Source	Sadia
Rationale	Enhances security by ensuring links do not remain valid indefinitely.
Business Rule	N/A
Dependencies	FR-4
Priority	Medium

3.3.1.6 FR-6: Account.PasswordStrengthValidation

Identifier	FR-6
Title	Account.PasswordStrengthValidation
Requirement	The system shall enforce password strength criteria, requiring at least one uppercase letter, one lowercase letter, one number, and a minimum of eight characters.
Source	Sadia

Rationale	Ensure users create secure passwords to protect their accounts.
Business Rule	N/A
Dependencies	FR-1
Priority	High

3.3.2 UC-1.2 User Login

3.3.2.1 FR-7: Account.Login

Identifier	FR-7
Title	Account.Login
Requirement	The system shall allow users to log in using email and password after successful registration and verification.
Source	Rabeea
Rationale	Users must authenticate themselves to access the system.
Business Rule	BR-1: Only verified accounts can log in.
Dependencies	FR-8
Priority	High

3.3.2.2 FR-8: Account.Authenticate

Identifier	FR-8
Title	Account.Authenticate
Requirement	The system shall validate user credentials against stored records before granting access.
Source	Rabeea
Rationale	Prevents unauthorized access
Business Rule	BR-1: Only verified accounts can log in.
Dependencies	FR-7
Priority	High

3.3.2.3 FR-9: Account.LoginError

Identifier	FR-6
Title	Account.LoginError
Requirement	If login fails, the system shall display an appropriate error message indicating incorrect credentials or an unverified account.
Source	Rabeea
Rationale	Helps users resolve login issues.
Business Rule	BR-1: Users should have a limited number of login attempts before temporary lockout.

Dependencies	FR-8
Priority	Medium

3.3.3 UC-1.3 Google Authentication

3.3.3.1 FR-10: Account.GoogleLogin

Identifier	FR-10
Title	Account.GoogleLogin
Requirement	The system shall allow users to sign in using their Google credentials via OAuth authentication.
Source	Sadia
Rationale	Provides a seamless and secure login experience.
Business Rule	BR-1: Only Google accounts with valid credentials can be used for authentication.
Dependencies	FR-11
Priority	High

3.3.3.2 FR-11: Account.GoogleRedirect

Identifier	FR-11
Title	Account.GoogleRedirect
Requirement	Upon selecting Google login, the system shall redirect the user to Google's authentication page.
Source	Sadia
Rationale	Ensures authentication is done securely through Google's authorization servers.

Business Rule	N/A
Dependencies	FR-10
Priority	High

3.3.3.3 FR-12: Account.GoogleCredentialVerification

Identifier	FR-12
Title	Account.GoogleCredentialVerification
Requirement	The system shall verify user identity and retrieve necessary user details (name, email, profile picture) from Google.
Source	Sadia
Rationale	Ensures only valid Google accounts can access the system.
Business Rule	BR-1: Only Google accounts with valid credentials can be used for authentication.
Dependencies	FR-11
Priority	High

3.3.3.4 FR-13: Account.GoogleAutoRegister

Identifier	FR-13
Title	Account.GoogleAutoRegister
Requirement	If the user is logging in with Google for the first time, the system shall create a new account using the retrieved Google details.
Source	Sadia
Rationale	Allows new users to register seamlessly without filling out additional forms.
Business Rule	N/A

Dependencies	FR-12
Priority	High

3.3.3.5 FR-14: Account.GoogleLinkedLogin

Identifier	FR-14
Title	Account.GoogleLinkedLogin
Requirement	If a user has previously linked their Google account, they shall be logged in without re-entering credentials.
Source	Sadia
Rationale	Enhances the login experience by reducing repeated authentication steps.
Business Rule	BR-1: Only Google accounts with valid credentials can be used for authentication.
Dependencies	FR-11
Priority	High

3.3.3.6 FR-15: Account.GoogleAuthFailure

Identifier	FR-15
Title	Account.GoogleAuthFailure
Requirement	If authentication with Google fails, the system shall display an error message and prompt the user to retry or use another login method.
Source	Sadia

Rationale	Ensures users can retry authentication or choose an alternative login method.
Business Rule	BR-1: If authentication fails multiple times, users should be prompted to verify their Google account manually.
Dependencies	FR-11
Priority	Medium

3.3.4 UC-1.4 Password Recovery

3.3.4.1 FR-16: Account.PasswordResetRequest

Identifier	FR-16
Title	Account.PasswordResetRequest
Requirement	The system shall allow users to request a password reset link by entering their registered email address.
Source	Rabeea
Rationale	Enables users to regain access to their accounts in case of forgotten passwords.
Business Rule	N/A
Dependencies	FR-17
Priority	High

3.3.4.2 FR-17: Account.ResetLinkGeneration

Identifier	FR-17
Title	Account.ResetLinkGeneration
Requirement	The system shall generate a one-time password reset link and send it to the user’s registered email address.

Source	Rabeea
Rationale	Ensures secure password recovery.
Business Rule	BR-1: Reset links must be unique and expire after a set time.
Dependencies	FR-16
Priority	High

3.3.4.3 FR-18: Account.ResetLinkExpiration

Identifier	FR-18
Title	Account.ResetLinkExpiration
Requirement	The system shall set a 30-minute expiration period for password reset links.
Source	Rabeea
Rationale	Prevents unauthorized access through old reset links.
Business Rule	N/A
Dependencies	FR-17
Priority	High

3.3.4.4 FR-19: Account.InvalidResetRequest

Identifier	FR-19
Title	Account.InvalidResetRequest
Requirement	If the email entered for password recovery is not registered, the system shall display an error message and prompt the user to enter a valid email.
Source	Rabeea

Rationale	Ensures users provide a valid email for password reset.
Business Rule	N/A
Dependencies	FR-16
Priority	Medium

3.3.4.5 FR-20: Account.NewPasswordValidation

Identifier	FR-20
Title	Account.NewPasswordValidation
Requirement	The system shall enforce password strength requirements for new passwords, ensuring security.
Source	Rabeea
Rationale	Protects accounts from weak password vulnerabilities.
Business Rule	BR-1: The new password must meet a minimum strength requirement.
Dependencies	FR-21
Priority	High

3.3.4.6 FR-21: Account.PasswordUpdate

Identifier	FR-21
Title	Account.PasswordUpdate
Requirement	Once the user enters a valid new password, the system shall update the password and allow the user to log in.
Source	Rabeea
Rationale	Ensures the user can regain access after password reset.

Business Rule	N/A
Dependencies	FR-20
Priority	High

3.3.5 UC-1.5 Logout

3.3.5.1 FR-22: Account.Logout

Identifier	FR-22
Title	Account.Logout
Requirement	The system shall allow users to log out, terminating their active session securely.
Source	Rabeea
Rationale	Ensures session security.
Business Rule	N/A
Dependencies	N/A
Priority	High

3.3.5.2 FR-23: Account.SessionTermination

Identifier	FR-23
Title	Account.SessionTermination
Requirement	Upon logout, the system shall clear all session-related data and redirect the user to the login page.
Source	Rabeea
Rationale	Prevents unauthorized access to sensitive data.
Business Rule	N/A
Dependencies	FR-22

Priority	High

3.3.5.3 FR-24: Account.LogoutConfirmation

Identifier	FR-24
Title	Account.LogoutConfirmation
Requirement	The system shall display a confirmation prompt when a user attempts to log out.
Source	Rabeea
Rationale	Prevents accidental logouts.
Business Rule	N/A
Dependencies	FR-22
Priority	Medium

3.3.6 UC-1.6: Access Sample Podcasts

3.3.6.1 FR-25: Podcasts.ViewSamples

Identifier	**FR-25**
Title	Podcasts.ViewSamples
Requirement	The system shall allow users to browse and view sample podcasts from the dashboard.
Source	System
Rationale	Users may explore sample podcasts for inspiration before starting their own podcast.
Business Rule	BR-1: Only verified users can access sample podcasts.
Dependencies	None
Priority	High

3.3.6.2 FR-26: Podcasts.SearchSamples

Identifier	**FR-26**
Title	Podcasts.SearchSamples
Requirement	The system shall allow users to search for specific sample podcasts using filters.
Source	System
Rationale	Users should be able to find specific sample podcasts efficiently.
Business Rule	BR-1: Only verified users can search sample podcasts.
Dependencies	FR-25
Priority	Medium

3.3.7 UC-1.7: Start a Podcast Session

3.3.7.1 FR-27: PodcastSession.Start

Identifier	**FR-27**
Title	PodcastSession.Start
Requirement	The system shall allow users to start a new podcast session with either an AI Host or AI Guest.
Source	System
Rationale	Users should be able to initiate a podcast session as part of their creative process.
Business Rule	BR-1: Only verified users can start a podcast session.
Dependencies	None
Priority	High

3.3.7.2 FR-28: PodcastSession.Resume

Identifier	**FR-28**
Title	PodcastSession.Resume
Requirement	If a session is incomplete, the system shall allow users to resume from the last saved session.
Source	System
Rationale	Users should be able to continue an unfinished podcast session without losing progress.
Business Rule	BR-1: Only users with previous session data can resume.
Dependencies	FR-27
Priority	Medium

3.3.8 UC-1.8: View Dashboard

3.3.8.1 FR-29: Dashboard.ViewContent

Identifier	**FR-29**
Title	Dashboard.ViewContent
Requirement	The system shall display a list of relevant podcast content based on the user's profile and followed creators.
Source	System
Rationale	Users should see personalized content upon accessing the dashboard.
Business Rule	BR-1: Content must adhere to moderation policies.
Dependencies	None
Priority	High

3.3.8.2 FR-30: Dashboard.HandleNoConnection

Identifier	**FR-30**
Title	Dashboard.HandleNoConnection
Requirement	The system shall display an error message and attempt a reload when no internet connection is available.
Source	System
Rationale	Users should be informed of connectivity issues when accessing the dashboard.
Business Rule	None
Dependencies	FR-29
Priority	Medium

3.3.9 UC-1.9: Create and Personalize User Profile

3.3.9.1 FR-31: Profile.Create

Identifier	**FR-31**
Title	Profile.Create
Requirement	The system shall allow users to create and personalize their profile by adding a display name, bio, and profile picture.
Source	System
Rationale	A personalized profile helps users engage with the platform.
Business Rule	BR-1: Users must provide a unique display name.
Dependencies	None
Priority	High

3.3.9.2 FR-32: Profile.Update

Identifier	**FR-32**
Title	Profile.Update
Requirement	Users shall be able to update their profile details, including bio and profile picture.
Source	System
Rationale	Users should have the flexibility to update their profile information.
Business Rule	BR-1: Profile updates should be saved immediately.
Dependencies	FR-31
Priority	High

3.3.10 UC-1.10: Edit Profile

3.3.10.1 FR-33: Profile.EditDetails

Identifier	**FR-33**
Title	Profile.EditDetails
Requirement	Users shall be able to edit their username, phone number, and password.
Source	System
Rationale	Users need to manage their profile information efficiently.
Business Rule	BR-1: Only one account can be registered per username.
Dependencies	FR-31, FR-32
Priority	High

3.3.10.2 FR-34: Profile.ValidatePassword

Identifier	**FR-34**
Title	Profile.ValidatePassword
Requirement	The system shall validate that new passwords meet the required security criteria.
Source	System
Rationale	Ensuring strong passwords enhances security.
Business Rule	BR-1: Password must contain alphabets and digits.
Dependencies	FR-33
Priority	High

3.3.11 UC-1.11: Follow Creators and View Updates

3.3.11.1 FR-35: FollowCreator

Identifier	**FR-35**
Title	FollowCreator
Requirement	The system shall allow users to follow and unfollow creators.
Source	System
Rationale	Users should be able to curate their content feed.
Business Rule	BR-1: Users can follow or unfollow creators at any time.
Dependencies	None
Priority	High

3.3.11.2 FR-36: ViewCreatorUpdates

Identifier	**FR-36**
Title	ViewCreatorUpdates
Requirement	The system shall display updates from followed creators in the user's feed.
Source	System
Rationale	Users should be informed of new content from creators they follow.
Business Rule	BR-1: Feed updates should be in real-time.
Dependencies	FR-35
Priority	High

3.3.12 UC-1.12: View Activity Feed of Followed Creators

3.3.12.1 FR-37: ActivityFeed.View

Identifier	**FR-37**
Title	ActivityFeed.View
Requirement	The system shall allow users to view recent activities of followed creators.
Source	System
Rationale	Users should stay updated on creator activities.
Business Rule	BR-1: Activity feed updates should be in real-time.
Dependencies	FR-36
Priority	High

3.3.12.2 FR-38: ActivityFeed.HandleNoData

Identifier	**FR-38**
Title	ActivityFeed.HandleNoData
Requirement	If no activity is available, the system shall inform the user and suggest following more creators.
Source	System
Rationale	Users should understand why their activity feed is empty.
Business Rule	None
Dependencies	FR-37
Priority	Medium

3.3.13 UC-2.1 Voice Cloning Setup

3.3.13.1 FR-39: VoiceCloning.RecordSample

Identifier	**FR-39**
Title	VoiceCloning.RecordSample
Requirement	The system shall allow users to record a voice sample for cloning.
Source	Rabeea
Rationale	Enables AI-generated voices.
Business Rule	BR-11: Recording must be clear and meet quality standards.
Dependencies	FR-40
Priority	High

3.3.13.2 FR-40: VoiceCloning.UploadSample

Identifier	**FR-40**
Title	VoiceCloning.UploadSample
Requirement	The system shall allow users to upload a voice sample in supported formats.
Source	Rabeea
Rationale	Provides flexibility for AI voice generation.
Business Rule	BR-12: Only WAV, MP3 formats are allowed.
Dependencies	FR-41
Priority	High

3.3.13.3 FR-41: VoiceCloning.ProcessSample

Identifier	**FR-41**
Title	VoiceCloning.ProcessSample
Requirement	The system shall process the uploaded or recorded voice sample for AI cloning.
Source	Sadia
Rationale	Ensures high-quality AI-generated voice.
Business Rule	BR-13: Processing must maintain original voice characteristics.
Dependencies	FR-42
Priority	High

3.3.13.4 FR-42: VoiceCloning.PreviewGeneratedVoice

Identifier	**FR-42**
Title	VoiceCloning.PreviewGeneratedVoice
Requirement	The system shall allow users to preview the generated AI voice before saving.
Source	Sadia
Rationale	Allows user verification of quality.
Business Rule	BR-14: Users can re-record or modify AI voice if unsatisfied.
Dependencies	FR-41
Priority	Medium

3.3.14 UC-3.1 Knowledge Base Setup

3.3.14.1 FR-43: KnowledgeBase.UploadData

Identifier	**FR-43**
Title	KnowledgeBase.UploadData
Requirement	The system shall allow users to upload structured and unstructured data (PDF, CSV, JSON, URLs).
Source	Rabeea
Rationale	Ensures AI has relevant information for retrieval.
Business Rule	BR-15: Only supported formats can be uploaded.
Dependencies	FR-44
Priority	High

3.3.14.2 FR-44: KnowledgeBase.ProcessData

Identifier	**FR-44**
Title	KnowledgeBase.ProcessData
Requirement	The system shall clean, format, and structure uploaded data for efficient AI retrieval.
Source	Sadia
Rationale	Ensures high-quality and relevant AI responses.
Business Rule	BR-16: Metadata must be attached to each data chunk.
Dependencies	FR-45
Priority	High

3.3.14.3 FR-45: KnowledgeBase.VectorEmbedding

Identifier	**FR-45**
Title	KnowledgeBase.VectorEmbedding
Requirement	The system shall generate vector embeddings for better AI retrieval performance.
Source	Rabeea
Rationale	Enhances AI's ability to match queries with relevant data.
Business Rule	BR-17: Vectorization must ensure minimal data loss.
Dependencies	FR-44
Priority	High

3.3.14.4 FR-46: KnowledgeBase.MetadataTagging

Identifier	**FR-46**
Title	KnowledgeBase.MetadataTagging
Requirement	The system shall attach metadata to uploaded data for indexing and retrieval.
Source	Sadia
Rationale	Ensures accurate filtering and searching of knowledge base information.
Business Rule	BR-18: Metadata must follow a standard format.
Dependencies	FR-41
Priority	Medium

3.3.15 UC-4.1 AI Agent Customization

3.3.15.1 FR-47: AIAgent.CustomizeAppearance

Identifier	**FR-47**
Title	AIAgent.CustomizeAppearance
Requirement	The system shall allow users to set a custom name, avatar, and voice for their AI agent.
Source	Rabeea
Rationale	Enables personalized podcasting experiences.
Business Rule	BR-19: Users must configure AI before recording.
Dependencies	FR-48
Priority	High

3.3.15.2 FR-48: AIAgent.SavePreferences

Identifier	**FR-48**
Title	AIAgent.SavePreferences
Requirement	The system shall save AI agent settings for future sessions.
Source	Sadia
Rationale	Provides users with a consistent podcasting experience.
Business Rule	BR-20: Users can modify settings before each session.
Dependencies	FR-47
Priority	Medium

3.3.15.3 FR-49: AIAgent.LoadPreferences

Identifier	**FR-49**
Title	AIAgent.LoadPreferences
Requirement	The system shall allow users to load previously saved AI preferences automatically.
Source	Rabeea
Rationale	Ensures user convenience and efficiency.
Business Rule	BR-21: Preferences must be stored securely.
Dependencies	FR-48
Priority	Medium

3.3.16 UC-5.1 Record Podcast Session

3.3.16.1 FR-50: PodcastSession.StartRecording

Identifier	**FR-50**
Title	PodcastSession.StartRecording
Requirement	The system shall allow users to start recording a podcast session with AI-generated or real co-hosts.
Source	Rabeea
Rationale	Enables users to create and capture podcast content.
Business Rule	BR-22: Recording must start only after user confirmation.
Dependencies	FR-51
Priority	High

3.3.16.2 FR-51: PodcastSession.PauseRecording

Identifier	**FR-51**
Title	PodcastSession.PauseRecording
Requirement	The system shall allow users to pause an ongoing podcast recording and resume later.
Source	Sadia
Rationale	Provides flexibility for users during recordings.
Business Rule	BR-23: Paused recordings should not be lost.
Dependencies	FR-50
Priority	Medium

3.3.16.3 FR-52: PodcastSession.StopRecording

Identifier	**FR-52**
Title	PodcastSession.StopRecording
Requirement	The system shall allow users to stop a podcast recording and save the session.
Source	Rabeea
Rationale	Ensures completed recordings are stored securely.
Business Rule	BR-24: Users must confirm before stopping the session.
Dependencies	FR-50
Priority	High

3.3.16.4 FR-53: PodcastSession.SetRecordingLimit

Identifier	**FR-53**
Title	PodcastSession.SetRecordingLimit
Requirement	The system shall enforce a maximum recording time per session.
Source	Rabeea
Rationale	Prevents excessively long recordings that could impact performance.
Business Rule	BR-25: Recording limit should be configurable by the user.
Dependencies	FR-50
Priority	Medium

3.3.16.5 FR-54: PodcastSession.AutoSaveRecording

Identifier	**FR-54**
Title	PodcastSession.AutoSaveRecording
Requirement	The system shall periodically auto-save the recording to prevent data loss.
Source	Sadia
Rationale	Ensures users do not lose content in case of crashes.
Business Rule	BR-26: Auto-save should trigger every 5 minutes.
Dependencies	FR-50
Priority	High

3.3.17 UC-5.2 Edit, Review, and Export Podcast

3.3.17.1 FR-56: PodcastSession.LoadEditingInterface

Identifier	**FR-56**
Title	PodcastSession.LoadEditingInterface
Requirement	The system shall load an audio editing interface when the user selects "Edit Recording."
Source	Rabeea
Rationale	Provides users with a dedicated editing workspace.
Business Rule	BR-28: Editing interface should load within 5 seconds.
Dependencies	FR-52
Priority	High

3.3.17.2 FR-57: PodcastSession.TrimAudio

Identifier	**FR-57**
Title	PodcastSession.TrimAudio
Requirement	The system shall allow users to trim recorded audio to remove unwanted parts.
Source	Sadia
Rationale	Ensures refined podcast content.
Business Rule	BR-29: Trimmed sections should be reversible until final save.
Dependencies	FR-56
Priority	High

3.3.17.3 FR-58: PodcastSession.RearrangeClips

Identifier	**FR-58**
Title	PodcastSession.RearrangeClips
Requirement	The system shall allow users to rearrange audio clips within the timeline.
Source	Rabeea
Rationale	Provides flexibility in podcast structuring.
Business Rule	BR-30: Rearranged clips must maintain smooth transitions.
Dependencies	FR-56
Priority	High

3.3.17.4 FR-59: PodcastSession.MergeAudio

Identifier	**FR-59**
Title	PodcastSession.MergeAudio
Requirement	The system shall allow users to merge multiple audio segments into a single file.
Source	Sadia
Rationale	Provides flexibility in podcast editing.
Business Rule	BR-31: Merged audio must maintain original quality.
Dependencies	FR-57, FR-58
Priority	Medium

3.3.17.5 FR-60: PodcastSession.ApplyFilters

Identifier	**FR-60**
Title	PodcastSession.ApplyFilters
Requirement	The system shall allow users to apply noise reduction and enhancement filters.
Source	Rabeea
Rationale	Ensures high-quality audio output.
Business Rule	BR-32: Audio enhancements must be previewed before final save.
Dependencies	FR-59
Priority	Medium

3.3.17.6 FR-61: PodcastSession.SaveEditedRecording

Identifier	**FR-61**
Title	PodcastSession.SaveEditedRecording
Requirement	The system shall allow users to save the edited recording.
Source	Sadia
Rationale	Ensures changes are stored securely.
Business Rule	BR-33: Users must confirm before overwriting the original file.
Dependencies	FR-60
Priority	High

3.3.17.7 FR-62: PodcastSession.UpdateFinalAudio

Identifier	**FR-62**
Title	PodcastSession.UpdateFinalAudio
Requirement	The system shall replace the original recording with the updated final version.
Source	Rabeea
Rationale	Ensures the final version is stored and ready for publishing.
Business Rule	BR-34: System should maintain a backup of the original file.
Dependencies	FR-61
Priority	High

3.3.17.8 FR-63: PodcastSession.PreviewFinalAudio

Identifier	**FR-63**
Title	PodcastSession.PreviewFinalAudio

Requirement	The system shall allow users to preview the final edited podcast before saving.
Source	Rabeea
Rationale	Ensures users can review their content before finalizing.
Business Rule	BR-31: Users must confirm before saving the final version.
Dependencies	FR-62
Priority	High

3.3.17.9 FR-64: PodcastSession.AdjustVolumeLevels

Identifier	**FR-64**
Title	PodcastSession.AdjustVolumeLevels
Requirement	The system shall allow users to adjust volume levels for consistency before saving.
Source	Sadia
Rationale	Ensures clear and balanced audio output.
Business Rule	BR-32: Volume normalization should be automatic but adjustable.
Dependencies	FR-63
Priority	Medium

3.3.17.10FR-65: PodcastSession.AddEpisodeMetadata

Identifier	**FR-65**
Title	PodcastSession.AddEpisodeMetadata
Requirement	The system shall allow users to enter episode metadata (title, description, tags) before saving.
Source	Rabeea
Rationale	Ensures podcasts are properly categorized for future reference.

Business Rule	BR-33: Metadata fields should be required before final save.
Dependencies	FR-64
Priority	High

3.3.17.11FR-66: PodcastSession.PublishPodcast

Identifier	**FR-66**
Title	PodcastSession.PublishPodcast
Requirement	The system shall allow users to publish the podcast directly to selected platforms.
Source	Sadia
Rationale	Enables easy distribution of podcasts.
Business Rule	BR-34: Users must authenticate platform access before publishing.
Dependencies	FR-65
Priority	High

3.3.17.12FR-67: PodcastSession.GenerateFileName

Identifier	**FR-67**
Title	PodcastSession.GenerateFileName
Requirement	The system shall automatically generate a file name based on episode metadata.
Source	Rabeea
Rationale	Ensures a standardized file naming format.
Business Rule	BR-35: Users can override the generated file name if needed.
Dependencies	FR-65
Priority	Medium

3.3.17.13FR-68: PodcastSession.ExportAudio

Identifier	**FR-68**
Title	PodcastSession.ExportAudio
Requirement	The system shall allow users to export the finalized podcast in MP3/WAV formats.
Source	Sadia
Rationale	Enables users to distribute their podcast on different platforms.
Business Rule	BR-36: Users must select a compatible format before export.
Dependencies	FR-67
Priority	High

3.3.17.14FR-69: PodcastSession.SaveDraft

Identifier	**FR-69**
Title	PodcastSession.SaveDraft
Requirement	The system shall allow users to save an incomplete podcast session as a draft.
Source	Rabeea
Rationale	Allows users to return and finalize content later.
Business Rule	BR-37: Drafts must be auto-saved periodically.
Dependencies	FR-65
Priority	Medium

3.3.18 UC-6.1 AI Host & AI Guest Interaction

3.3.18.1 FR-70: PodcastAI.InitializeSession

Identifier	**FR-70**
Title	PodcastAI.InitializeSession
Requirement	The system shall allow users to initiate a podcast session with an AI Host or AI Guest.
Source	Rabeea
Rationale	Enables AI-powered podcasting experiences.
Business Rule	BR-41: AI settings must be pre-configured before the session starts.
Dependencies	None
Priority	High

3.3.18.2 FR-71: PodcastAI.LoadSettings

Identifier	**FR-71**
Title	PodcastAI.LoadSettings
Requirement	The system shall load the pre-configured AI settings, voice model, and knowledge base before starting the session.
Source	Sadia
Rationale	Ensures AI uses the correct configurations for smooth interaction.
Business Rule	BR-42: AI must have access to the selected knowledge base before interaction.
Dependencies	FR-70
Priority	High

3.3.18.3 FR-72: PodcastAI.HostIntroduction

Identifier	**FR-72**
Title	PodcastAI.HostIntroduction
Requirement	The AI Host shall introduce the episode and set the discussion agenda.
Source	Rabeea
Rationale	Provides a structured and engaging podcast experience.
Business Rule	BR-43: AI introductions must align with the selected episode theme.
Dependencies	FR-71
Priority	High

3.3.18.4 FR-73: PodcastAI.GuestResponses

Identifier	**FR-73**
Title	PodcastAI.GuestResponses
Requirement	The AI Guest shall dynamically respond to user queries and share insights.
Source	Sadia
Rationale	Ensures an interactive and engaging podcast conversation.
Business Rule	BR-44: AI responses must be contextually relevant.
Dependencies	FR-72
Priority	High

3.3.18.5 FR-74: PodcastAI.ManageTransitions

Identifier	**FR-74**
Title	PodcastAI.ManageTransitions

Requirement	The AI Host shall facilitate smooth topic transitions during the podcast session.
Source	Rabeea
Rationale	Ensures structured and seamless conversations.
Business Rule	BR-45: Transitions should align with the episode flow and pacing.
Dependencies	FR-73
Priority	Medium

3.3.18.6 FR-75: PodcastAI.GenerateSummary

Identifier	**FR-75**
Title	PodcastAI.GenerateSummary
Requirement	The AI Host shall generate and provide a structured summary of the discussion at the end of the session.
Source	Sadia
Rationale	Allows users to review key discussion points.
Business Rule	BR-46: Summaries must highlight the main topics and insights covered.
Dependencies	FR-74
Priority	Medium

3.3.19 UC-7.1 Create and Export Video Podcast

3.3.19.1 FR-76: PodcastAvatars.DisplayPredefinedAvatars

Identifier	**FR-76**
Title	PodcastAvatars.DisplayPredefinedAvatars
Requirement	The system shall provide a selection of predefined avatars for users to choose from.

Source	System
Rationale	Ensures that users have quick access to avatars without uploading their own.
Business Rule	BR-47: Predefined avatars must be diverse and customizable.
Dependencies	None
Priority	High

3.3.19.2 FR-77: PodcastAvatars.UploadCustomAvatar

Identifier	**FR-77**
Title	PodcastAvatars.UploadCustomAvatar
Requirement	The system shall allow users to upload a custom avatar for each speaker.
Source	Creator
Rationale	Provides personalization options for users.
Business Rule	BR-48: Uploaded avatars must meet file format and size requirements.
Dependencies	FR-76
Priority	High

3.3.19.3 FR-78: PodcastAvatars.AssignAvatarsToSpeakers

Identifier	**FR-78**
Title	PodcastAvatars.AssignAvatarsToSpeakers
Requirement	The system shall allow users to assign an avatar to each speaker in the podcast.
Source	Creator
Rationale	Ensures each speaker has a unique visual identity.
Business Rule	BR-49: Each speaker must be assigned an avatar before proceeding.

Dependencies	FR-76, FR-77
Priority	High

3.3.19.4 FR-79: PodcastAvatars.ModifyAvatars

Identifier	**FR-79**
Title	PodcastAvatars.ModifyAvatars
Requirement	The system shall allow users to change assigned avatars before finalizing selections.
Source	Creator
Rationale	Provides flexibility in avatar selection.
Business Rule	BR-50: Users must confirm changes before applying.
Dependencies	FR-78
Priority	Medium

3.3.19.5 FR-80: PodcastAvatars.ValidateAvatarSelection

Identifier	**FR-80**
Title	PodcastAvatars.ValidateAvatarSelection
Requirement	The system shall prompt users to choose or upload an avatar before proceeding.
Source	System
Rationale	Ensures compliance with avatar selection rules.
Business Rule	BR-51: Users cannot proceed without assigning an avatar.
Dependencies	FR-78
Priority	High

3.3.19.6 FR-81: PodcastVisuals.DisplayPredefinedVisuals

Identifier	**FR-81**
Title	PodcastVisuals.DisplayPredefinedVisuals
Requirement	The system shall provide a library of background themes, overlays, and effects.
Source	System
Rationale	Allows users to select professional and engaging visuals.
Business Rule	BR-52: Predefined visuals must be categorized for easier selection.
Dependencies	None
Priority	High

3.3.19.7 FR-82: PodcastVisuals.UploadCustomVisuals

Identifier	**FR-82**
Title	PodcastVisuals.UploadCustomVisuals
Requirement	The system shall allow users to upload custom visuals for the podcast.
Source	Creator
Rationale	Provides customization for unique podcast branding.
Business Rule	BR-53: Uploaded visuals must meet format and resolution guidelines.
Dependencies	FR-81
Priority	High

3.3.19.8 FR-83: PodcastPreview.PlayVideo

Identifier	**FR-83**
Title	PodcastPreview.PlayVideo
Requirement	The system shall allow users to preview the full video podcast before exporting.
Source	Creator
Rationale	Ensures users can review the final product before publishing.
Business Rule	BR-54: Preview must match the final exported video quality.
Dependencies	None
Priority	High

3.3.19.9 FR-84: PodcastPreview.ConfirmQuality

Identifier	**FR-84**
Title	PodcastPreview.ConfirmQuality
Requirement	The system shall allow users to confirm video quality before proceeding to export.
Source	Creator
Rationale	Prevents low-quality exports and ensures professional output.
Business Rule	BR-55: Users must approve quality before export.
Dependencies	FR-83
Priority	Medium

3.3.19.10FR-85: PodcastExport.ExportVideo

Identifier	**FR-85**
Title	PodcastExport.ExportVideo

Requirement	The system shall allow users to export the video podcast in MP4 format.
Source	Creator
Rationale	Enables easy sharing and publishing.
Business Rule	BR-56: Exported videos must maintain the original quality.
Dependencies	FR-84
Priority	High

3.3.20 UC-7.4 Request Podcast Summary

3.3.20.1 FR-86: PodcastSummary.GenerateSummary

Identifier	**FR-86**
Title	PodcastSummary.GenerateSummary
Requirement	The system shall generate an AI-based summary for a selected podcast episode.
Source	System
Rationale	Provides quick insights and key takeaways for users.
Business Rule	BR-57: Summaries must highlight main discussion points.
Dependencies	None
Priority	High

3.3.20.2 FR-87: PodcastSummary.CustomizeSummaryFormat

Identifier	**FR-87**
Title	PodcastSummary.CustomizeSummaryFormat
Requirement	The system shall allow users to choose between a detailed or brief summary format.

Source	User
Rationale	Provides flexibility in how users consume summaries.
Business Rule	BR-58: Users must be able to switch summary formats dynamically.
Dependencies	FR-86
Priority	Medium

3.3.21 UC-7.5 Search for Specific Information in a Podcast Summary

3.3.21.1 FR-88: PodcastSummary.SearchKeywords

Identifier	**FR-88**
Title	PodcastSummary.SearchKeywords
Requirement	The system shall allow users to search for specific topics or keywords within a podcast summary.
Source	User
Rationale	Helps users find relevant content quickly.
Business Rule	BR-59: Search results must be contextually relevant.
Dependencies	FR-86
Priority	High

3.3.21.2 FR-89: PodcastSummary.HighlightResults

Identifier	**FR-89**
Title	PodcastSummary.HighlightResults
Requirement	The system shall highlight matching sections of the summary based on the search query.
Source	System

Rationale	Improves user experience in locating information.
Business Rule	BR-60: Highlighted sections must be clearly visible.
Dependencies	FR-88
Priority	Medium

3.3.22 UC-7.6 Rate Podcast Session

3.3.22.1 FR-90: PodcastRating.ProvideRating

Identifier	**FR-90**
Title	PodcastRating.ProvideRating
Requirement	The system shall allow users to rate a completed podcast session on a scale (e.g., 1-5 stars).
Source	User
Rationale	Collects user feedback for improving podcast quality.
Business Rule	BR-61: Users can only rate sessions they have hosted or attended.
Dependencies	None
Priority	High

3.3.22.2 FR-91: PodcastRating.PreventDuplicateRatings

Identifier	**FR-91**
Title	PodcastRating.PreventDuplicateRatings
Requirement	The system shall prevent users from rating the same session multiple times.
Source	System
Rationale	Ensures fair and accurate rating data.

Business Rule	BR-62: Users can update their rating but not submit duplicates.
Dependencies	FR-90
Priority	Medium

3.3.23 UC-8.1 Subscription Management

3.3.23.1 FR-92: Subscription.DisplayPlans

Identifier	**FR-92**
Title	Subscription.DisplayPlans
Requirement	The system shall display all available subscription plans.
Source	System
Rationale	Ensures users can view and compare plans before subscribing.
Business Rule	BR-63: Plans must include pricing, features, and billing cycle.
Dependencies	None
Priority	High

3.3.23.2 FR-93: Subscription.ProcessPayment

Identifier	**FR-93**
Title	Subscription.ProcessPayment
Requirement	The system shall securely process subscription payments.
Source	System
Rationale	Ensures safe transactions for subscription-based access.
Business Rule	BR-64: Payment must be validated through a secure gateway.

Dependencies	FR-92
Priority	High

3.3.23.3 FR-94: Subscription.ManageAutoRenewal

Identifier	**FR-94**
Title	Subscription.ManageAutoRenewal
Requirement	The system shall allow users to enable or disable auto-renewal.
Source	User
Rationale	Provides users control over their recurring payments.
Business Rule	BR-65: Auto-renewal settings must be user-configurable.
Dependencies	FR-93
Priority	Medium

3.3.23.4 FR-95: Subscription.HandlePaymentFailure

Identifier	**FR-95**
Title	Subscription.HandlePaymentFailure
Requirement	The system shall notify users and prompt them to update payment details if payment fails.
Source	System
Rationale	Prevents unintended subscription cancellations.
Business Rule	BR-66: Users must be given at least one retry attempt.
Dependencies	FR-93
Priority	High

3.3.24 UC-8.2 Free Content Access Limit

3.3.24.1 FR-96: FreeAccess.TrackUsage

Identifier	**FR-96**
Title	FreeAccess.TrackUsage
Requirement	The system shall track the number of free content sessions used by a user.
Source	System
Rationale	Ensures users do not exceed free content limits.
Business Rule	BR-67: Free usage count must reset after a billing cycle.
Dependencies	None
Priority	High

3.3.24.2 FR-97: FreeAccess.PromptSubscription

Identifier	**FR-97**
Title	FreeAccess.PromptSubscription
Requirement	The system shall prompt users to subscribe once the free content limit is reached.
Source	System
Rationale	Encourages users to upgrade for unlimited access.
Business Rule	BR-68: Prompt must include subscription benefits.
Dependencies	FR-96
Priority	High

3.3.25 UC-8.3 Subscription Expiry Reminder

3.3.25.1 FR-98: Subscription.SendExpiryNotification

Identifier	**FR-98**
Title	Subscription.SendExpiryNotification
Requirement	The system shall notify users before their subscription expires.
Source	System
Rationale	Ensures users are aware of expiration dates to prevent service interruption.
Business Rule	BR-69: Notifications must be sent at least 3 days before expiry.
Dependencies	FR-94
Priority	High

3.3.25.2 FR-99: Subscription.RenewSubscription

Identifier	**FR-99**
Title	Subscription.RenewSubscription
Requirement	The system shall allow users to renew their subscription before expiry.
Source	User
Rationale	Prevents loss of access due to expired subscriptions.
Business Rule	BR-70: Renewals must process through a secure payment gateway.
Dependencies	FR-98
Priority	High

3.3.26 UC-9.1 Ask a Question from FAQs Chatbot

3.3.26.1 FR-100: FAQsChatbot.OpenChatbot

Identifier	**FR-100**
Title	FAQsChatbot.OpenChatbot
Requirement	The system shall allow users to open the FAQs chatbot from the menu.
Source	System
Rationale	Ensures users can quickly access FAQ support.
Business Rule	BR-71: The chatbot must be accessible at all times.
Dependencies	None
Priority	High

3.3.26.2 FR-101: FAQsChatbot.ProcessUserQuery

Identifier	**FR-101**
Title	FAQsChatbot.ProcessUserQuery
Requirement	The system shall process user queries and retrieve relevant FAQ answers.
Source	System
Rationale	Ensures that users get accurate responses to their questions.
Business Rule	BR-72: Responses must come from the verified FAQs database.
Dependencies	FR-100
Priority	High

3.3.26.3 FR-102: FAQsChatbot.HandleNoMatch

Identifier	**FR-102**
Title	FAQsChatbot.HandleNoMatch
Requirement	If no exact match is found, the system shall suggest related FAQs.
Source	System
Rationale	Provides alternative answers when a direct match is unavailable.
Business Rule	BR-73: Suggested FAQs must be contextually relevant.
Dependencies	FR-101
Priority	Medium

3.3.26.4 FR-103: FAQsChatbot.ImproveResponses

Identifier	**FR-103**
Title	FAQsChatbot.ImproveResponses
Requirement	The system shall log chatbot interactions to improve future responses.
Source	System
Rationale	Helps refine chatbot accuracy over time.
Business Rule	BR-74: User interactions should be analyzed periodically.
Dependencies	FR-101
Priority	Medium

3.3.27 UC-9.2 View Suggested FAQs

3.3.27.1 FR-104: FAQsChatbot.DisplayCommonQuestions

Identifier	**FR-104**
Title	FAQsChatbot.DisplayCommonQuestions
Requirement	The system shall display commonly asked questions when the chatbot is opened.
Source	System
Rationale	Helps users find answers quickly without typing queries.
Business Rule	BR-75: The FAQ list should update based on recent user activity.
Dependencies	None
Priority	High

3.3.27.2 FR-105: FAQsChatbot.AllowFAQSelection

Identifier	**FR-105**
Title	FAQsChatbot.AllowFAQSelection
Requirement	The system shall allow users to select a suggested FAQ to view its answer.
Source	User
Rationale	Simplifies navigation within the chatbot.
Business Rule	BR-76: Selected FAQs must display full, structured responses.
Dependencies	FR-104
Priority	High

3.3.27.3 FR-106: FAQsChatbot.HandleNoFAQsAvailable

Identifier	**FR-106**
Title	FAQsChatbot.HandleNoFAQsAvailable
Requirement	If no suggested FAQs are available, the system shall provide a manual search option.
Source	System
Rationale	Ensures users can still find answers even if no common FAQs exist.
Business Rule	BR-77: Search results must prioritize relevance.
Dependencies	FR-104
Priority	Medium

3.3.28 UC-10.1 Receive Notifications for Audience Engagement

3.3.28.1 FR-107: Notifications.GenerateEngagementNotification

Identifier	**FR-107**
Title	Notifications.GenerateEngagementNotification
Requirement	The system shall generate real-time notifications for likes, comments, shares, and follows.
Source	System
Rationale	Ensures creators are informed of audience interactions.
Business Rule	BR-78: Notifications must be sent instantly.
Dependencies	None
Priority	High

3.3.28.2 FR-108: Notifications.StoreEngagementHistory

Identifier	**FR-108**
Title	Notifications.StoreEngagementHistory
Requirement	The system shall store all engagement notifications in the creator's notification panel.
Source	System
Rationale	Allows creators to review past interactions.
Business Rule	BR-79: Notifications should be stored for at least 30 days.
Dependencies	FR-107
Priority	Medium

3.3.29 UC-10.2 User-to-User Messaging and Calls

3.3.29.1 FR-109: Messaging.SendMessage

Identifier	**FR-109**
Title	Messaging.SendMessage
Requirement	The system shall allow users to send real-time text messages.
Source	User
Rationale	Enables seamless communication between users.
Business Rule	BR-80: Users can only message those who have enabled messaging.
Dependencies	None
Priority	High

3.3.29.2 FR-110: Messaging.InitiateCall

Identifier	**FR-110**
Title	Messaging.InitiateCall
Requirement	The system shall allow users to initiate voice and video calls.
Source	User
Rationale	Supports real-time collaboration.
Business Rule	BR-81: Calls can only be made to users with calling enabled.
Dependencies	None
Priority	High

3.3.30 UC-10.3 Engage with Podcasts (Like, Comment, Share)

3.3.30.1 FR-111: PodcastEngagement.LikePodcast

Identifier	**FR-111**
Title	PodcastEngagement.LikePodcast
Requirement	The system shall allow users to like podcast episodes.
Source	User
Rationale	Encourages audience interaction.
Business Rule	BR-82: Users can like a podcast only once.
Dependencies	None
Priority	High

3.3.30.2 FR-112: PodcastEngagement.CommentOnPodcast

Identifier	**FR-112**
Title	PodcastEngagement.CommentOnPodcast
Requirement	The system shall allow users to comment on podcast episodes.
Source	User
Rationale	Facilitates discussion and feedback.
Business Rule	BR-83: Comments must follow community guidelines.
Dependencies	None
Priority	High

3.3.31 UC-11.1 Manage User Accounts

3.3.31.1 FR-113: Admin.ViewUserAccounts

Identifier	**FR-113**
Title	Admin.ViewUserAccounts
Requirement	The system shall allow admins to search and view user accounts.
Source	Admin
Rationale	Enables administrators to manage users efficiently.
Business Rule	BR-86: Only authorized admins can access user details.
Dependencies	None
Priority	High

3.3.31.2 FR-114: Admin.ManageUserStatus

Identifier	**FR-114**
Title	Admin.ManageUserStatus
Requirement	The system shall allow admins to deactivate, reactivate, or delete user accounts.
Source	Admin
Rationale	Provides flexibility in user management.
Business Rule	BR-87: Users with active subscriptions cannot be deleted.
Dependencies	FR-113
Priority	High

3.3.32 UC-11.2 Review Reported Content

3.3.32.1 FR-115: Admin.ViewReportedContent

Identifier	**FR-115**
Title	Admin.ViewReportedContent
Requirement	The system shall allow admins to review flagged podcasts and comments.
Source	Admin
Rationale	Ensures content moderation and compliance.
Business Rule	BR-88: Only admins with moderation rights can review content.
Dependencies	None
Priority	High

3.3.32.2 FR-116: Admin.TakeActionOnReportedContent

Identifier	**FR-116**
Title	Admin.TakeActionOnReportedContent
Requirement	The system shall allow admins to approve, remove, or warn users regarding reported content.
Source	Admin
Rationale	Enables proper handling of flagged content.
Business Rule	BR-89: All actions must be logged for audit purposes.
Dependencies	FR-115
Priority	High

3.3.33 UC-11.3 View Platform Analytics

3.3.33.1 FR-117: Admin.ViewAnalyticsDashboard

Identifier	**FR-117**
Title	Admin.ViewAnalyticsDashboard
Requirement	The system shall provide an analytics dashboard displaying user activity and content engagement.
Source	System
Rationale	Helps admins monitor platform performance.
Business Rule	BR-90: Analytics should be updated in real-time.
Dependencies	None
Priority	High

3.3.33.2 FR-118: Admin.FilterAnalyticsData

Identifier	**FR-118**
Title	Admin.FilterAnalyticsData
Requirement	The system shall allow admins to filter analytics by date range and content type.
Source	Admin
Rationale	Enables detailed trend analysis.
Business Rule	BR-91: Filters should not impact real-time updates.
Dependencies	FR-117
Priority	Medium

3.3.34 UC-11.4 Manage Subscriptions & Payments

3.3.34.1 FR-119: Admin.ViewSubscriptionDetails

Identifier	**FR-119**
Title	Admin.ViewSubscriptionDetails
Requirement	The system shall allow admins to view active subscriptions and payment history.
Source	Admin
Rationale	Enables tracking of subscription payments.
Business Rule	BR-92: Only finance-privileged admins can access subscription details.
Dependencies	None
Priority	High

3.3.34.2 FR-120: Admin.ManageRefundsAndDisputes

Identifier	**FR-120**
Title	Admin.ManageRefundsAndDisputes
Requirement	The system shall allow admins to process refunds and handle payment disputes.
Source	Admin
Rationale	Ensures smooth financial operations.
Business Rule	BR-93: Refund eligibility must align with platform policies.
Dependencies	FR-119
Priority	High

3.3.35 UC-11.5 Generate User Engagement Reports

3.3.35.1 FR-121: Admin.GenerateUserReports

Identifier	**FR-121**
Title	Admin.GenerateUserReports
Requirement	The system shall allow admins to generate reports on user engagement and podcast performance.
Source	Admin
Rationale	Provides insights into platform growth.
Business Rule	BR-94: Reports must include the latest available data.
Dependencies	None
Priority	High

3.3.35.2 FR-122: Admin.ScheduleAutoReports

Identifier	**FR-122**
Title	Admin.ScheduleAutoReports
Requirement	The system shall allow admins to schedule automated report generation.
Source	Admin
Rationale	Helps automate analytics tracking.
Business Rule	BR-95: Scheduled reports must be delivered via email.
Dependencies	FR-121
Priority	Medium

3.4 Non-Functional Requirements

This section specifies non-functional requirements other than constraints. These quality requirements should be specific, quantifiable and verifiable. The non-functional requirements related to our system include Usability, Performance, Security, Interoperability, and Integrity.

3.4.1 Usability

USE-1: 95% of users shall be able to navigate and use PodVerse within 5 minutes of initial interaction.

USE-2: The user interface shall be intuitive and accessible, ensuring that users can easily manage podcasts, interact with AI features, and explore content.

USE-3: The AI-powered features (e.g., AI Host, AI Guest, Podcast Summary) shall correctly interpret and respond to at least 90% of user queries.

USE-4: Users shall be able to initiate and maintain conversations, subscriptions, and interactions with minimal effort and latency.

USE-5: Error messages and system alerts should be clear, concise, and user-friendly to guide users in case of failures or incorrect inputs.

3.4.2 Performance

PER-1: AI-generated podcast summaries shall be generated within 10 seconds of a user request.

PER-2: The system shall start podcast recording within 3 seconds of user initiation.

PER-3: The system shall retrieve and load stored podcasts within 5 seconds of user request.

PER-4: Live interactions between AI and users in a podcast session shall maintain a response latency of less than 3 seconds.

3.4.3 Security

SEC-1: All passwords shall be stored in encrypted form.

SEC-2: Only authenticated users shall be allowed to create, edit, or publish podcasts.

SEC-3: User interactions, feedback, and messages shall be securely stored, ensuring privacy and data integrity.

SEC-4: The system shall implement role-based access control, restricting creator and viewer privileges appropriately.

SEC-5: All payment transactions shall be securely processed using encrypted connections.

3.4.4 Interoperability

IOP-1: PodVerse shall support audio file uploads in MP3 and WAV formats for podcast recording and voice cloning.

IOP-2: The system shall integrate with third-party authentication services such as Google for login.

IOP-3: PodVerse shall support video podcast exports in MP4 format.

3.4.5 Integrity

INT-1: The system shall protect against unauthorized modification, deletion, or corruption of podcast content.

INT-2: Uploaded podcast recordings shall remain unaltered and retrievable in their original form unless edited by the user.

INT-3: PodVerse shall support video podcast exports in MP4 format.

3.5 External Interface Requirements

3.5.1 User Interfaces Requirements

UI-1: PodVerse shall provide a light mode-friendly UI for a comfortable viewing experience.

UI-2: A left panel shall allow quick access to Dashboard, Podcasts, Messages, and Profile.

UI-3: A search bar shall be prominently displayed on the main dashboard for users to quickly find podcasts, creators, and topics.

UI-4: The system should provide real-time error messages and user-friendly notifications to guide users in resolving issues efficiently.

3.5.2 Software interfaces

SI-1: PodVerse shall integrate with Google OAuth API for secure user authentication via Google login.

SI-2: The system shall support MP3, WAV, and MP4 file formats for podcast recording, editing, and publishing.

3.5.3 Hardware interfaces

HI-1: The system shall allow access to a device's microphone for recording podcasts.

3.5.4 Communications interfaces

CI-1: The application shall provide notifications to creators and viewers about audience interactions, and direct messages, ensuring seamless engagement.

CI-2: The system shall enable instant messaging and calls between users using real-time communication protocols, ensuring low-latency interactions.

4. Design and Architecture

4.1 Architectural Design

The architectural design of PodVerse is modular, ensuring that each component of the system works together to achieve overall functionality. This section provides a high-level overview of how the system's modules collaborate to deliver the desired features.

4.1.1 MVC Architecture

We have adopted a MVC Architecture for the PodVerse application. This architecture style emphasizes the separation of concerns and facilitates modularity.

Presentation Layer (View):

This layer includes the user interface components that interact with the users, such as AudioCreation_View, VideoCreation_View, and ProfileSetting_View. It captures user input and displays output, enabling smooth navigation and interaction within the system.

Application Layer (Controller):

Controllers like AudioCreation_Controller, VoiceCloning_Controller, and Auth_Controller handle user actions, process inputs, coordinate between the view and model, and manage application logic such as podcast creation and authentication.

Business Logic Layer (Model):

This layer contains core logic and data models such as User, Podcast, VoiceSample, and Rating. It manages processing rules, validations, AI interactions, and handles the overall behavior of the system.

AI Services & External Layer:

This layer integrates external services like the Voice Cloning Engine, LLMs, RAG Retrieval, and Subscription APIs. It has intelligent features such as AI guest responses, voice synthesis, and content recommendations.

Data Access Layer:

It abstracts database interactions and performs CRUD operations through repositories. It accesses data stored in relational databases, vector stores, and cloud storage to support both structured and unstructured data needs.

Database Layer:

This layer consists of the actual storage systems, including relational databases for core data, vector stores for AI embeddings, and cloud storage for media files like audio and video podcasts.

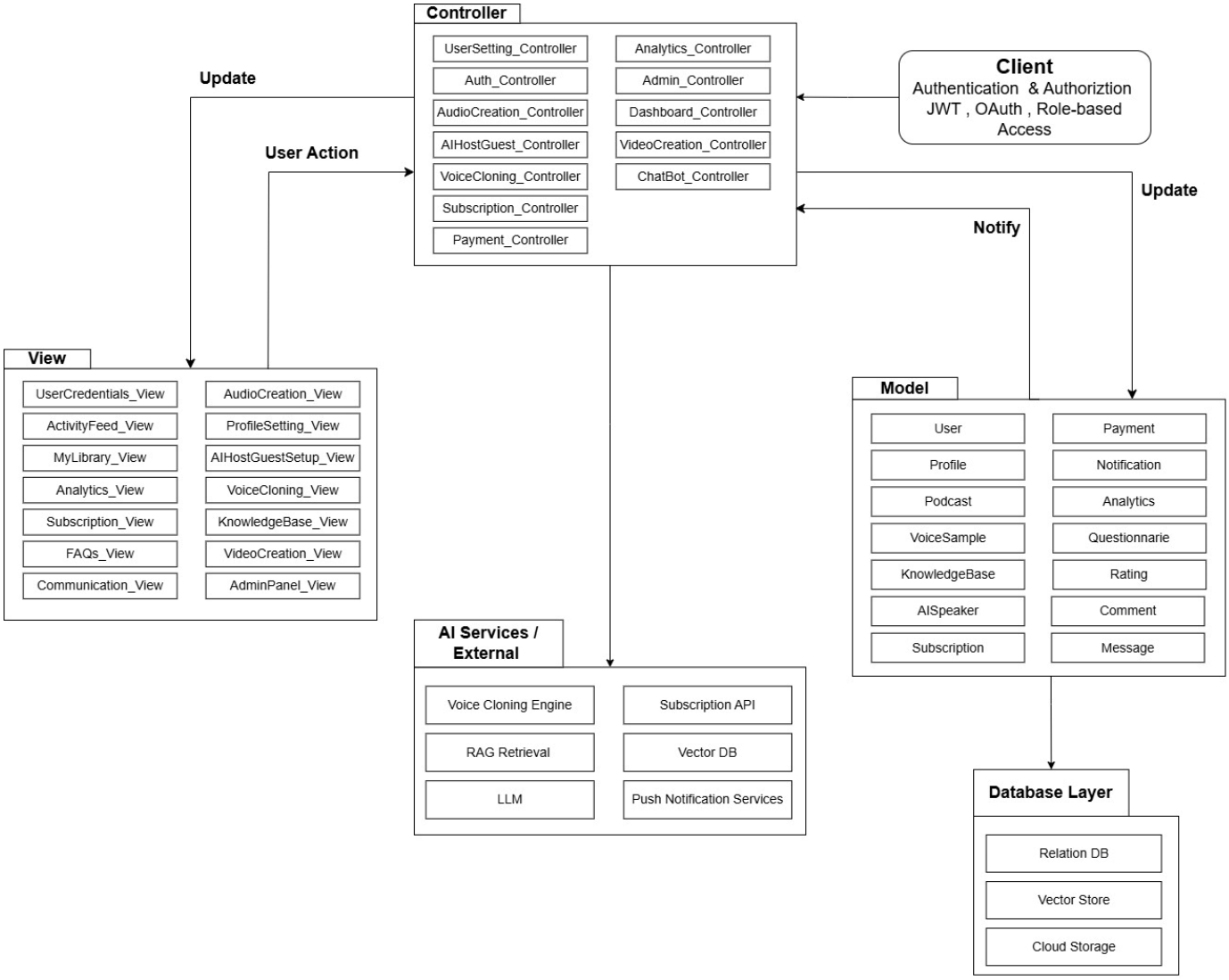


Figure 4: Architecture Diagram for PodVerse

4.2 Design Models

This section shows the design models for the system.

4.2.1 Activity Diagrams

4.2.1.1 Register Account

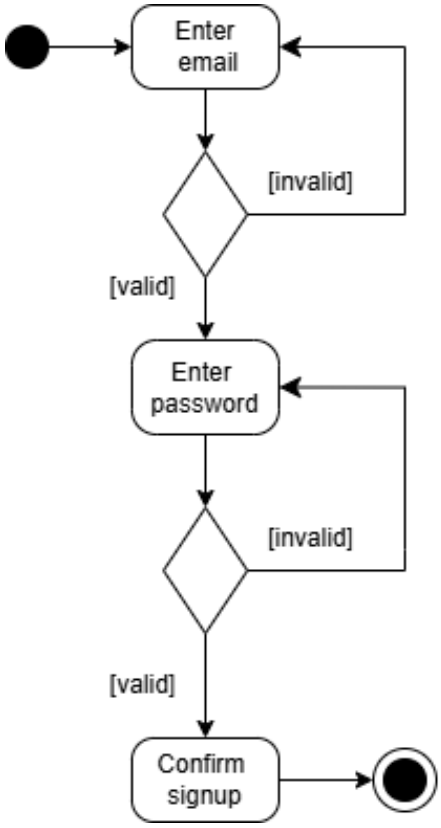


Figure 5: Activity Diagram for Sign up

4.2.1.2 Login

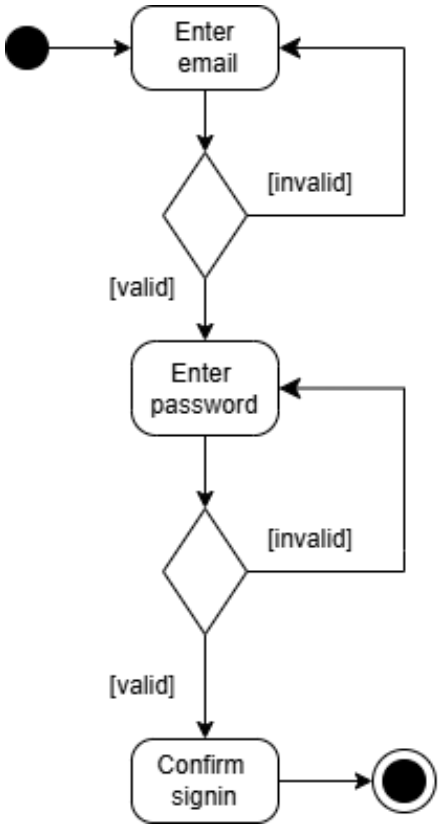


Figure 6: Activity Diagram for Sign in

4.2.1.3 Edit Profile

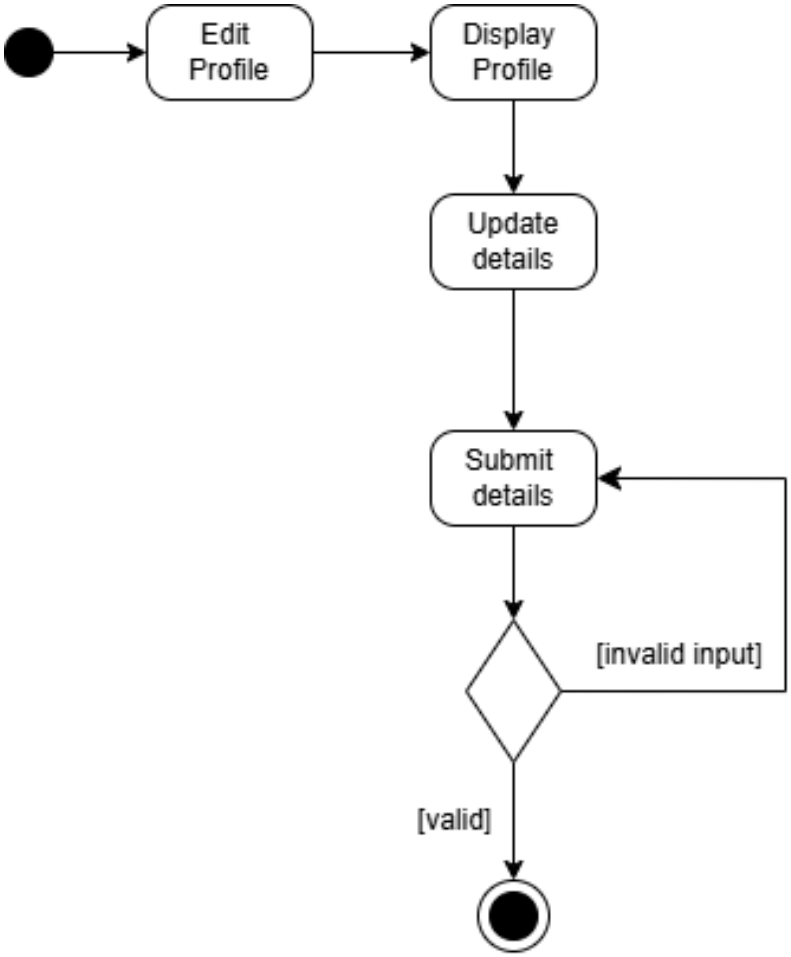


Figure 7: Activity Diagram for Edit Profile

4.2.1.4 Voice Cloning

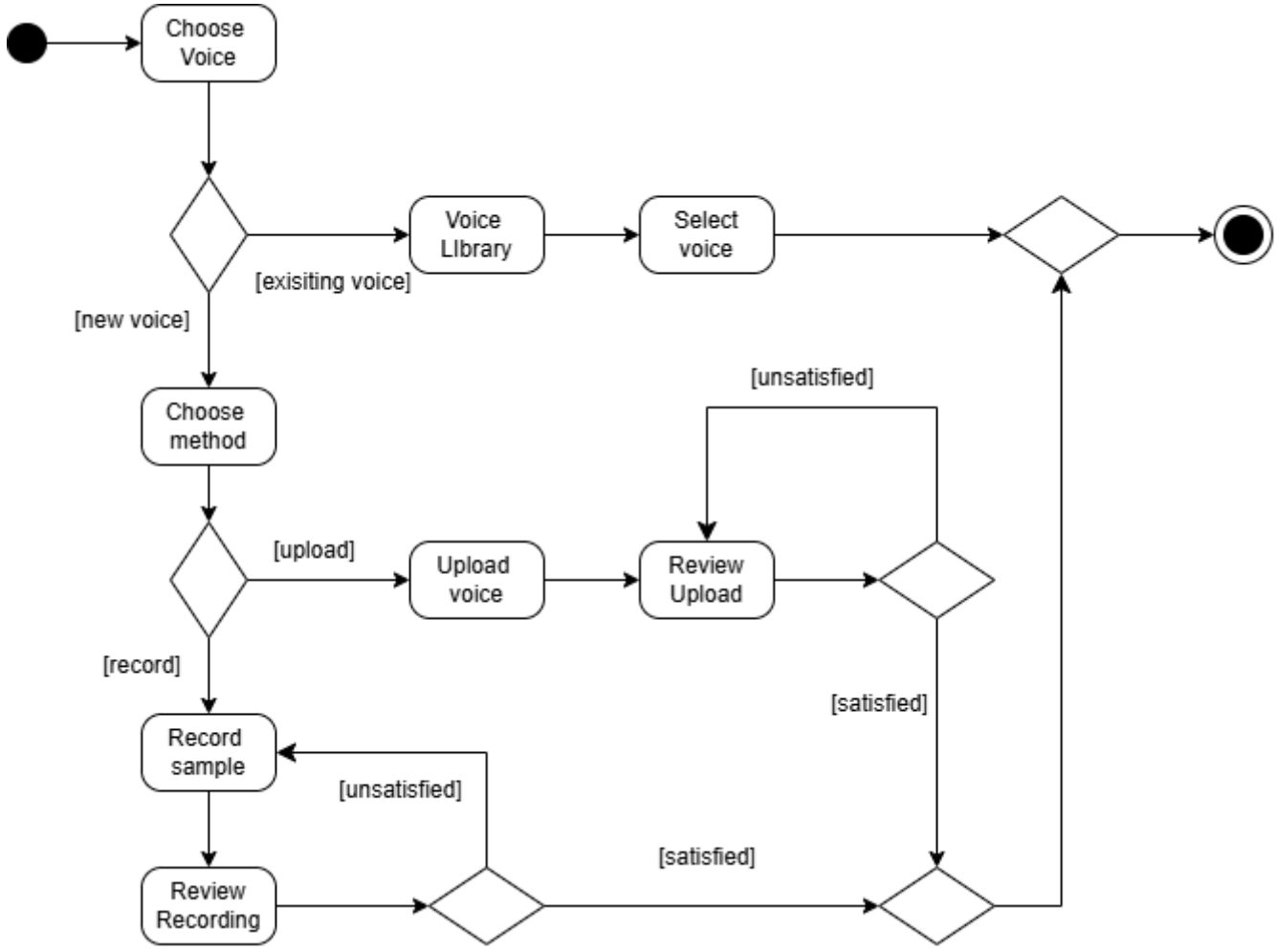


Figure 8: Activity Diagram for Voice Cloning

4.2.1.5 Knowledge Base Setup

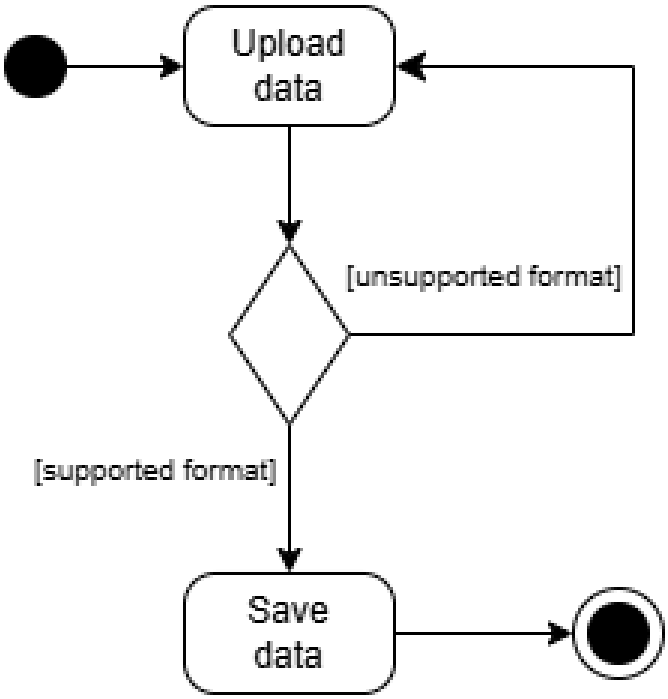


Figure 9: Activity Diagram for Knowledge Base Setup

4.2.1.6 AI Speaker Customization

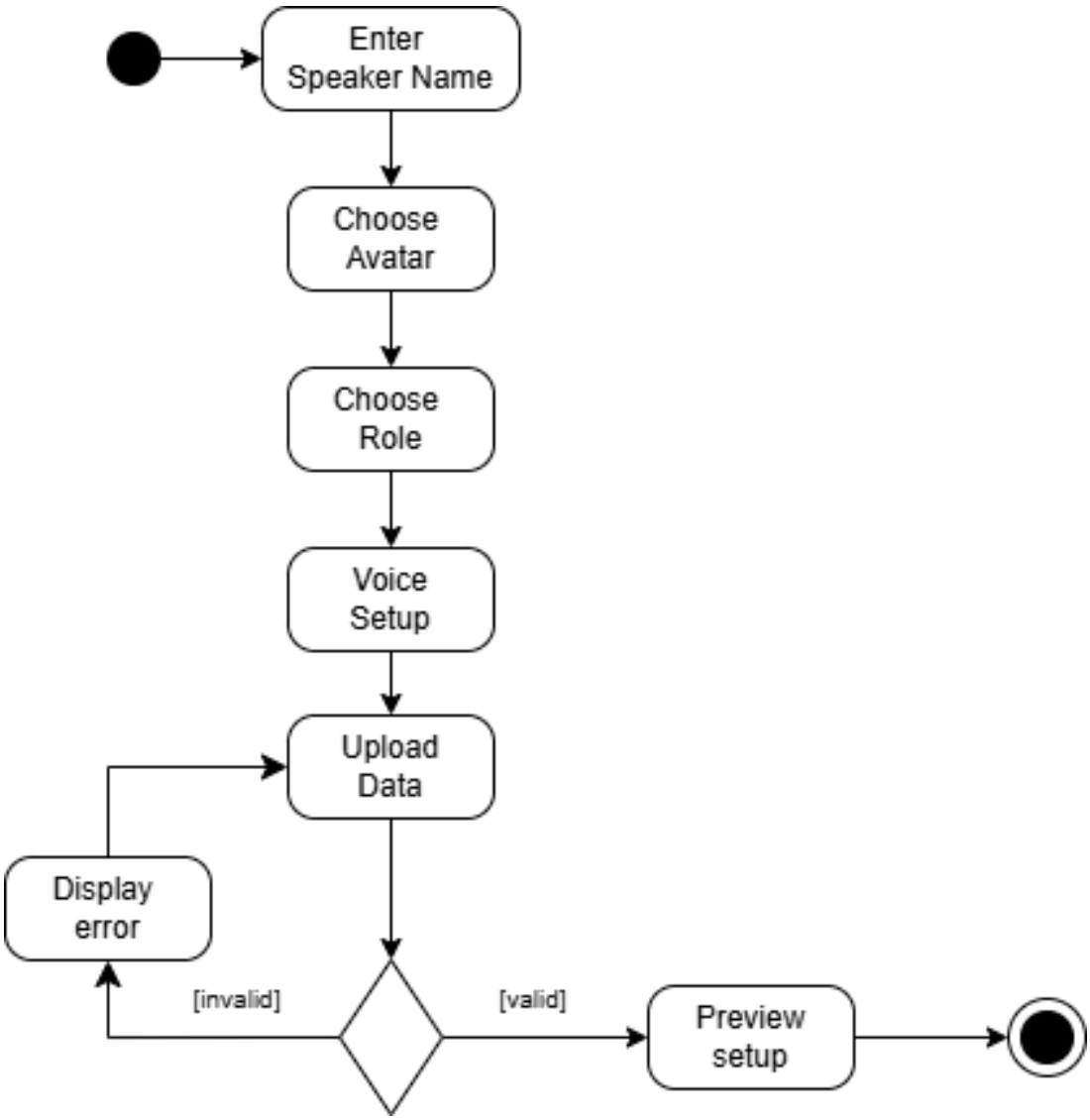


Figure 10: Activity Diagram for AI Speaker Customization

4.2.1.7 Podcast Audio Creation



Figure 11: Activity Diagram for Podcast Audio Creation

4.2.1.8 Podcast Video Creation

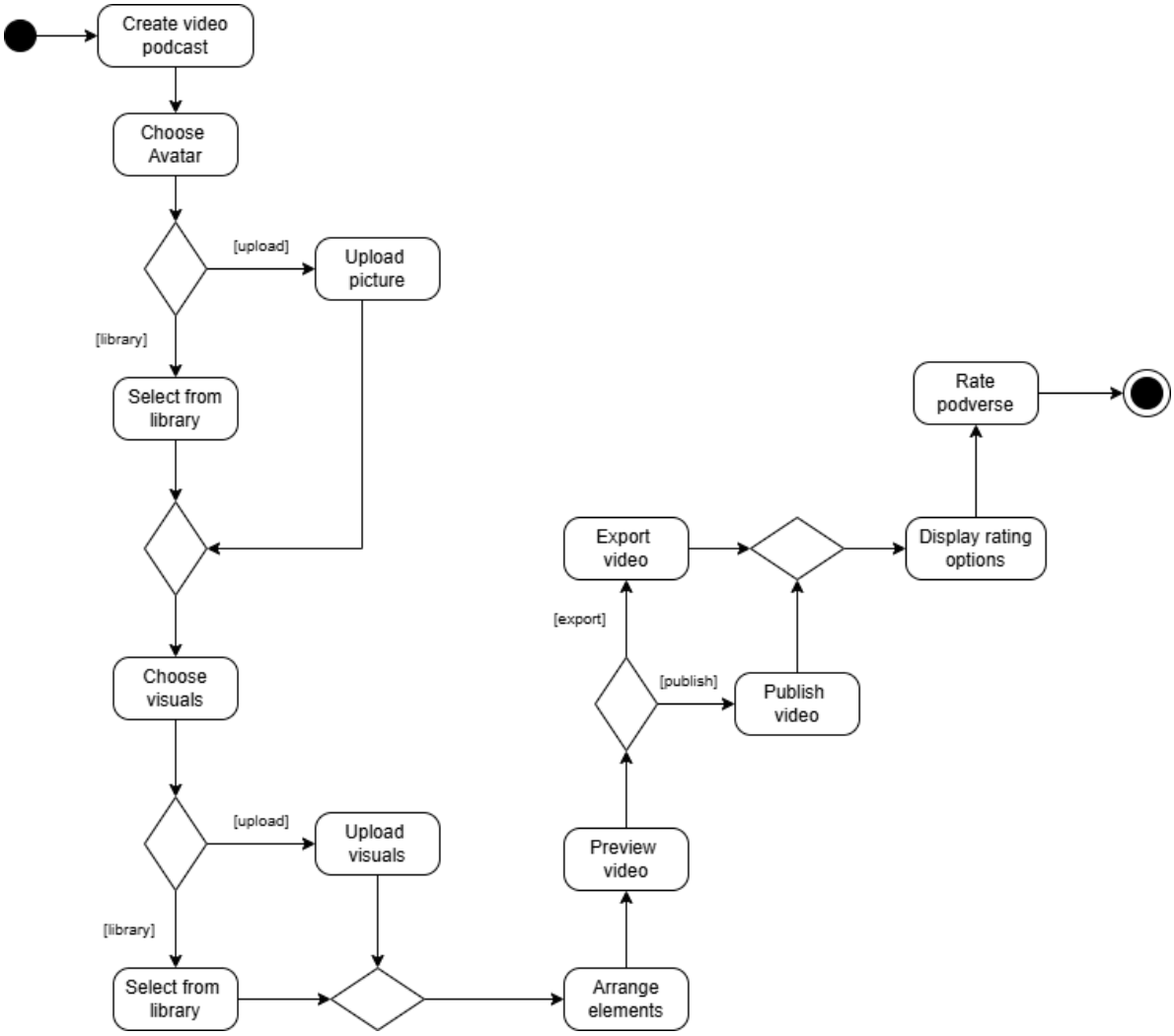


Figure 12: Activity Diagram for Podcast Video Creation

4.2.1.9 Request Podcast Summary

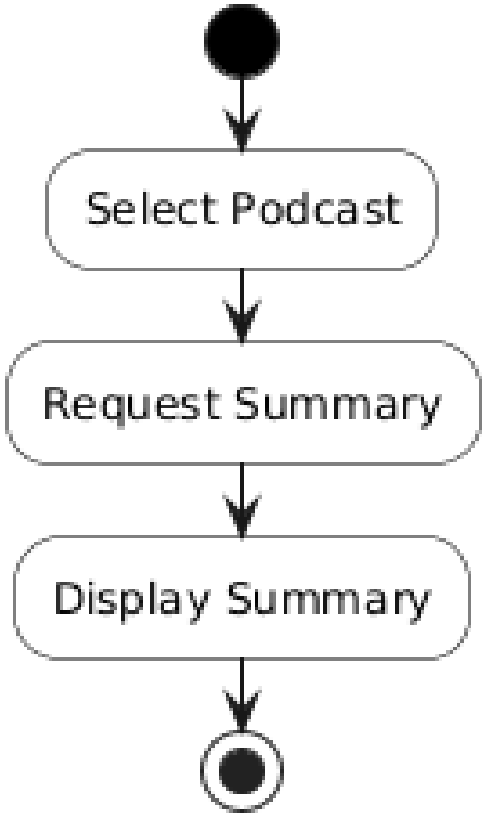


Figure 13: Activity Diagram for Podcast Summary

4.2.1.10 Subscription and Payment

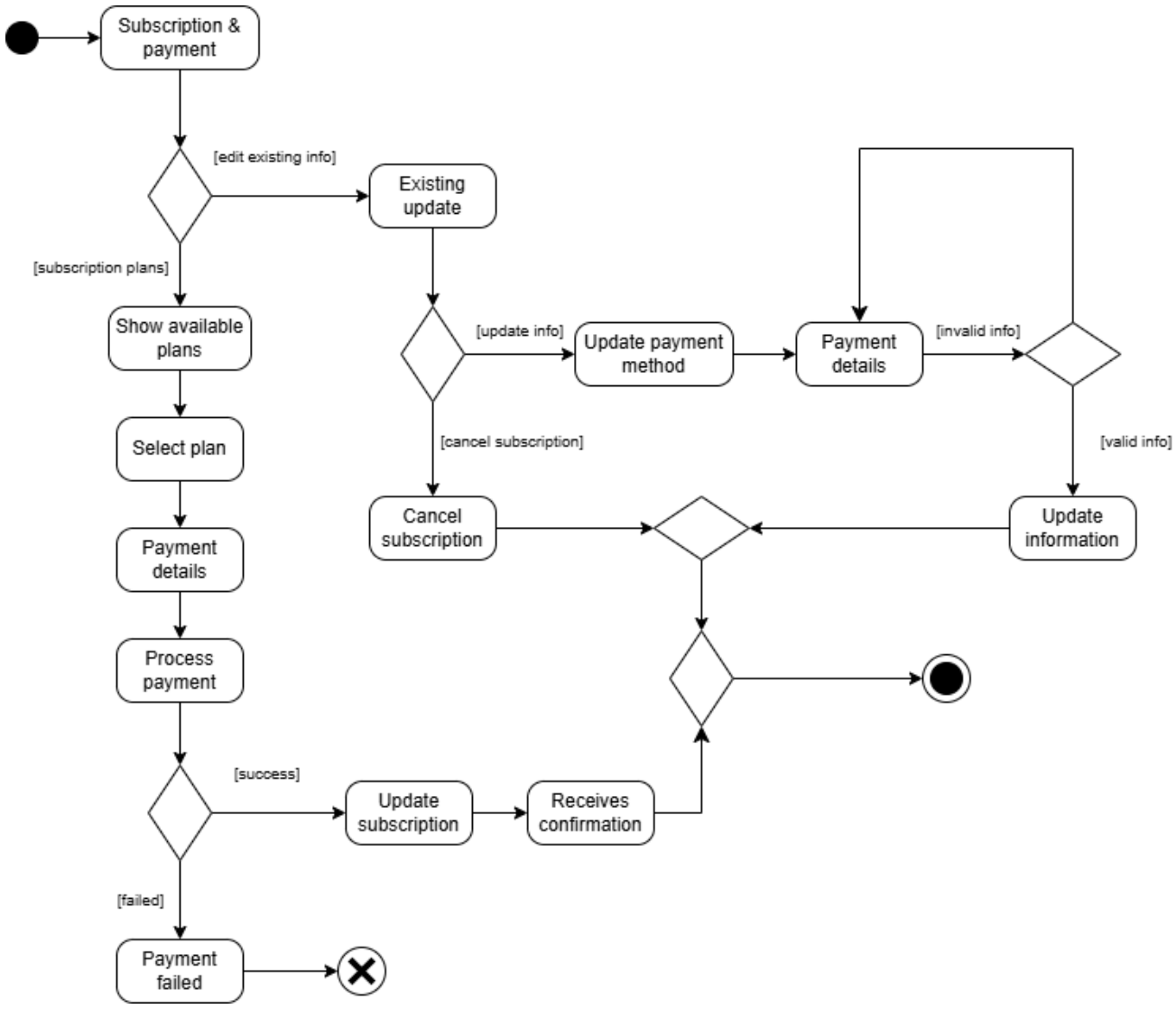


Figure 14: Activity Diagram for Subscription and Payment

4.2.1.11 FAQs Chatbot

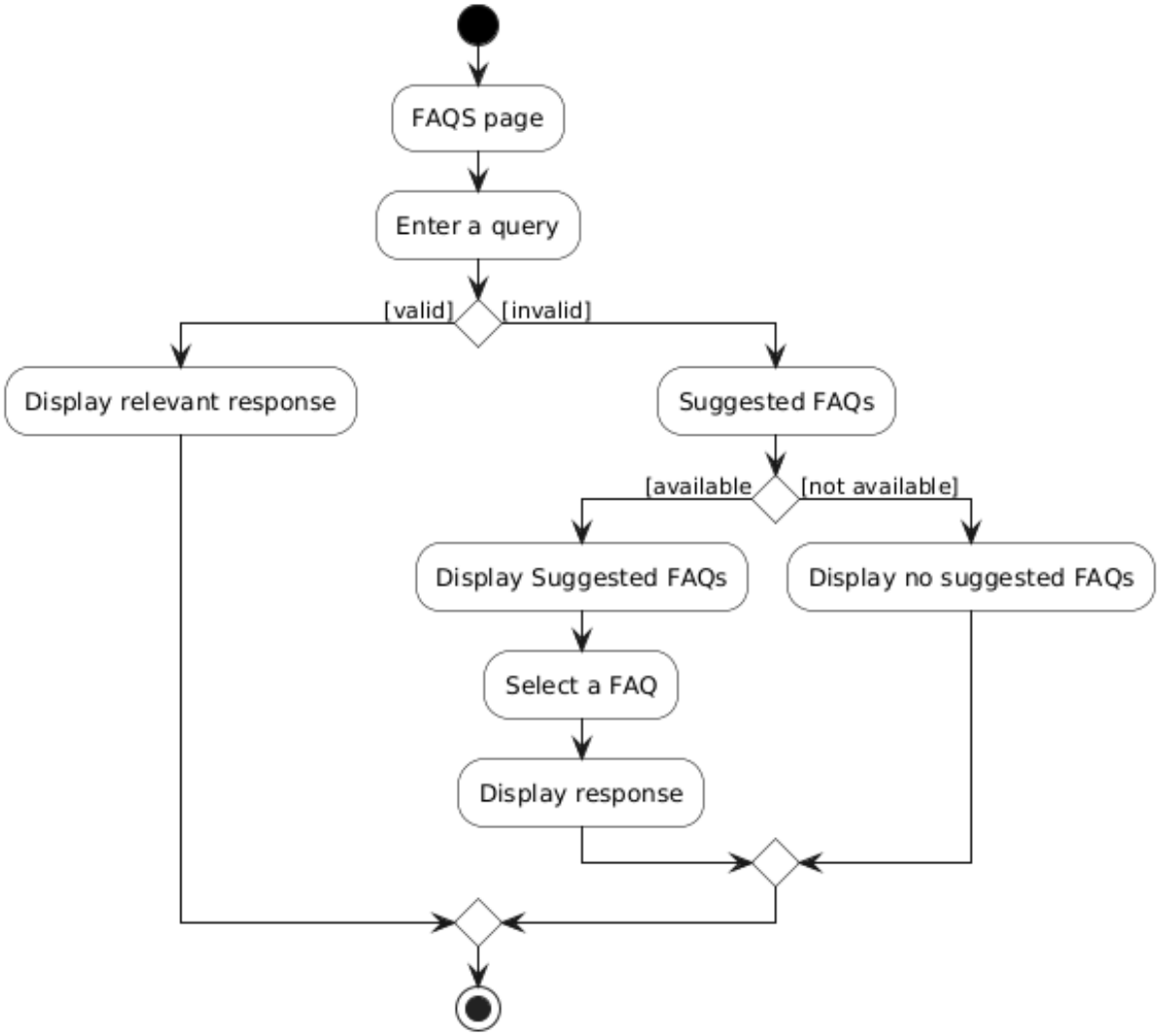


Figure 15: Activity Diagram for FAQs Chatbot

4.2.1.12 User-to-User Messaging and Calls

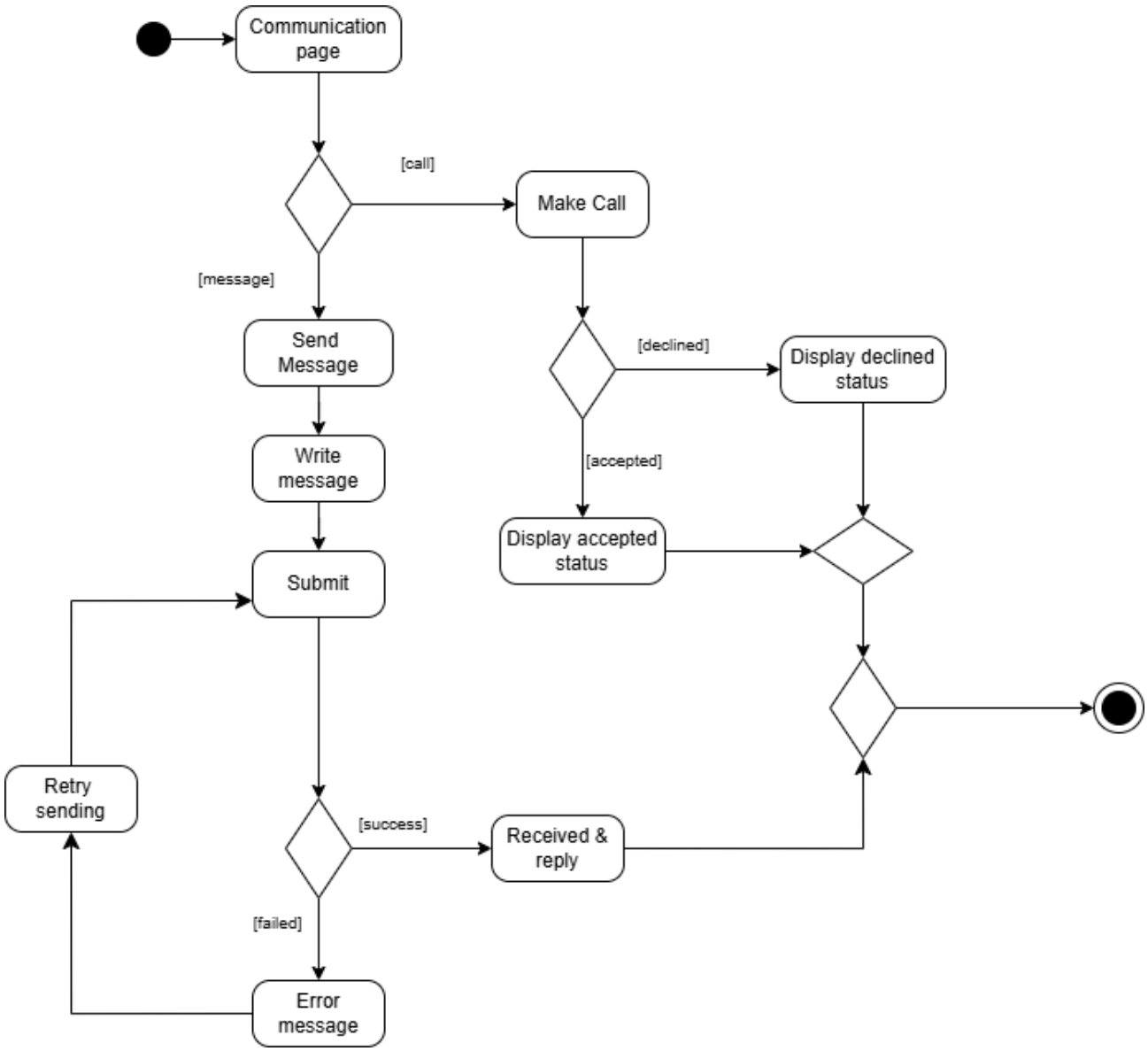


Figure 16: Activity Diagram for User-to-User Messaging and Calls

4.2.1.13 Manage Users and Subscriptions

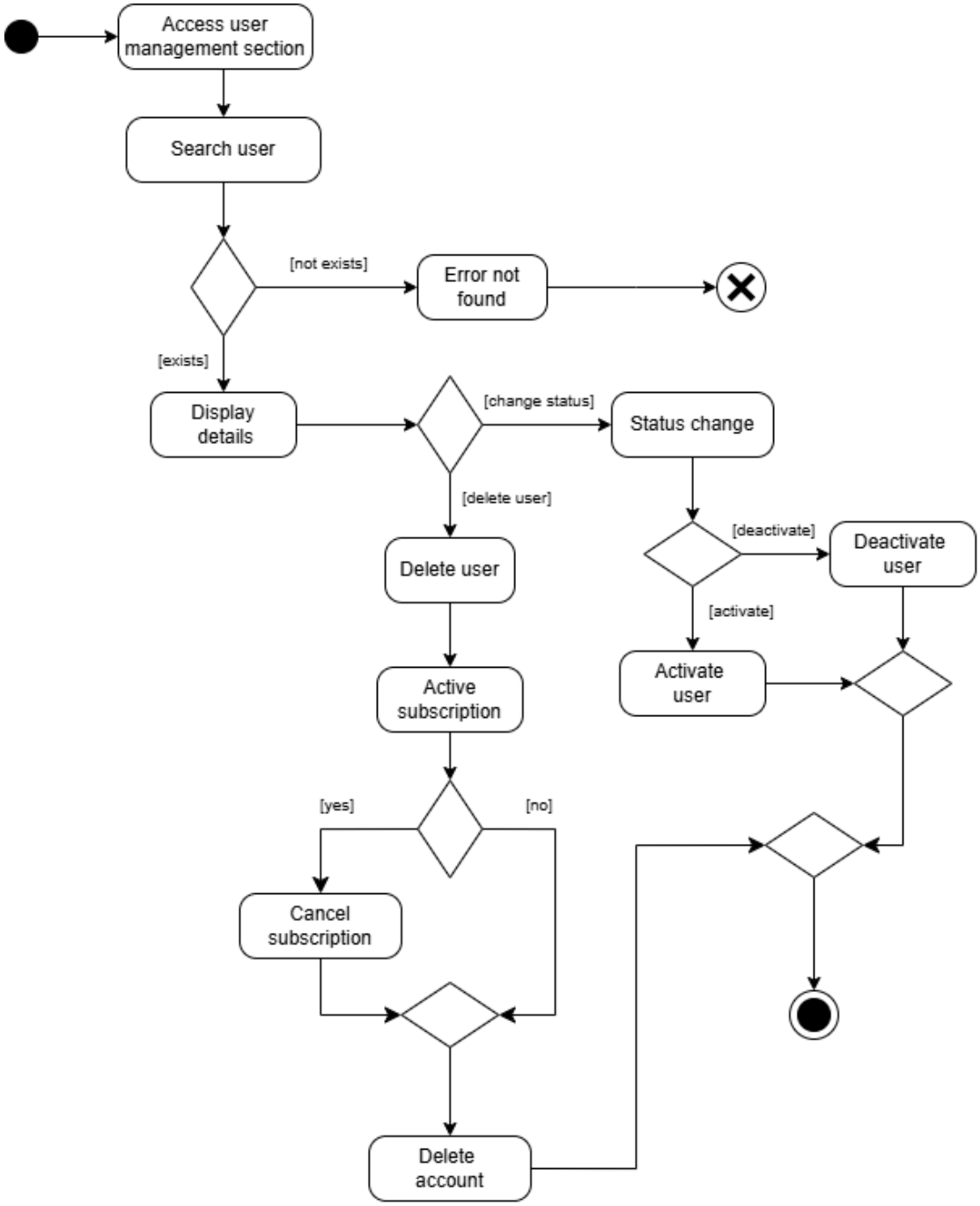


Figure 17: Activity Diagram for Managing Users and Subscriptions

4.2.1.14 Logout

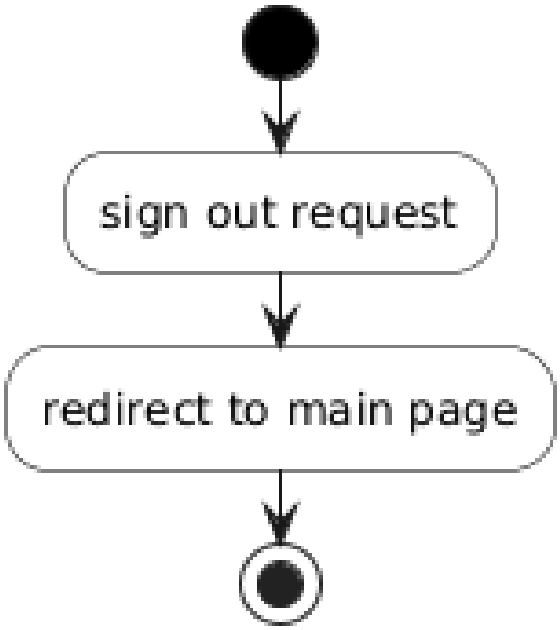


Figure 18: Activity Diagram for Sign out

4.2.2 Class Diagram

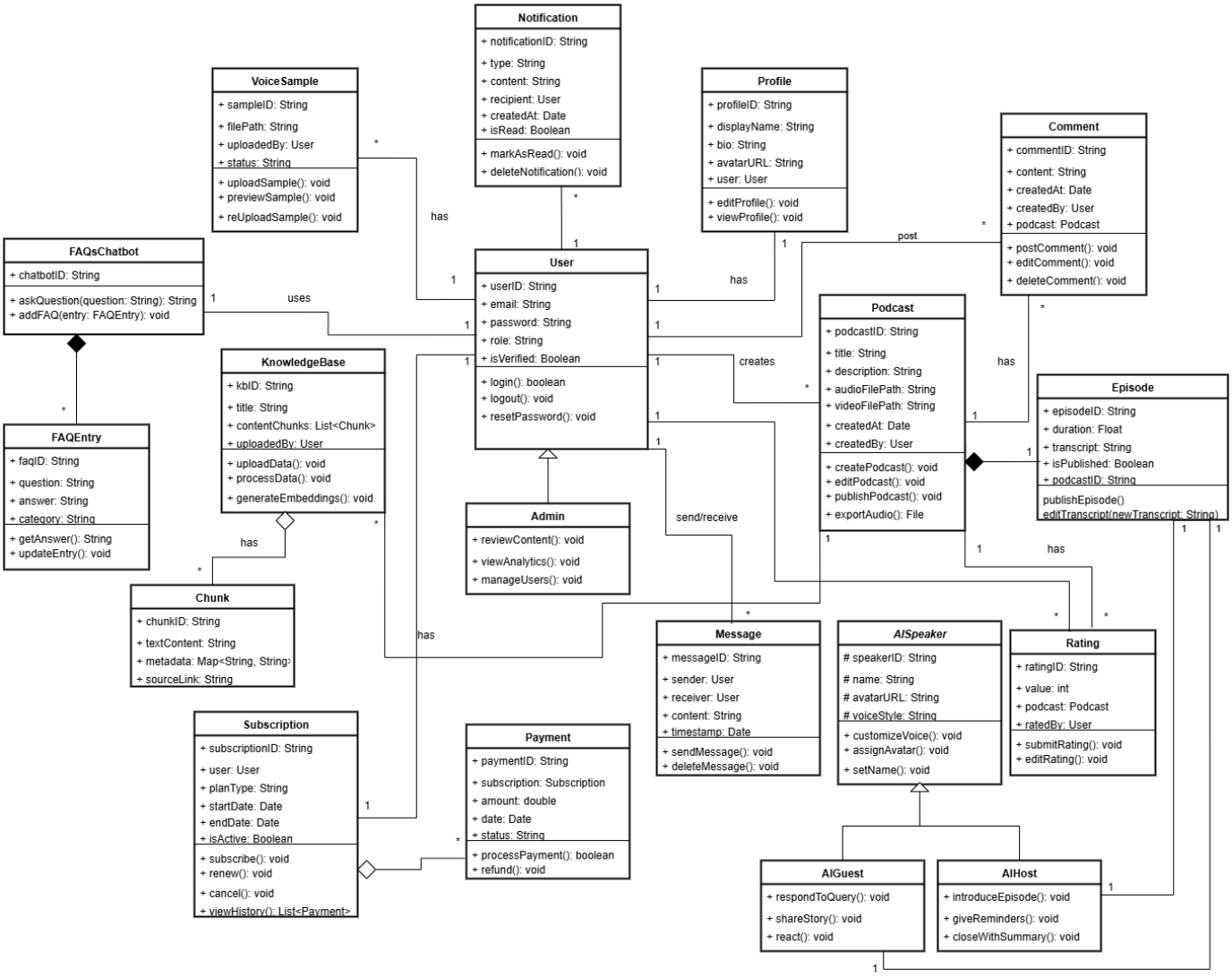


Figure 19: Class Diagram for PodVerse

4.2.3 Sequence Diagrams

Following are the sequence diagrams of PodVerse:

4.2.3.1 Module 1: User Authentication

Sequence diagram of module 1.

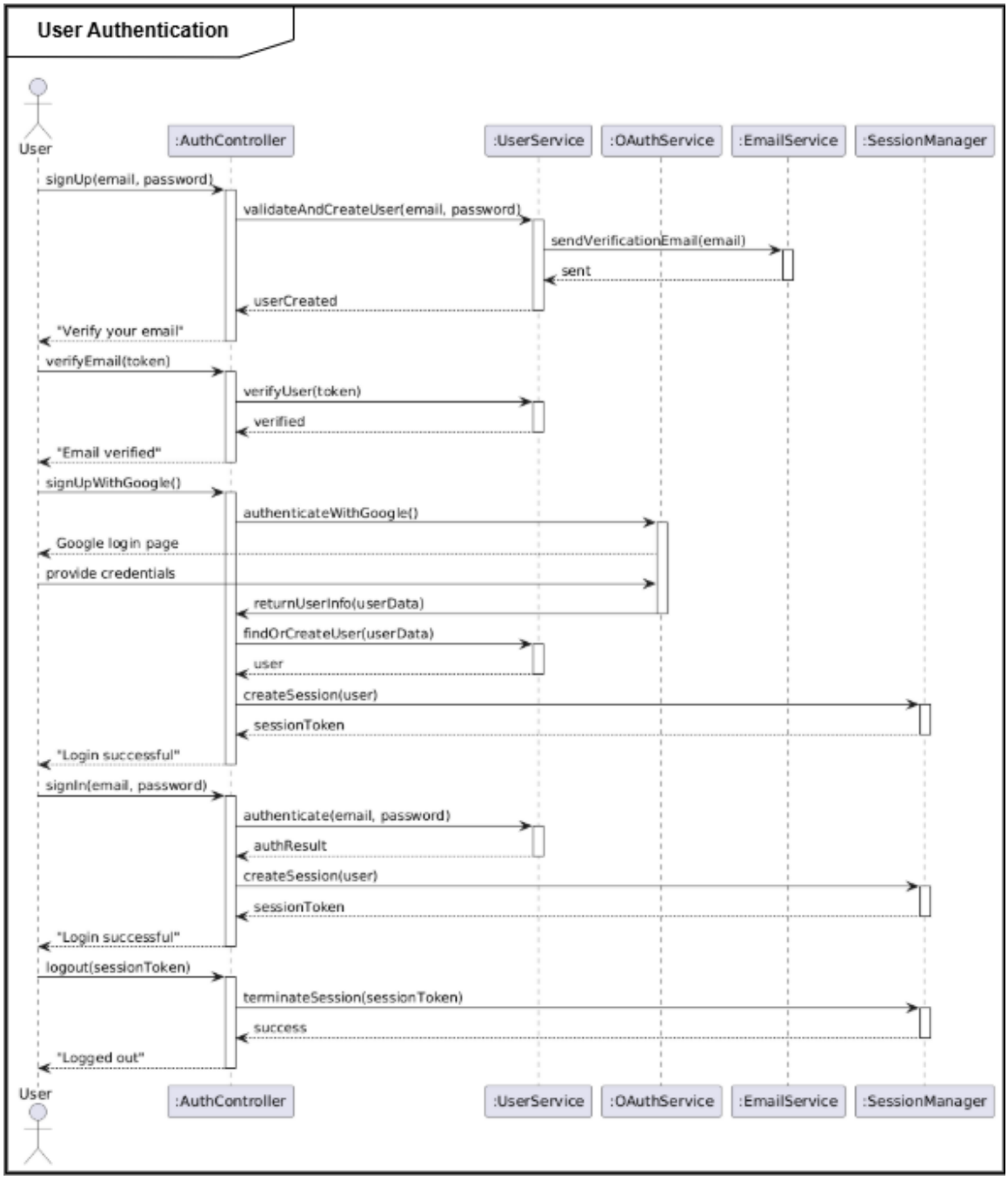


Figure 20: Sequence diagram of module 1

4.2.3.2 Module 2: Voice Cloning

Sequence diagram of module 2.

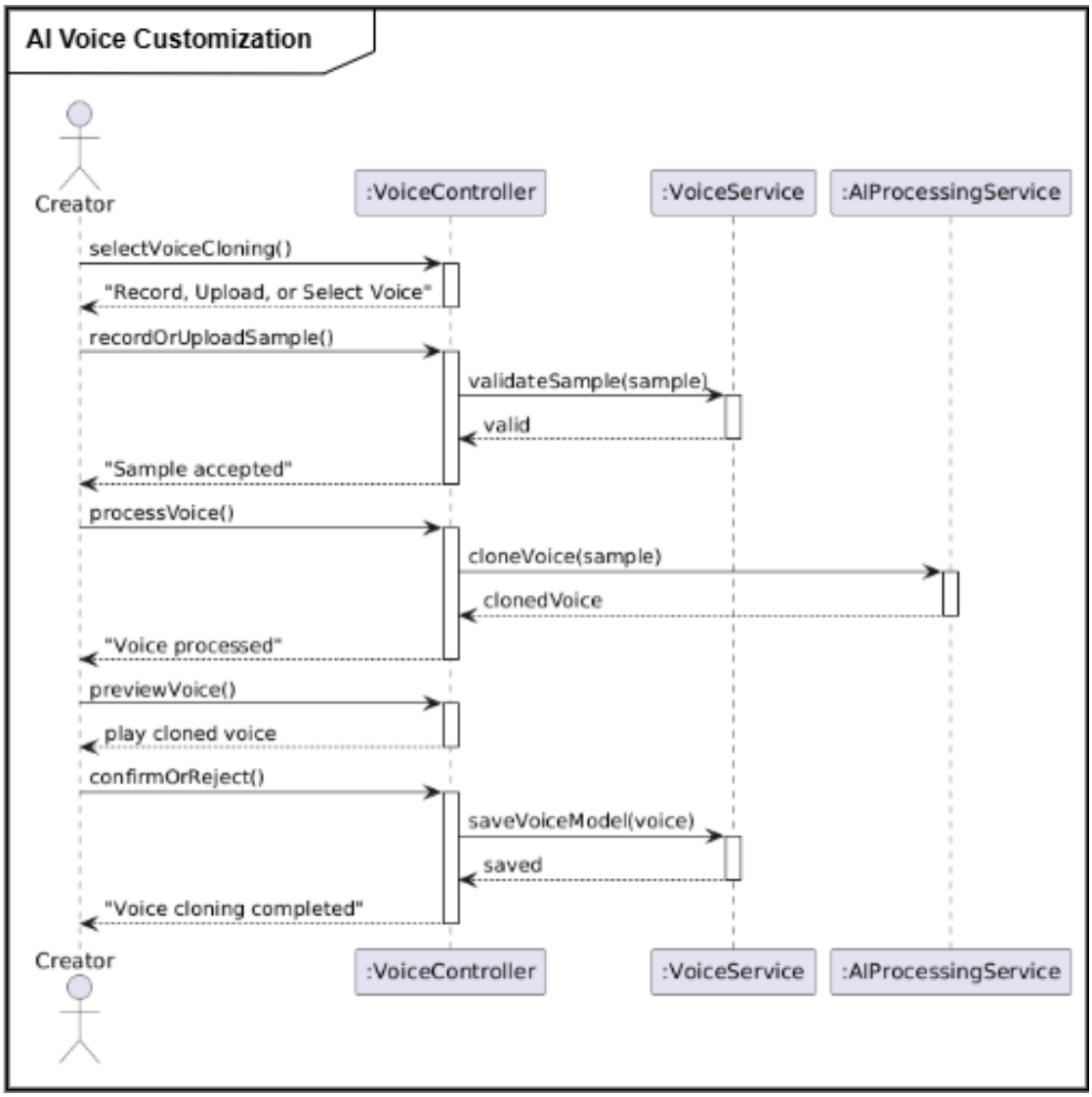


Figure 21: Sequence diagram of module 2

4.2.3.3 Module 3: Knowledge Base Setup

Sequence diagram of module 3.

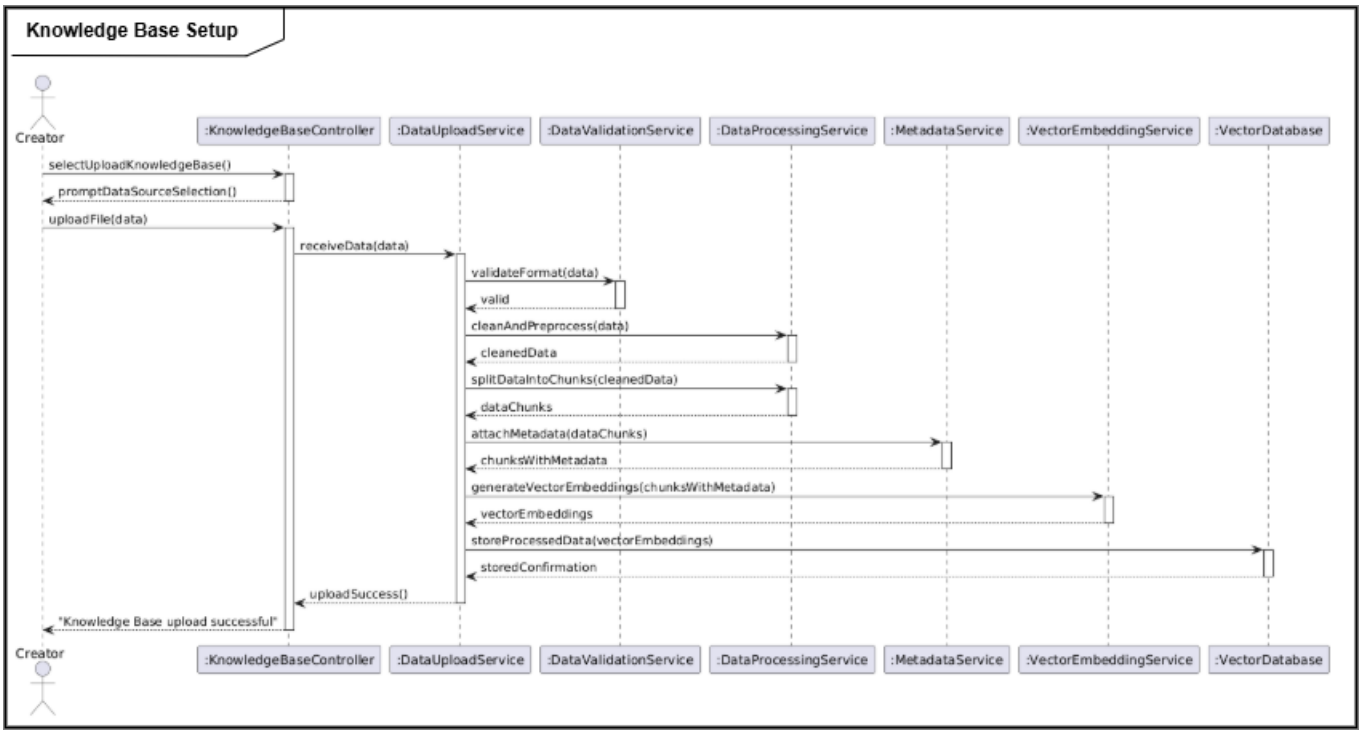


Figure 22: Sequence diagram of module 3

4.2.3.4 Module 4: RAG Retrieval

Sequence diagram of module 4.

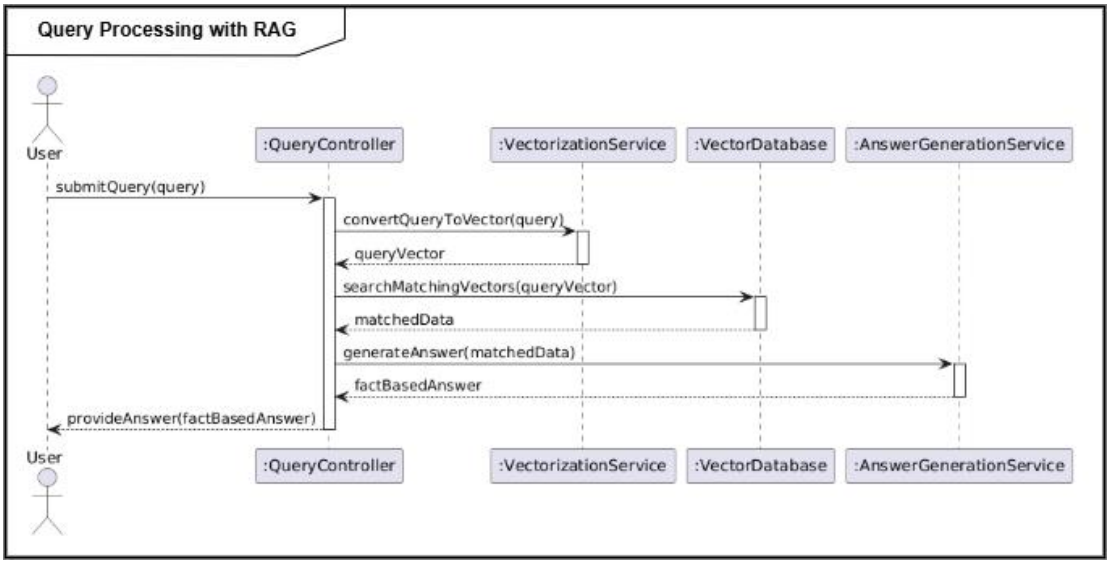


Figure 23: Sequence diagram of module 4

4.2.3.5 Module 6: Podcast Audio Creation

Sequence diagram of module 6.

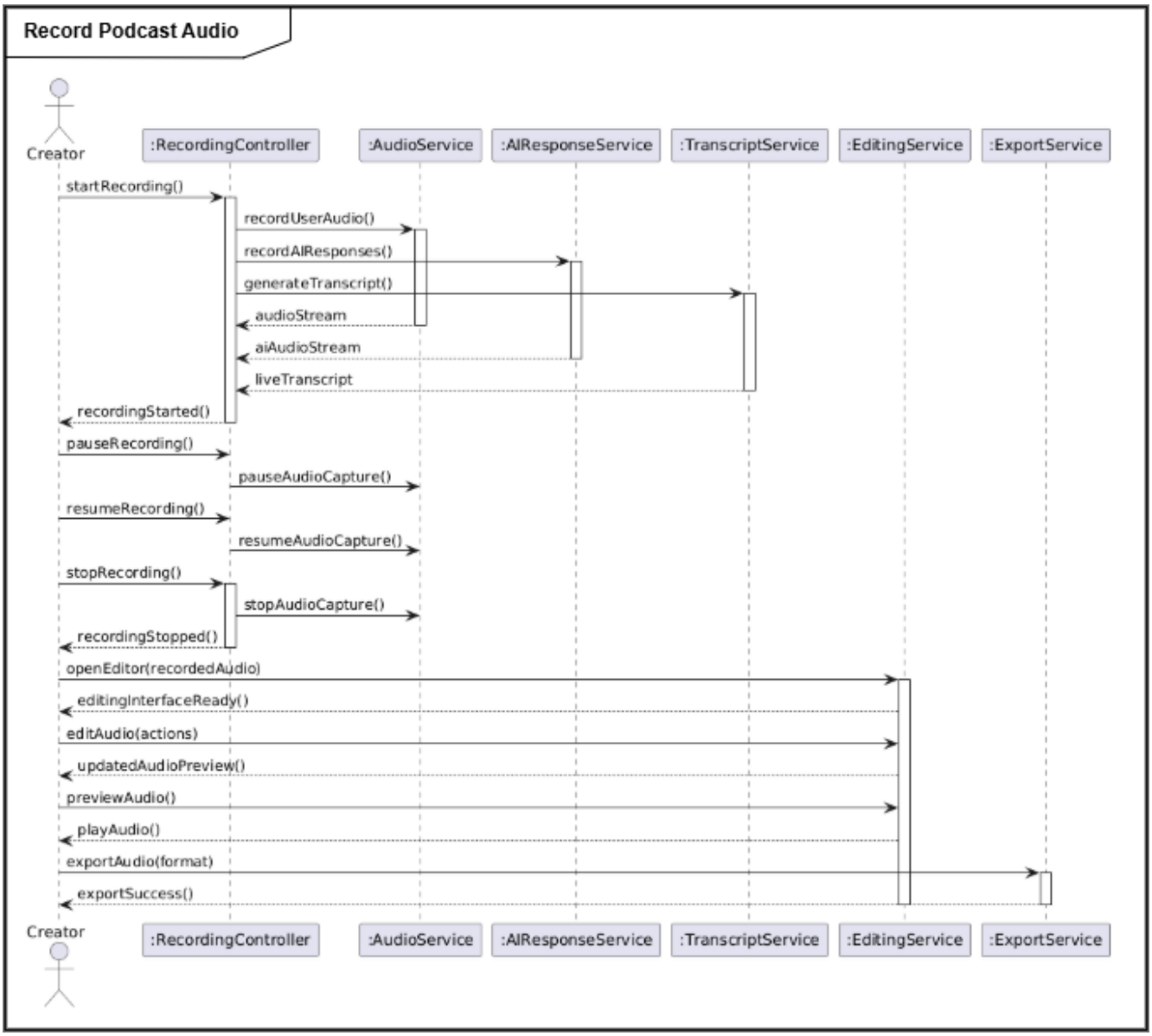


Figure 24: Sequence diagram of module 6

4.2.3.6 Module 9: Podcast Video Creation and Publishing

Sequence diagram of module 9.

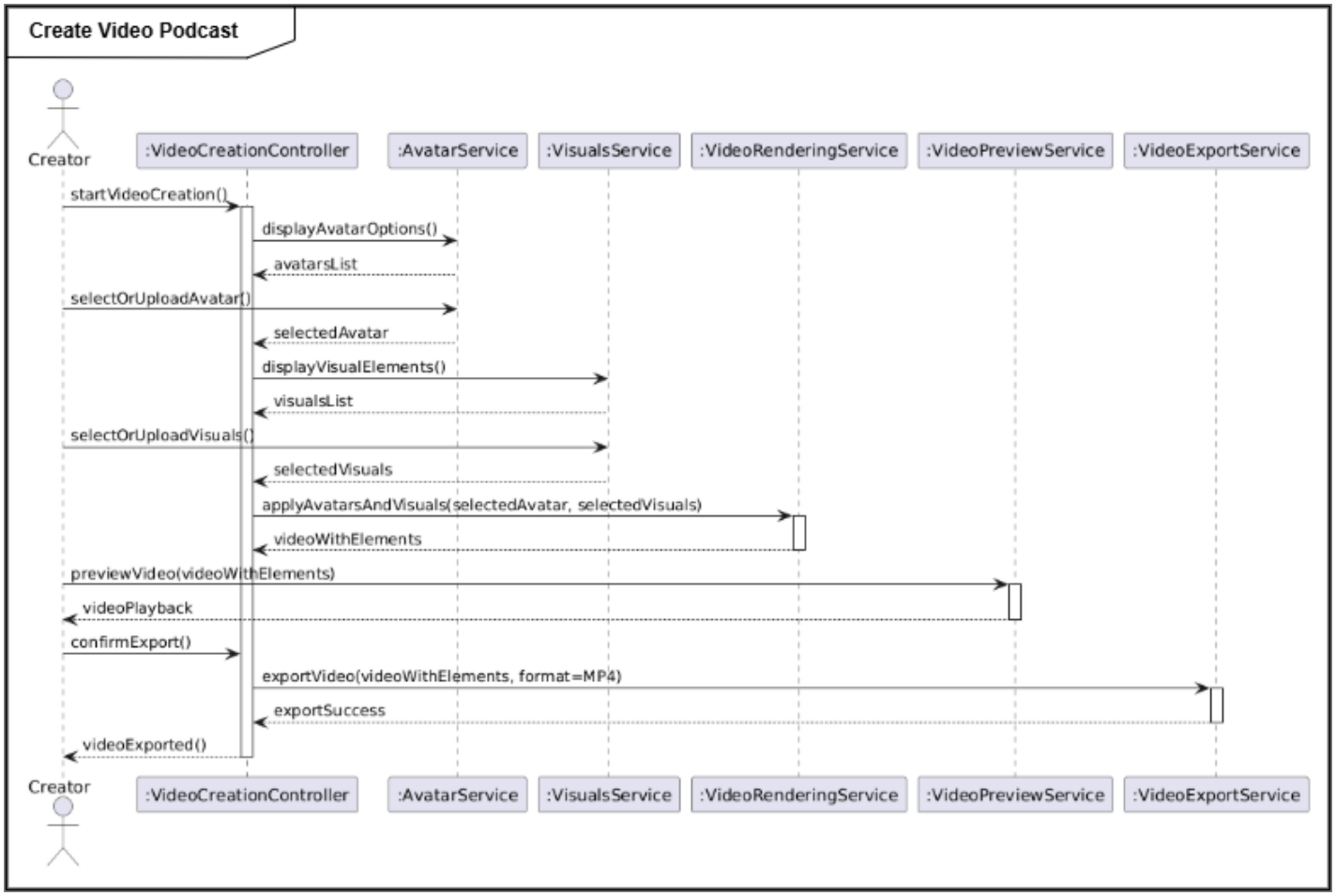


Figure 25: Sequence diagram of module 9

4.2.3.7 Module 10: Subscription and Payment

Sequence diagram of module 10.

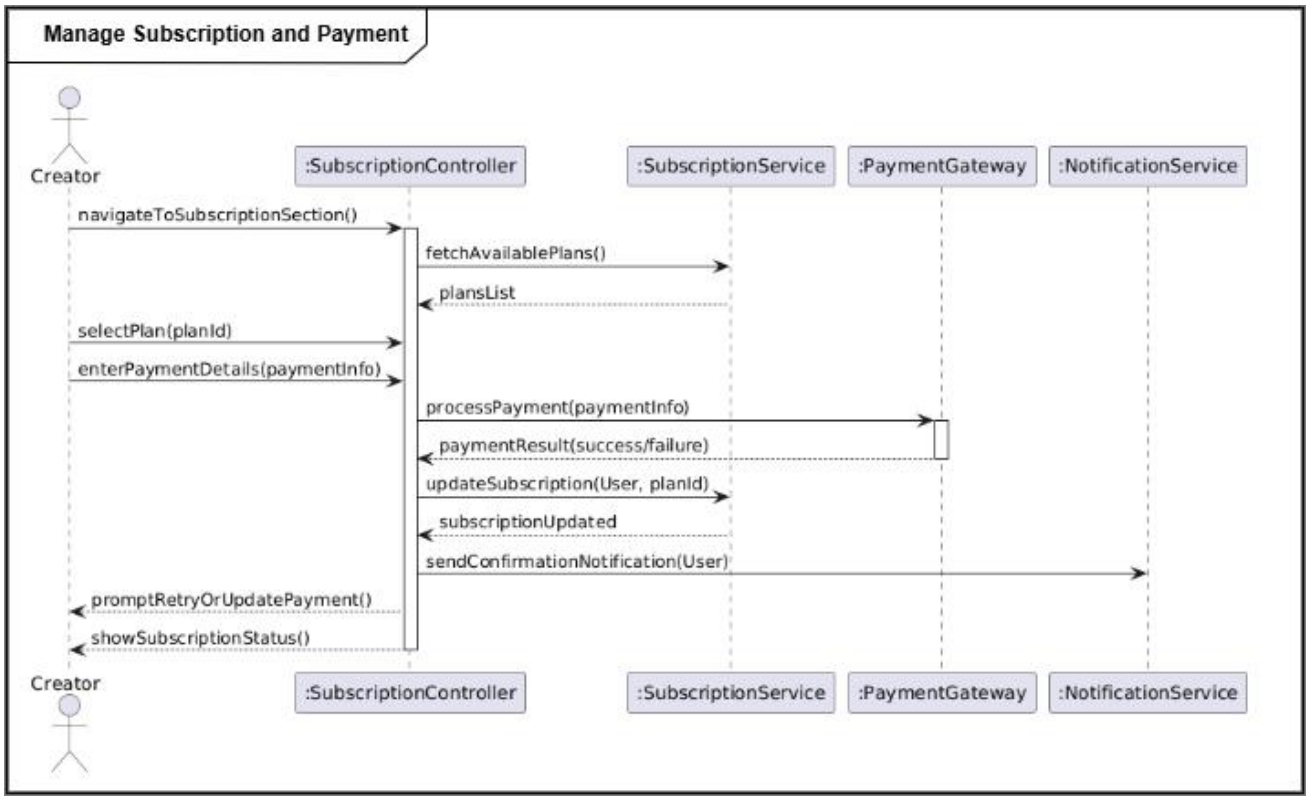


Figure 26: Sequence diagram of module 10

4.2.3.8 Module 11: FAQs Chatbot

Sequence diagram of module 11.

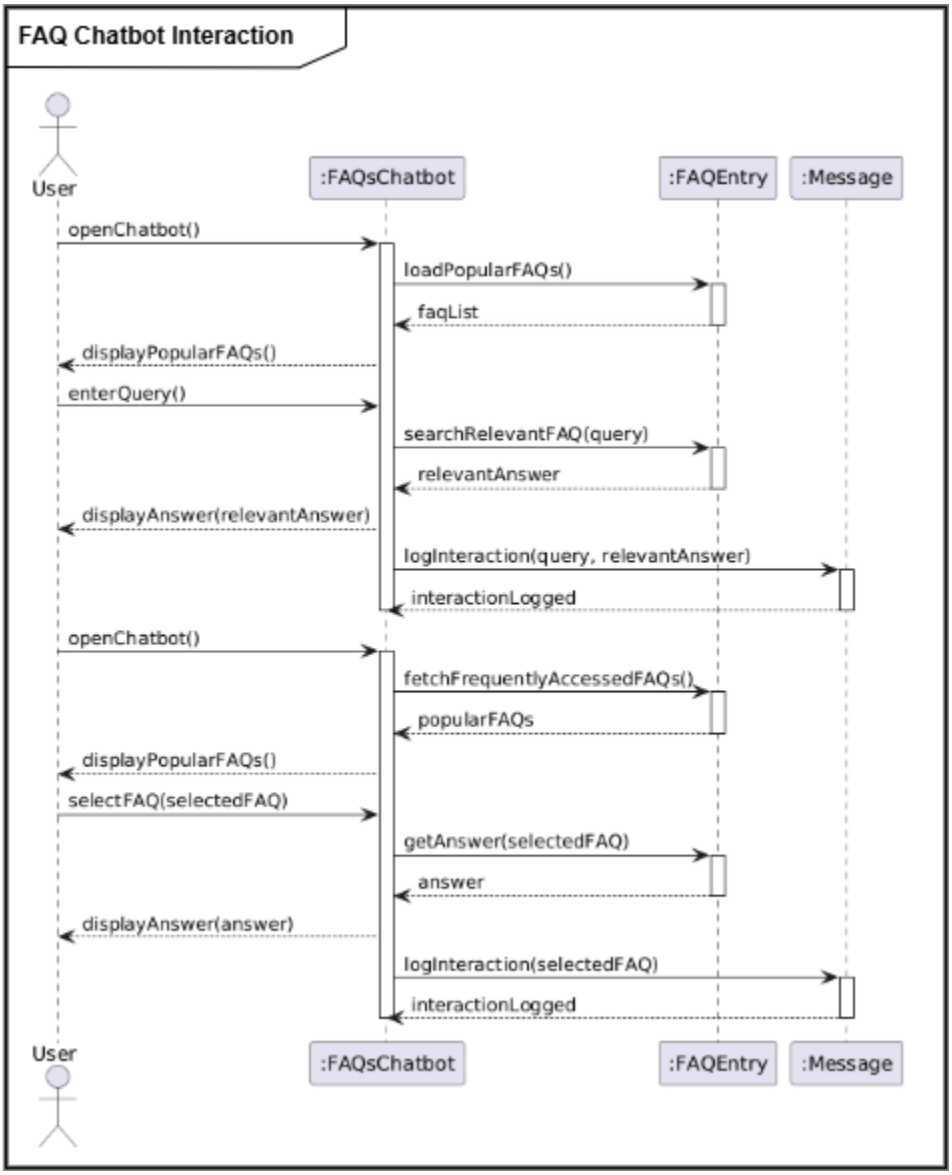


Figure 27: Sequence diagram of module 11

4.2.3.9 Module 12: Audience Engagement

Sequence diagram of module 12.

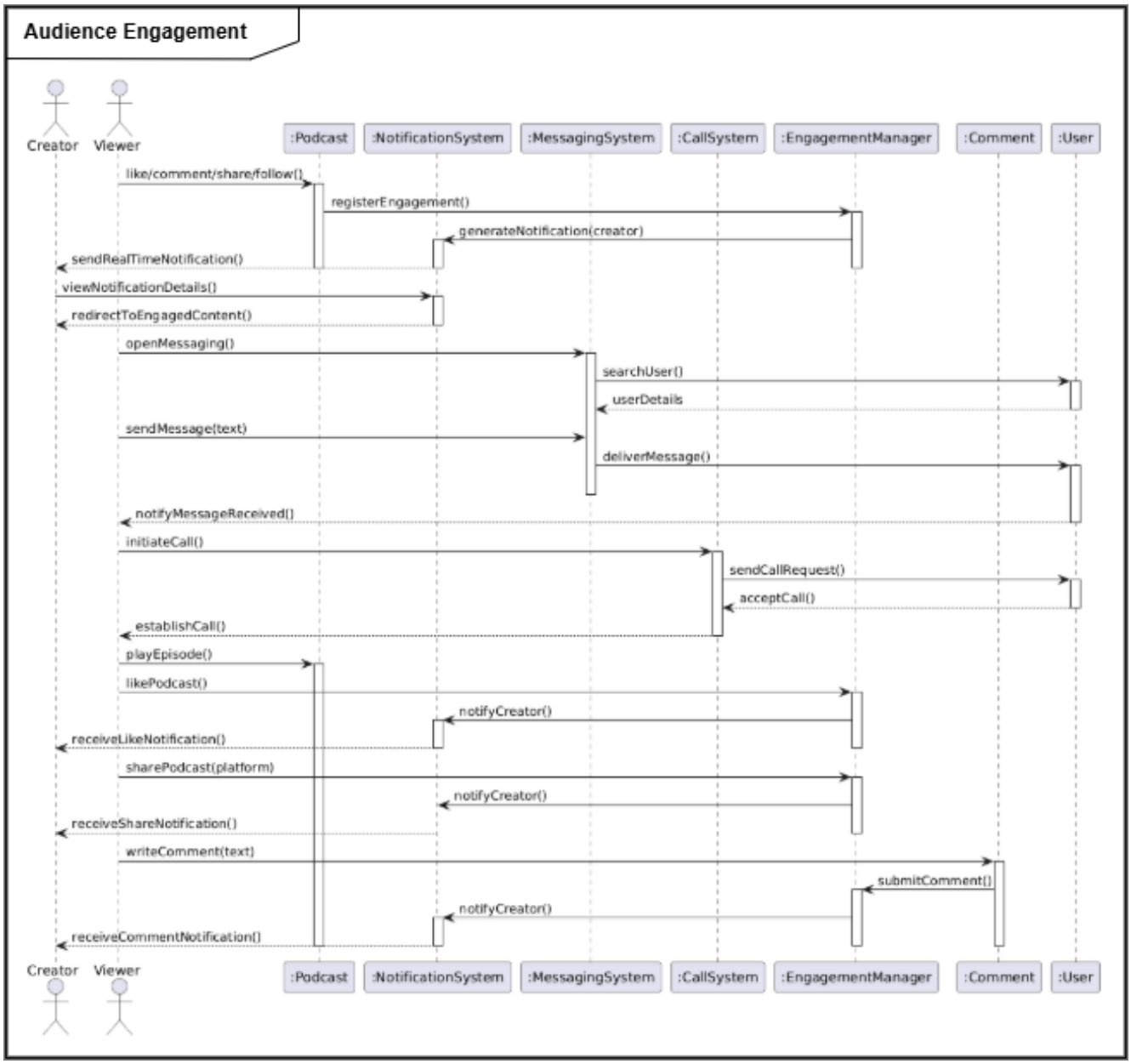


Figure 28: Sequence diagram of module 12

4.2.3.10 Module 13: Admin Panel and Analytics

Sequence diagram of module 13.

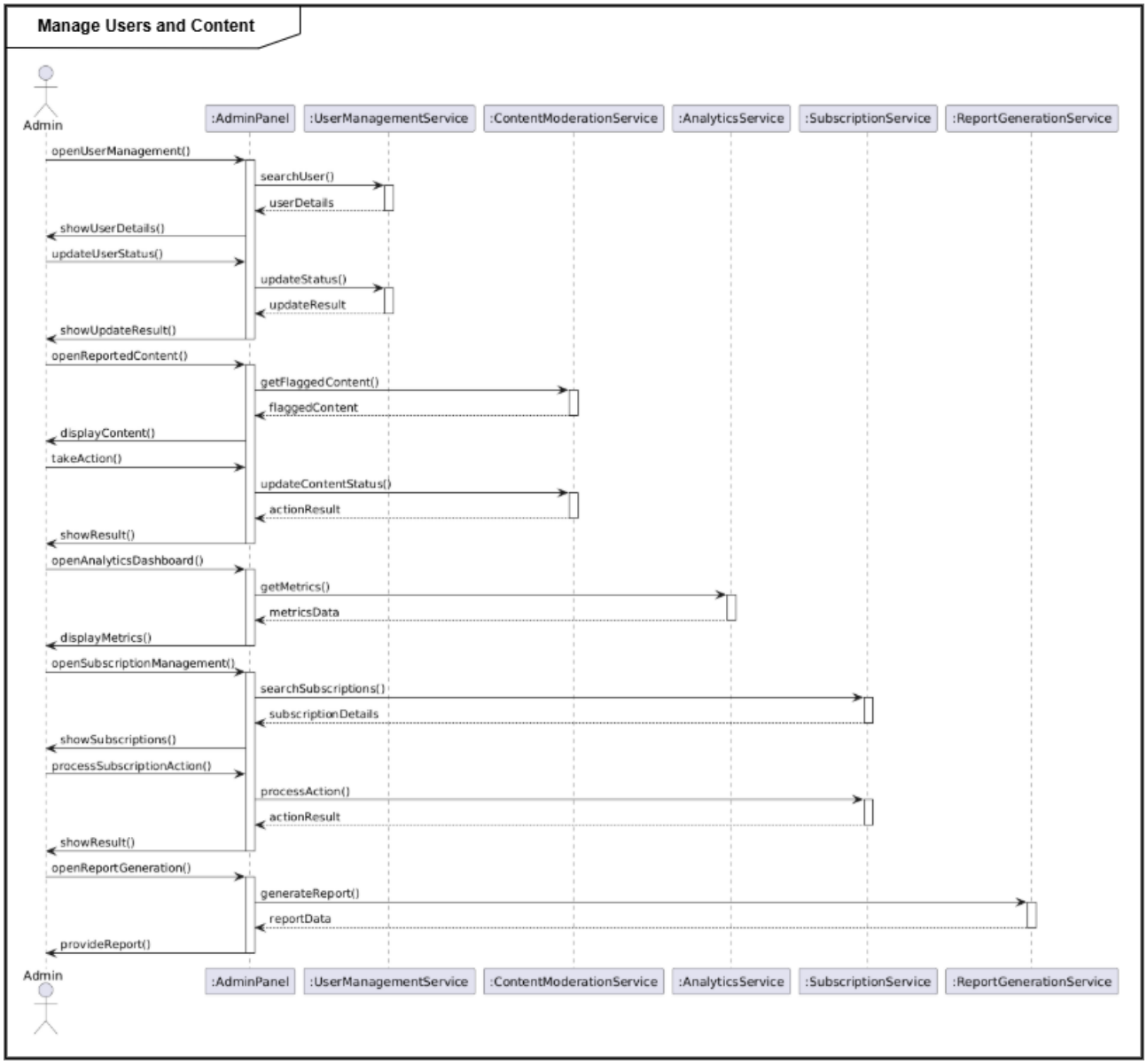


Figure 29: Sequence diagram of module 13

4.2.4 State Transition Diagrams

The following are the state transition diagrams of the backend processes of our system:

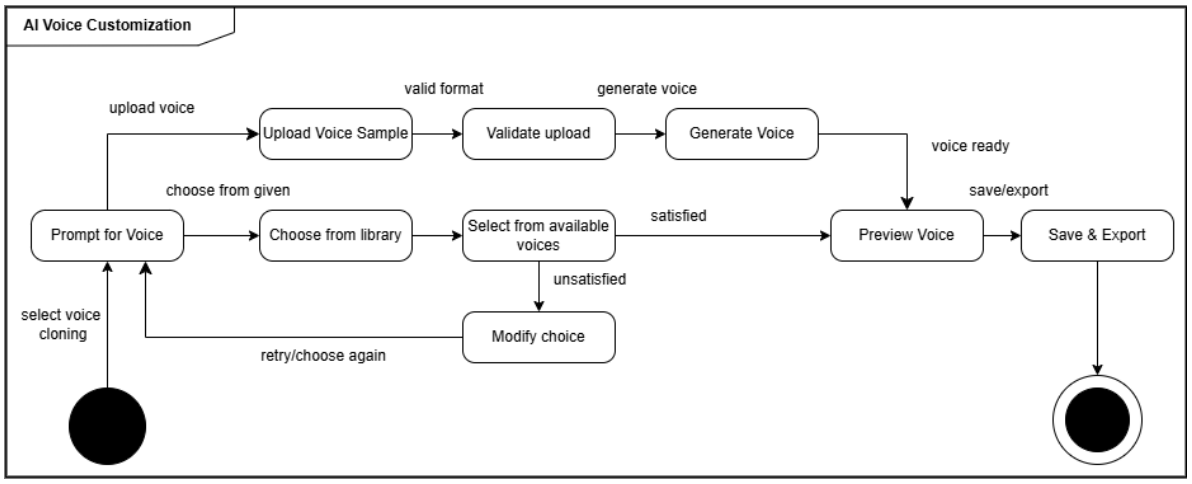


Figure 30: State transition diagram of AI Voice Customization

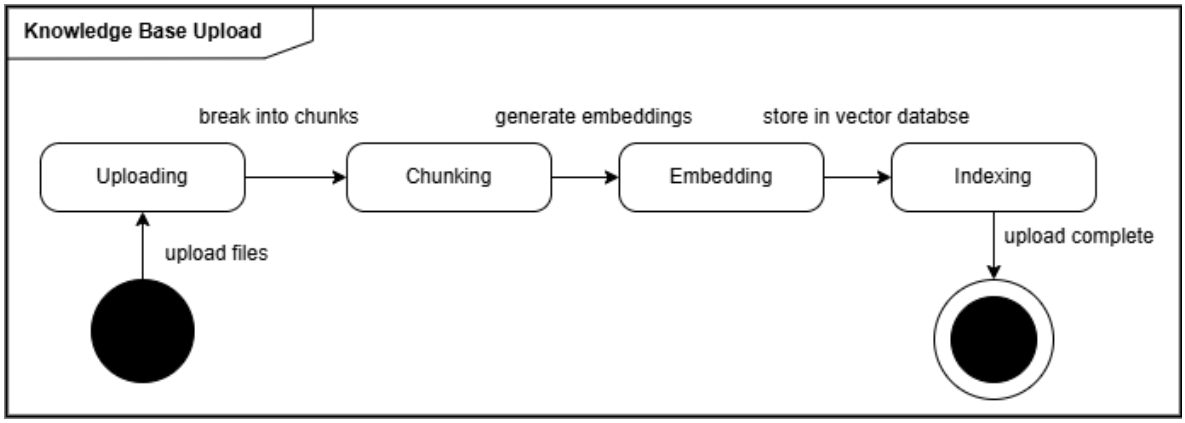


Figure 31: State transition diagram of Knowledge Base Upload

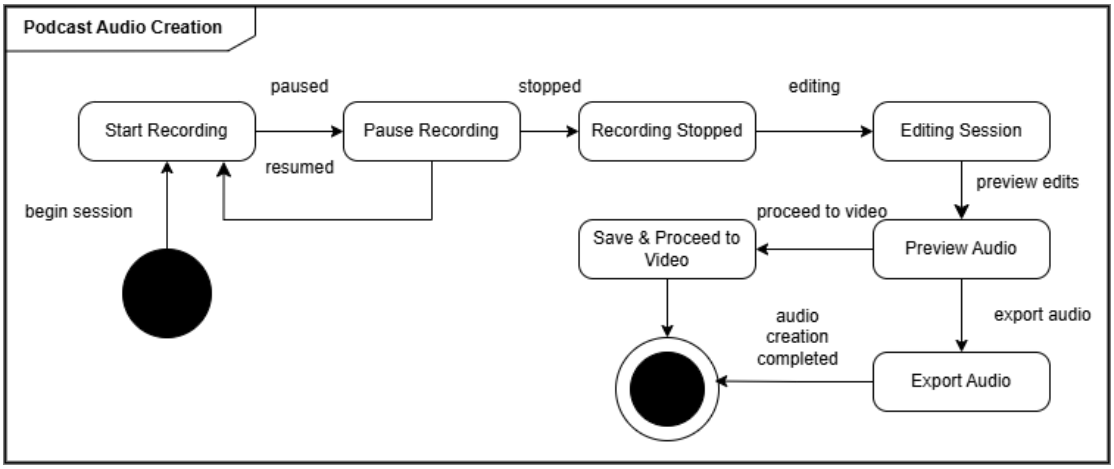


Figure 32: State transition diagram of Podcast Audio Creation

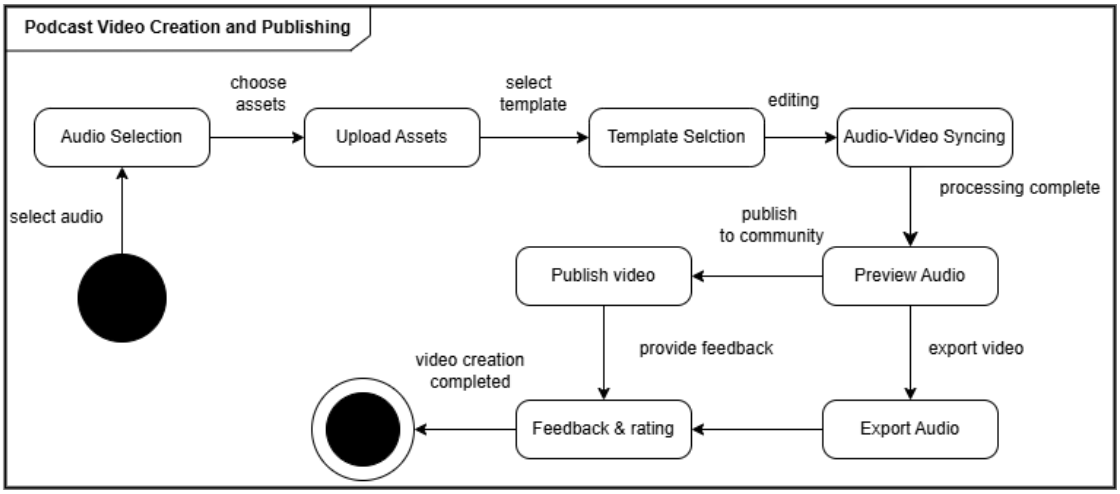


Figure 33: State transition diagram of Podcast Video Creation and Publishing

4.3 Data Design

The data design of PodVerse is as follows:

4.3.1 Data Dictionary

Following is the data dictionary of PodVerse:

4.3.1.1 User Collection

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  full_name TEXT,
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  is_verified BOOLEAN DEFAULT FALSE,
  otp TEXT,
  provider TEXT DEFAULT 'email' CHECK (provider IN ('email', 'google')),
  reset_password_token TEXT,
  reset_password_expires TIMESTAMP,
  profile_url TEXT DEFAULT 'https://example.com/default-profile.png',
  created_at TIMESTAMP DEFAULT NOW()
);
```

Table 39: Data Collection of 'User'

Field Name	Data Type	Constraints	Description
id	UUID	Primary Key	Unique user identifier
full_name	Text	None	User's full name
email	Text	Unique, Not Null	User's email address
password_hash	Text	Not Null	Hashed password
is_verified	Boolean	Default: false	Email verification status
otp	Text	None	One-Time Password for verification
provider	Text	Enum: email/google	Authentication provider
reset_password_token	Text	None	Token for password reset
reset_password_expires	Timestamp	None	Token expiry time
profile_url	Text	Default URL	User profile image

created_at	Timestamp	Default: now()	Account creation timestamp

4.3.1.2 Podcast Collection

```
CREATE TABLE podcasts (
  podcast_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  created_by UUID REFERENCES users(id),
  title TEXT NOT NULL,
  description TEXT,
  category TEXT,
  video_file_path TEXT,
  transcript TEXT,
  cover_image_url TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Table 40: Data Collection of 'Podcast'

Field Name	Data Type	Constraints	Description
podcast_id	UUID	Primary Key	Unique podcast identifier
created_by	UUID	Foreign Key	Linked user ID (creator)
title	Text	Not Null	Podcast title
description	Text	None	Description of the podcast
category	Text	None	Genre/category
video_file_path	Text	None	URL of uploaded podcast video
transcript	Text	None	Auto-generated transcript
cover_image_url	Text	None	Podcast cover/thumbnail
created_at	Timestamp	Default: now()	Creation timestamp

4.3.1.3 Voice Sample Collection

```
CREATE TABLE voice_samples (
  sample_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  uploaded_by UUID REFERENCES users(id),
  sample_audio_url TEXT,
  clone_model_id TEXT,
  created_at TIMESTAMP DEFAULT NOW());
```

Table 41: Data Collection of 'Voice Sample'

Field Name	Data Type	Constraints	Description
sample_id	UUID	Primary Key	Unique sample ID
uploaded_by	UUID	Foreign Key	User who uploaded the voice sample
sample_audio_url	Text	None	URL of uploaded audio sample
clone_model_id	Text	None	ID of trained clone model
created_at	Timestamp	Default: now()	Upload timestamp

4.3.1.4 Knowledge Base Collection

```
CREATE TABLE knowledge_base (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),
  title TEXT,
  source_type TEXT,
  chunk TEXT,
  file_id TEXT,
  uploaded_at TIMESTAMP DEFAULT NOW());
```

Table 42: Data Collection of 'Knowledge Base'

Field Name	Data Type	Constraints	Description
id	UUID	Primary Key	Unique knowledge base ID
user_id	UUID	Foreign Key	Linked user ID
title	Text	None	Name of uploaded knowledge item
source_type	Text	None	Type (PDF, URL, etc.)
chunk	Text	None	Extracted content chunk
file_id	Text	None	Reference to original file
uploaded_at	Timestamp	Default: now()	Upload timestamp

4.3.1.5 AI Speaker Collection

```
CREATE TABLE ai_speakers (
  speaker_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),
  name TEXT NOT NULL,
  language TEXT NOT NULL,
  voice_style TEXT,
  avatar_url TEXT,
  is_host BOOLEAN,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Table 43: Data Collection of 'AI Speaker'

Field Name	Data Type	Constraints	Description
speaker_id	UUID	Primary Key	AI speaker ID
user_id	UUID	Foreign Key	Linked user
name	Text	Not Null	AI speaker name
language	Text	Not Null	Spoken language
voice_style	Text	Nullable	Style (Calm, Energetic, etc.)
avatar_url	Text	Nullable	Avatar/3D model link

4.3.1.6 Subscription Collection

```
CREATE TABLE subscriptions (
  subscription_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),
  plan_type TEXT NOT NULL CHECK (plan_type IN ('Free', 'Pro', 'Premium')),
  start_date DATE,
  end_date DATE,
  is_active BOOLEAN DEFAULT TRUE
);
```

Table 44: Data Collection of 'Subscription'

Field Name	Data Type	Constraints	Description
subscriptionID	String	Primary Key	Unique ID of the subscription
UserID	String	Foreign Key	Subscribed user's ID

planType	String	Not Null	Plan (Free, Pro, Premium)
startDate	DateTime	None	Subscription start date
endDate	DateTime	None	Subscription end/renewal date
isActive	Boolean	Default: true	Whether the subscription is currently active

4.3.1.7 Notification Collection

```
CREATE TABLE notifications (
  notification_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),
  title TEXT,
  message TEXT,
  is_read BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Table 45: Data Collection of 'Notification'

Field Name	Data Type	Constraints	Description
notificationID	String	Primary Key	Unique notification ID
userID	String	Foreign Key	Recipient user ID
title	String	None	Title of the notification
message	String	None	Main content of the notification
createdAt	DateTime	None	Timestamp of the notification
isRead	Boolean	Default: false	Read/unread status

4.3.1.8 Payment Collection

```
CREATE TABLE payments (
  payment_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),
  subscription_id UUID REFERENCES subscriptions(subscription_id),
  amount NUMERIC(10,2) NOT NULL,
  status TEXT NOT NULL,
  payment_method TEXT,
  payment_date TIMESTAMP DEFAULT NOW());
```

Table 46: Data Collection of 'Payment'

Field Name	Data Type	Constraints	Description
paymentID	String	Primary Key	Unique ID of the payment
userID	String	Foreign Key	User who made the payment
subscriptionID	String	Foreign Key	Related subscription
amount	Float	Not Null	Amount paid
status	String	Not Null	Payment status
paymentDate	DateTime	Not Null	Date and time of payment

4.3.1.9 FAQs Chatbot Collection

```
CREATE TABLE faq_bots (
  bot_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),
  bot_name TEXT,
  description TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Table 47: Data Collection of 'FAQs Chatbot'

Field Name	Data Type	Constraints	Description
chatbotID	String	Primary Key	Unique chatbot session ID
userID	String	Foreign Key	User who started the session
sessionStart	DateTime	Not Null	When the session began
sessionEnd	DateTime	Nullable	When the session ended

4.3.1.10 FAQs Entry Collection

```
CREATE TABLE faq_entries (
  faq_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  bot_id UUID REFERENCES faq_bots(bot_id),
  question TEXT NOT NULL,
  answer TEXT NOT NULL,
  category TEXT,
```

```
  created_at TIMESTAMP DEFAULT NOW()
);
```

Table 48: Data Collection of 'FAQs Entry'

Field Name	Data Type	Constraints	Description
faqID	String	Primary Key	Unique ID of the FAQ entry
question	String	Not Null	FAQ question
answer	Text	Not Null	FAQ answer
category	String	Nullable	Category (e.g., Billing, Technical, General)
createdAt	DateTime	Not Null	Entry creation timestamp

4.3.1.11 Comment Collection

```
CREATE TABLE comments (
  comment_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),
  episode_id UUID REFERENCES podcasts(podcast_id),
  content TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP
);
```

Table 49: Data Collection of 'Comment'

Field Name	Data Type	Constraints	Description
commentID	String	Primary Key	Unique identifier for the comment
userID	String	Foreign Key	User who made the comment
episodeID	String	Foreign Key	Episode the comment is associated with
content	Text	Not Null	The actual comment text
createdAt	DateTime	Not Null	Timestamp when the comment was created
updatedAt	DateTime	Nullable	Timestamp when the comment was last edited

4.3.1.12 Rating Collection

```
CREATE TABLE ratings (
  rating_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),
  episode_id UUID REFERENCES podcasts(podcast_id),
  score INTEGER NOT NULL CHECK (score BETWEEN 1 AND 5),
  comment TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Table 50: Data Collection of 'Rating'

Field Name	Data Type	Constraints	Description
ratingID	String	Primary Key	Unique identifier for the rating
userID	String	Foreign Key	User who gave the rating
episodeID	String	Foreign Key	Episode that was rated
score	Integer	Not Null	Rating score (e.g., 1 to 5)
comment	String	Nullable	Optional comment/review associated with the rating
createdAt	DateTime	Not Null	When the rating was submitted

4.3.1.13 Message Collection

```
CREATE TABLE messages (
  message_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  sender_id UUID REFERENCES users(id),
  receiver_id UUID REFERENCES users(id),
  content TEXT NOT NULL,
  sent_at TIMESTAMP DEFAULT NOW(),
  is_read BOOLEAN DEFAULT FALSE
);
```

Table 51: Data Collection of 'Message'

Field Name	Data Type	Constraints	Description
messageID	String	Primary Key	Unique message ID
senderID	String	Foreign Key	User who sent the message
receiverID	String	Foreign Key	User who received the message

content	Text	Not Null	Message text
sentAt	DateTime	Not Null	Timestamp when the message was sent
isRead	Boolean	Default: false	Indicates if the message has been read

5. Implementation

This section contains the implementation details of the system. It includes algorithms, external APIs used by the system, user interface design and deployment details.

5.1 Algorithm

The following are the algorithms used in PodVerse:

Algorithm 1 Registration

Input: user_details (email, password)

Output: Registration status (success or error message)

1. Initialize database_connection
2. Input user_details (email, password, role).
3. Check if role is valid (Applicant or Recruiter):
4. If not valid, return: "Invalid role".
5. Normalize the email to lowercase.
6. Check if the email already exists using checkEmailExists(email):
7. If it exists, return "Email already exists".
8. Encrypt the password using bcrypt (encryptPassword(password)).
9. Save user_details (email and password) to the database.
10. Send a verification email using sendVerificationCode(email).
11. Return "Registration successful. Please verify your email.".

Algorithm 1.1 checkUsernameExists	**Algorithm 1.2: encryptPassword**
Input: username	**Input: password**
Output: boolean value	Output: encrypted_password
1: initialize database_connection 2: input email 3: user_record ← query database for user where email = input email 4: if (user_record exists) then 5: return true 6: else 7: return false	1: initialize encryption_algorithm 2: input password 3: encrypted_password ← encryption_algorithm.encrypt(password) 4: return encrypted_password

Algorithm 2 Login

Input: email,password

Output: Login status (success or error message)

1. Initialize database_connection.
2. Input email and password.
3. Normalize the email to lowercase.
4. Query the database for a user with the given email.
5. If no user is found, return "Invalid email".
6. Compare the provided password with the stored password using bcrypt.compare.
7. If the password is incorrect, return "Invalid password".
8. Check if the user is verified (isVerified).
9. If not verified, return "Email not verified".
10. Generate a secure token (setUser(user)).

11. Store the token as a cookie in the response.
12. Return "Login successful".

Algorithm 3 Voice Cloning	**Algorithm 5 RAGBasedAIConversation**
Input: User's audio sample (.wav)	**Input: audio_file, user_id**
Output: Cloned voice model ID	**Output: response_audio_path, answer_text**
1: Import required libraries (TTS.api.TTS, os, torch, functools) 2: Override torch.load to prevent weight-only loading issues 3: original_load ← torch.load 4: torch.load ← lambda *args, **kwargs: original_load(*args, **kwargs, weights_only=False) 5: Check for GPU availability 6: use_gpu ← torch.cuda.is_available() 7: Print "GPU acceleration enabled/disabled" based on use_gpu 8: Load XTTS v2 model using Coqui TTS API 9: tts ← TTS("tts_models/multilingual/multi-dataset/xtts_v2", gpu=use_gpu, progress_bar=False) 10: Function clone_voice(text_input, speaker_paths, output_path): 11: Call tts.tts_to_file with the following parameters: 12: text ← text_input 13: speaker_wav ← speaker_paths (list of WAV file paths) 14: file_path ← output_path 15: split_sentences ← True 16: language ← "en" 17: This generates cloned voice audio and saves it to output_path	1: Import required libraries (SpeechRecognition, Whisper, FAISS, HuggingFaceEmbeddings, ChatOpenAI, gTTS) 2: Function transcribe_audio(audio_file): 3: Save audio_file temporarily 4: Convert to WAV using FFmpeg 5: Load audio into recognizer 6: Try: 7: Transcribe audio using Google Speech Recognition or Whisper 8: Return transcribed_text 9: Except transcription error: 10: Return "Could not understand audio" 11: Function retrieve_user_knowledge(user_id): 12: Query MongoDB `knowledge_base` for chunks where user_id = input user_id 13: Convert chunks to LangChain Document format 14: Generate FAISS vector store using HuggingFaceEmbeddings 15: Return retriever 16: Function generate_ai_response(query_text, retriever): 17: Define custom PromptTemplate (friendly tone, context-aware) 18: Initialize RetrievalQA with LLM (ChatOpenAI) and retriever 19: Invoke QA chain with query_text 20: Return answer_text and source_documents 21: Function synthesize_audio(answer_text, user_id): 22: Use gTTS to synthesize answer_text into MP3 23: Save as "response_userid.mp3" 24: Return audio_file_path 25: Main Function handle_audio_conversation(audio_file, user_id): 26: transcribed_text ← transcribe_audio(audio_file) 27: If transcribed_text is empty or error: 28: Return error message 29: retriever ← retrieve_user_knowledge(user_id) 30: answer_text, sources ← generate_ai_response(transcribed_text, retriever) 31: audio_file_path ← synthesize_audio(answer_text, user_id) 32: Return audio_file_path, answer_text, sources

Algorithm 6 Audio Transcription
Input: tuser_id, audio_file (uploaded audio file)
Output: Transcribed text of the audio OR error message
1: Start transcription process. 2: Receive user_id and audio_file from the client. 3: Save the uploaded audio as a temporary file on the server. 4: Filename: temp_<original_filename> 5: Check if audio file is empty: 6: If file size is 0 bytes, return "Uploaded audio file is empty". 7: Convert audio file to WAV format (16kHz, mono) using FFmpeg: 8: Input: uploaded audio file 9: Output: converted.wav 10: If FFmpeg conversion fails, return the error. 11: Load converted.wav using speech_recognition.AudioFile. 12: Record the audio data from the WAV file. 13: Transcribe using Google Speech Recognition: 14: If speech is recognized successfully, proceed. 15: If speech is not understood, return "Could not understand audio".

16: If any exception occurs, return the error.
17: Delete temporary files (temp_<original_filename>, converted.wav).
18: Return the transcribed text in JSON response.

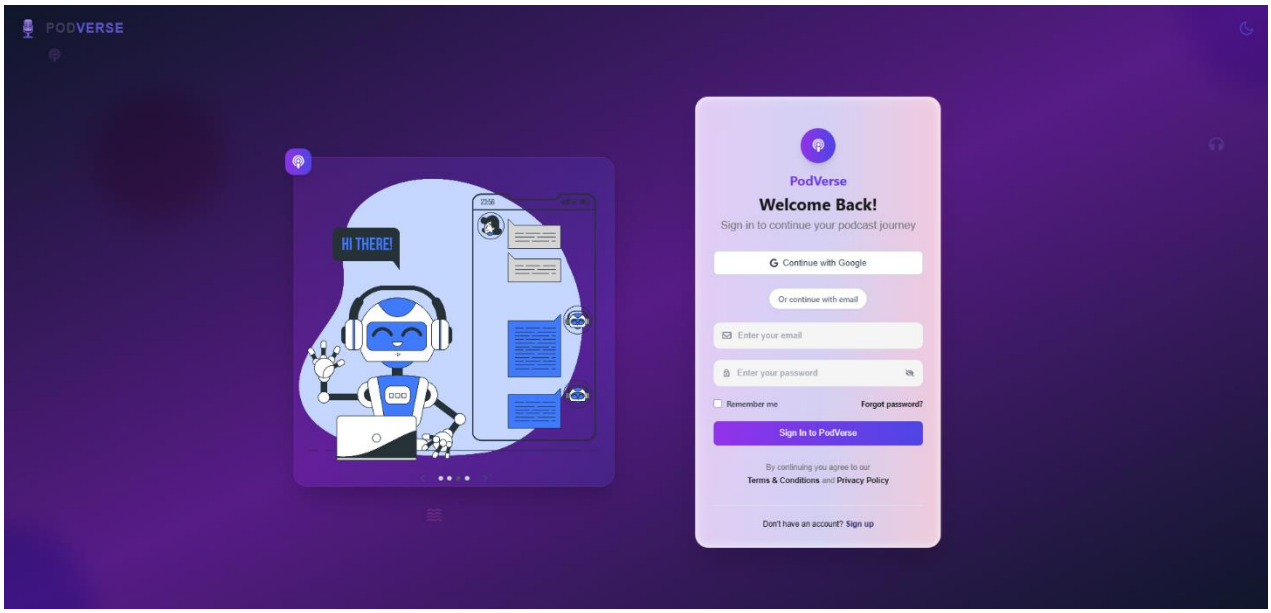
5.2 External APIs/SDKs

Table 52: Details of APIs used in the project

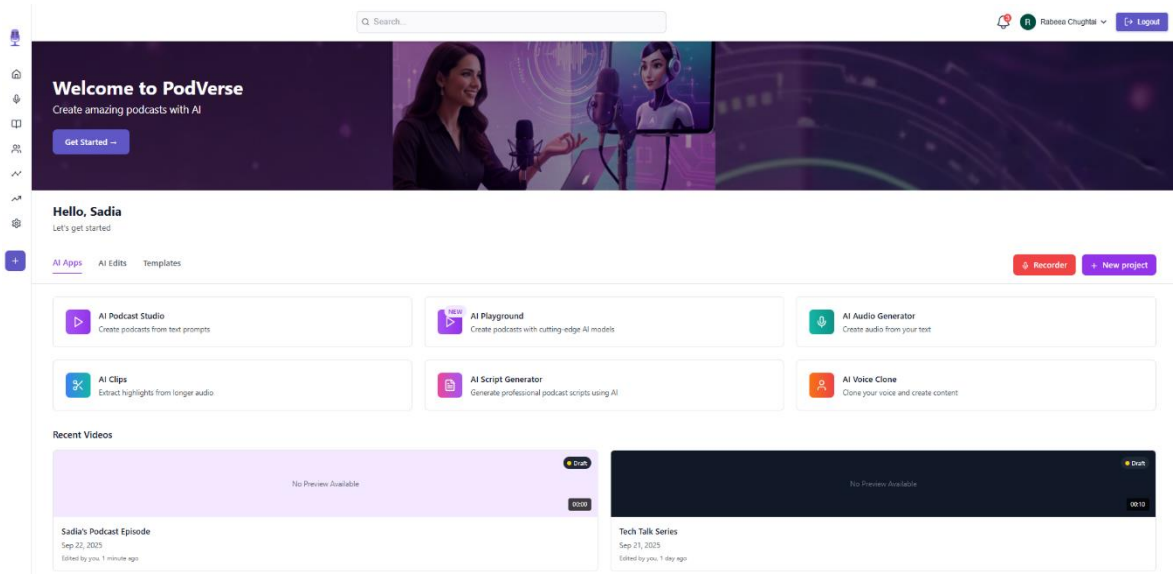
Name of API and version	Description of API	Purpose of usage	List down the API endpoints
Stripe (version 2020-08-27)	Credit Card payment integration	Sandbox used for the orders payment	stripe.paymentMethods.create
Supabase	Backend-as-a-Service providing authentication, database, and storage	Used for user authentication, managing podcast data, and storing user information	https://<project>.supabase.co/auth/v1

5.3 User Interface

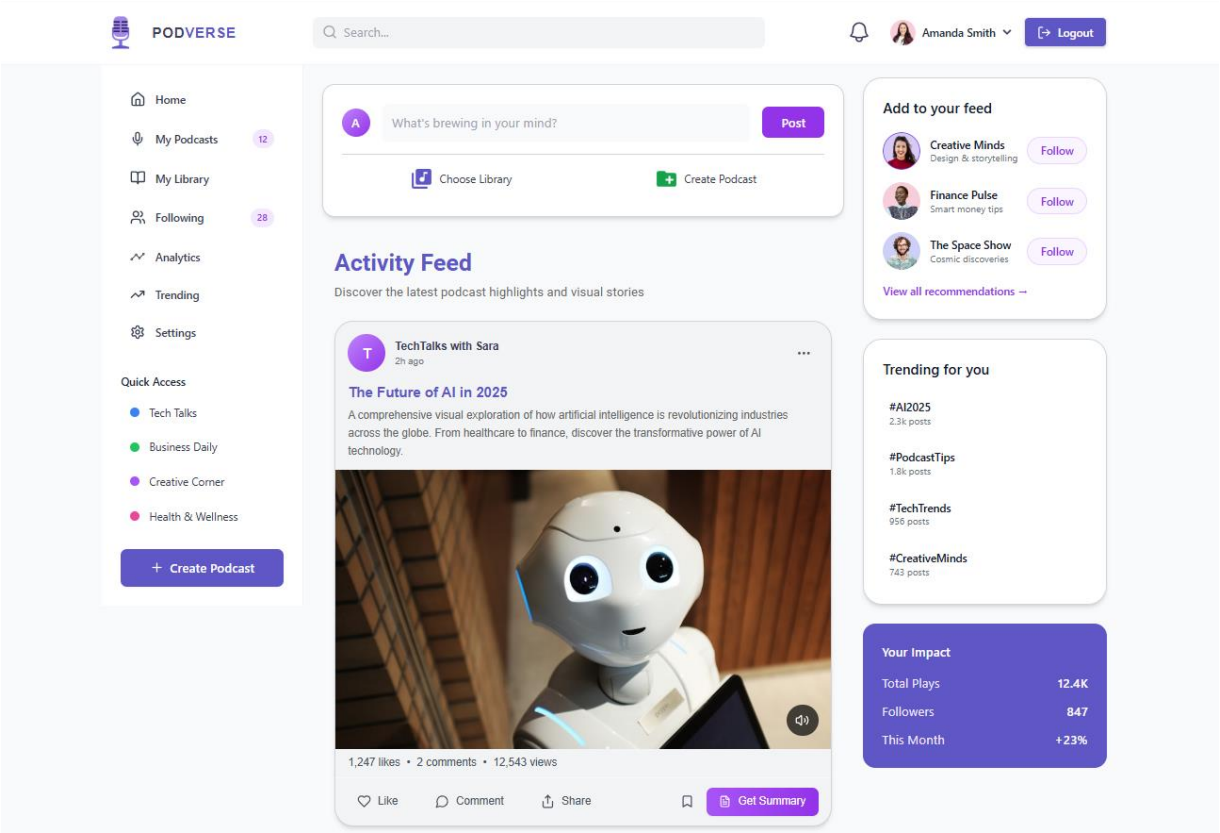
5.3.1 SignIn



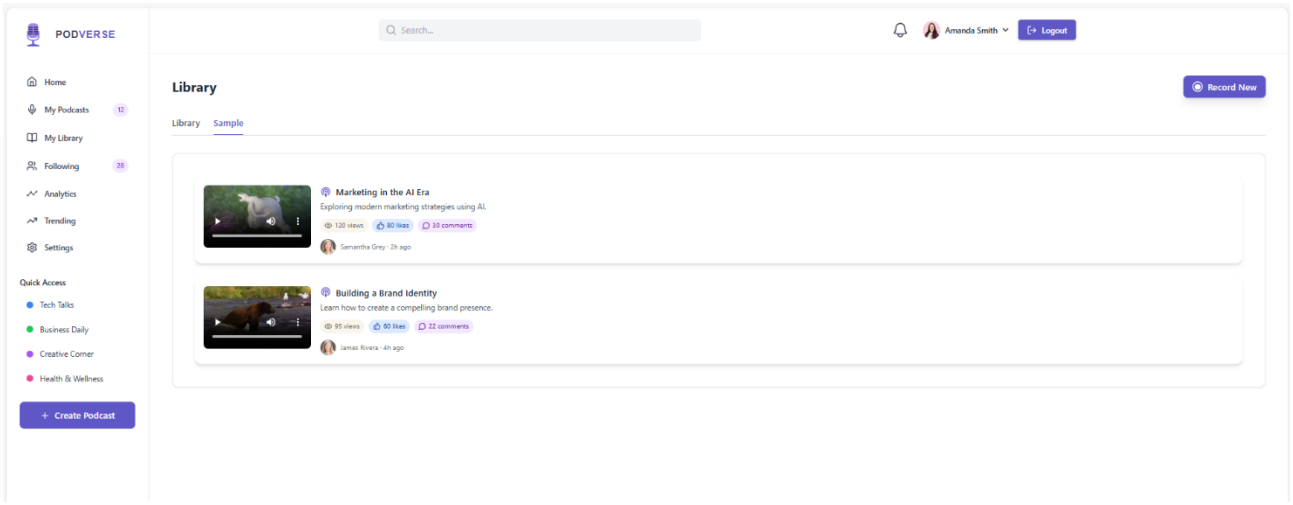
5.3.2 Library



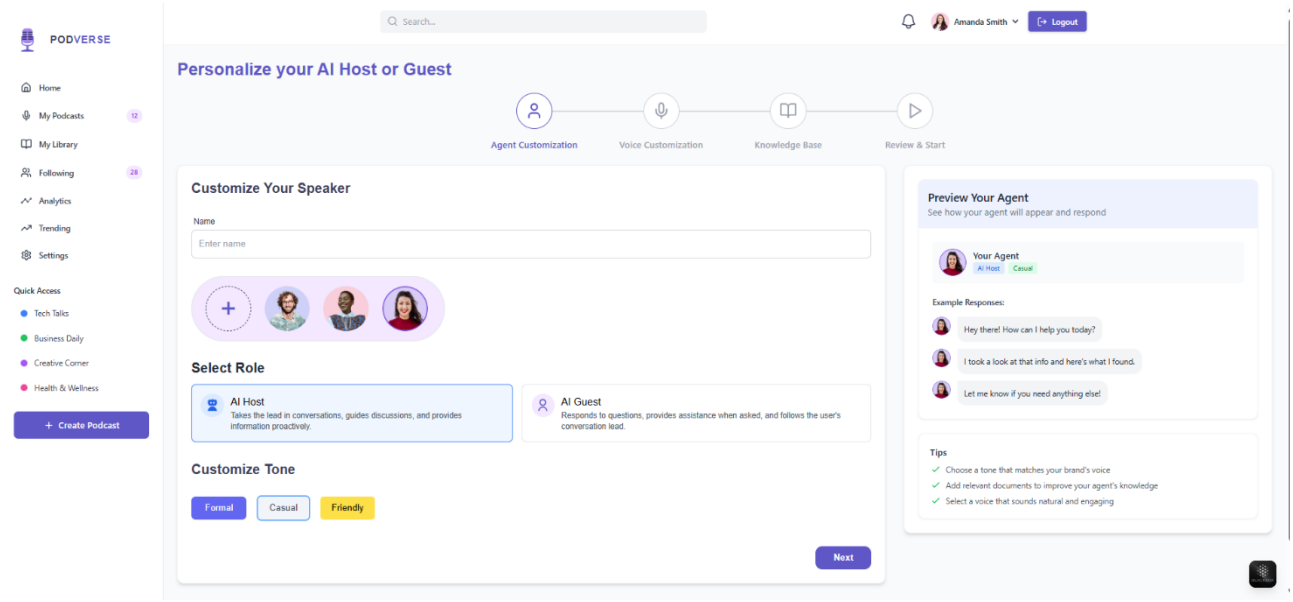
5.3.3 Dashboard



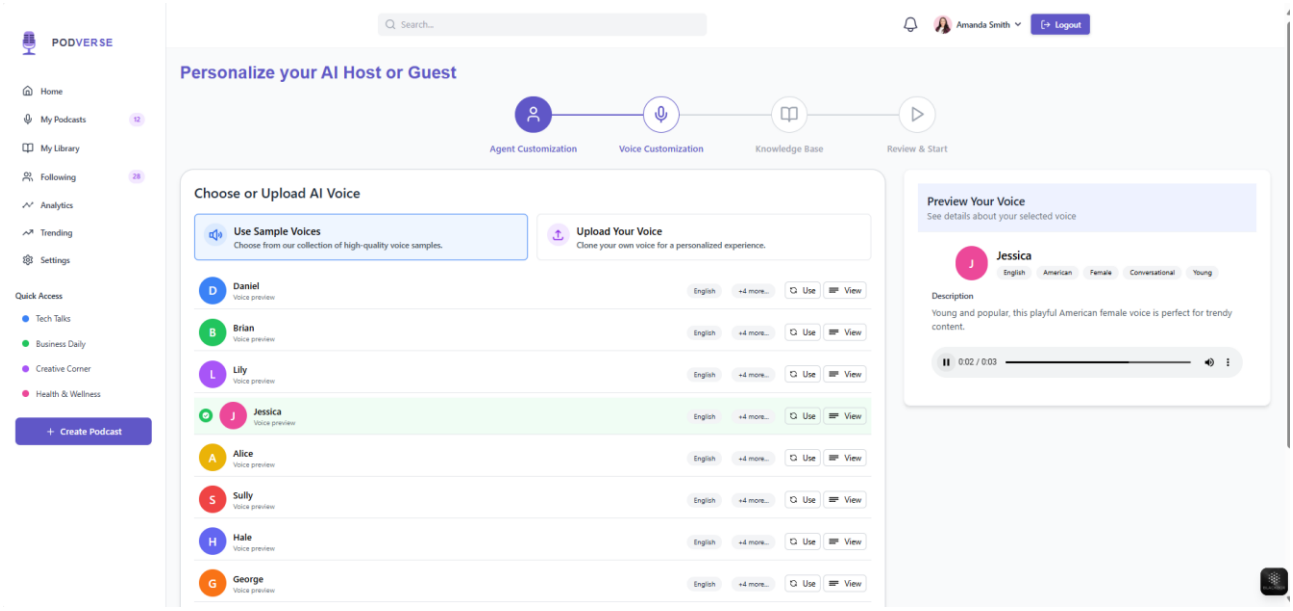
5.3.4 Samples



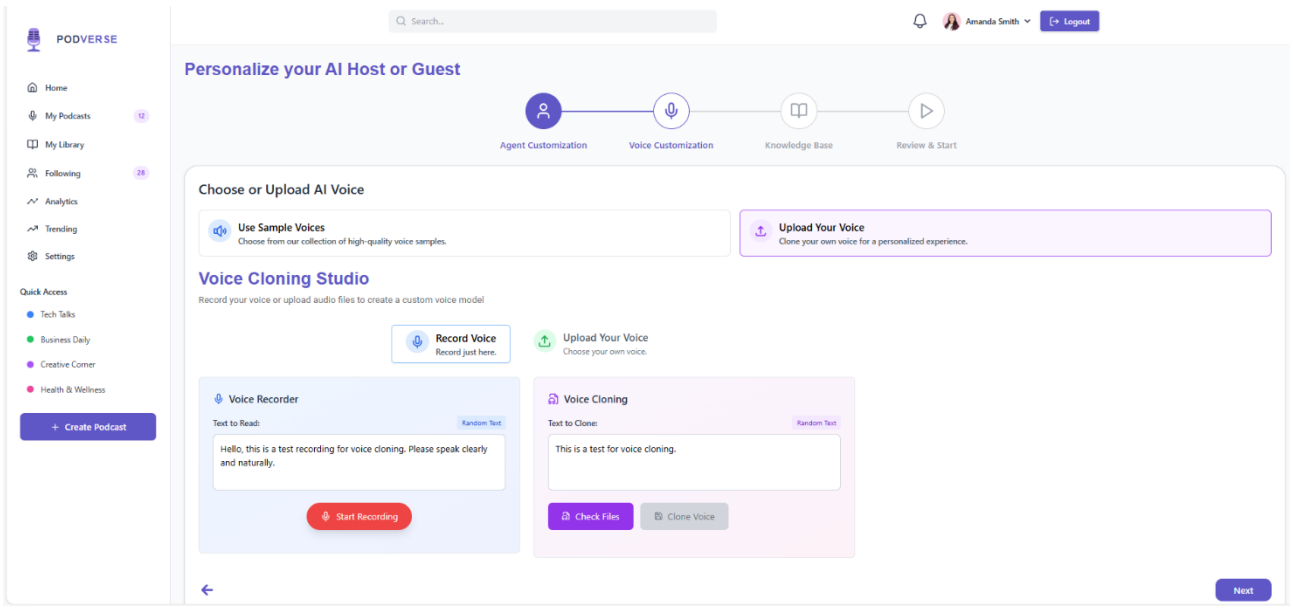
5.3.5 Speaker Customization



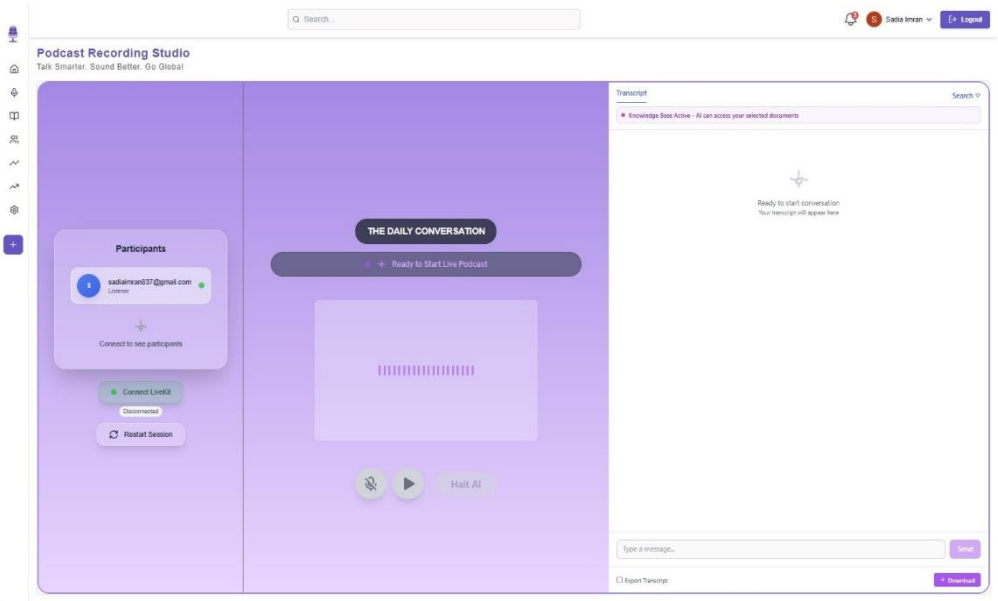
5.3.6 Default Voices



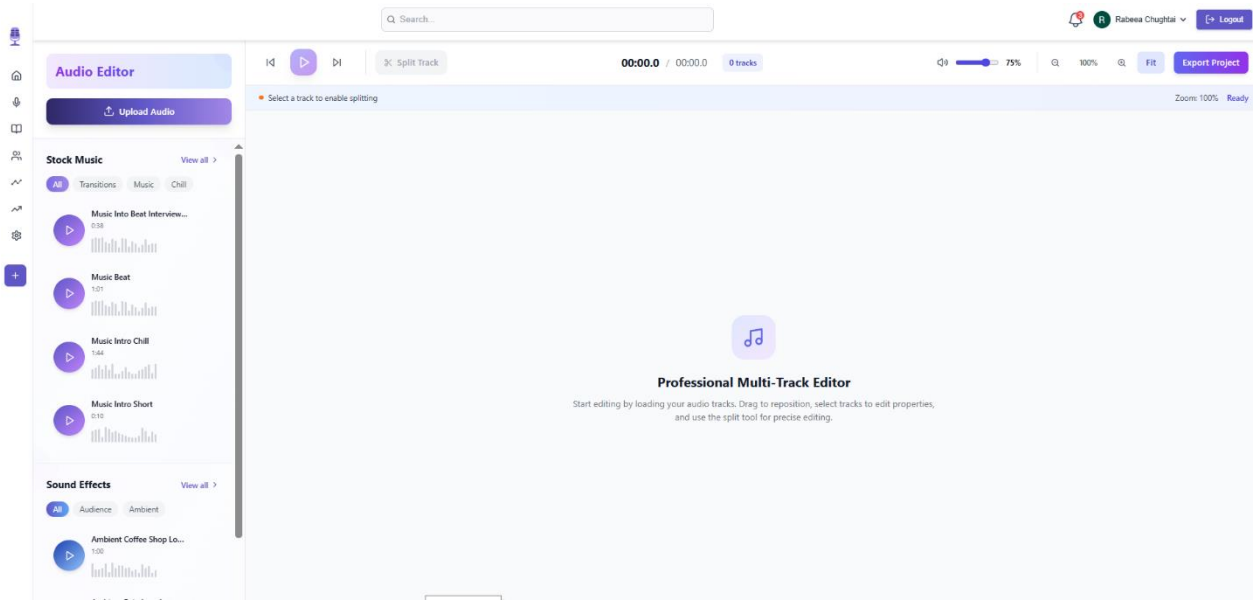
5.3.7 Voice Customization



5.3.8 Recording Studio



5.3.9 Editing Panel



6. Testing and Evaluation

This section includes unit testing, functional testing, and integration testing details for the modules we have developed so far. The purpose of unit testing is to ensure that each component of the software functions as intended.

6.1 Unit Testing

This section has the unit level test cases for PodVerse system:

Unit Testing 1: Email Validation Function

Table 53: Unit Test Cases for Email Validation Function

No.	Test Script	Input	Expected Output	Result
1	validateEmailFormat	Email: `abc@gmail.com`	true	Pass
2	validateEmailFormat	Email: `abc.gmail.com`	false	Pass
3	validateEmailFormat	Email: `` (empty)	false	Pass

Unit Testing 2: checkEmailExists()

Table 54: Unit Test Cases to Check Email Exists

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	checkEmailExists	Email: **sadiaimran837@gmail.com**, Password: Sadia@123	true	Pass
2	checkEmailExists	Email: **sadiaimran837@gmail.com**	Error: "Email already exists"	Pass

Unit Testing 3: Password Encryption & Verification

Table 55: Unit Test Cases for Password Encryption & Verification

No.	Test Script	Input	Expected Output	Result
1	encryptPassword()	Input: `Test123!`	Encrypted string returned	Pass

2	verifyPassword()	Input: `Test123!`, Hash	true	Pass
3	verifyPassword()	Input: `WrongPass`, Hash	false	Pass

Unit Testing 4: OTP Generator & Validator

Table 56: Unit Test Cases for OTP Generator & Validator

No.	Test Script	Input	Expected Output	Result
1	generateOTP()	Triggered	6-digit numeric OTP	Pass
2	verifyOTP()	Input: 123456, Match	success	Pass
3	verifyOTP()	Input: 999999, Mismatch	failure	Pass

Unit Testing 5: Voice Cloning File

Table 57: Unit Test Cases for Voice Cloning File

No.	Test Script	Input	Expected Output	Result
1	processVoiceSample()	File: voice.wav	Cleaned mono audio	Pass
2	extractVoiceProfile()	Cleaned audio	Voice profile (speaker vector) generated	Pass
3	saveVoiceProfile()	Profile data, user ID	Voice profile linked to user in database	Pass

Unit Testing 6: Summarization

Table 58: Unit Test Cases for Summarization

No.	Test Script	Input	Expected Output	Result
1	generateSummary()	Text: 1000 words	Summary ≤ 150 words	Pass

2	generateSummary()	Input: `` (empty)	Error: Missing transcript	Pass
3	saveSummaryToDB()	Summary, transcript_id	MongoDB write success	Pass

Unit Testing 7: Transcription Module

Table 59: Unit Test Cases for Transcription Module

No.	Test Case	Attribute and Value	Expected Result	Result
1	preprocessAudio()	`.wav` file	Cleaned 16kHz audio path	Pass
2	transcribeAudio()	Cleaned audio	Transcript string	Pass
3	saveTranscriptToDB()	Transcript, podcast_id	MongoDB write success	Pass

Unit Testing 8: AI-Conversation Reply Generator

Table 60: Unit Test Cases for AI-Conversation Reply Generator

No.	Test Script	Input	Expected Output	Result
1	generateReplyFromText()	Transcript: "Tell me something"	String reply from AI	Pass
2	saveToConversationLog()	user_id, AI reply	DB write success	Pass

Unit Testing 9: AI Speaker Customization

Table 61: Unit Test Cases for AI Speaker Customization

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Upload valid avatar image	File: avatar.png	Image preview displayed, file saved	Pass

2	Upload unsupported avatar file	File: avatar.txt	Error: "Only PNG, JPG supported"	Pass
3	Select speaker role	Option: AI Guest	Role saved in session	Pass
4	Upload valid knowledge base	File: kb_text.pdf	Text extracted and saved	Pass
5	Upload unsupported knowledge base	File: kb.exe	Error: "Invalid file format"	Pass
6	Save speaker config	Role + Avatar + KB	MongoDB profile updated	Pass

Unit Testing 10: Audio Recorder Module

Table 62: Unit Test Cases for Audio Recorder Module

No.	Test Script	Input	Expected Output	Result
1	startRecording()	Trigger	Mic access granted	Pass
2	stopRecording()	File path	Audio saved	Pass
3	uploadToFirebase()	File: audio.wav	URL returned	Pass

Unit Testing 11: Knowledge Base Processing

Table 63: Unit Test Cases for Knowledge Base Processing

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Upload valid .txt file	File: facts.txt	Text content extracted and saved	Pass
2	Upload valid .pdf file	File: data.pdf	Text extracted from PDF	Pass
3	Upload corrupted file	File: bad.pdf	Error: "Unable to read file content"	Pass

4	Link content to speaker config	Speaker ID: ai_6123abc	MongoDB updated with knowledge source	Pass

6.2 Functional Testing

The following are the functional test cases of PodVerse:

Functional Testing 1: SignUp & OTP Verification

Table 64: Functional Test Cases for SignUp & OTP Verification

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Register with valid details	Email: sadiaimran837@gmail.com, Password: Sadia@123	OTP sent to registered email	OTP screen appeared, email received	Pass
2	Duplicate email	Email: sadiaimran837@gmail.com	Error: "Email already in use"	Error message displayed	Pass

Functional Testing 2: Login

Table 65: Functional Test Cases for Login

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Valid login credentials	Email: rabeeachughtai1@gmail.com, Password: Rabeea@123	Dashboard loaded	User redirected to dashboard	Pass
2	Incorrect password	Email: rabeeachughtai1@gmail.com, Password: Wrong123	Error: "Invalid credentials"	Error toast displayed	Pass

Functional Testing 3: Podcast Transcription

Table 66: Functional Test Cases for Podcast Transcription

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Upload podcast audio	File: podcast.wav	Full transcript displayed on screen	Transcript text loaded in textarea	Pass

2	Upload empty or silent file	File: silence.wav	Error: “No speech detected”	Warning shown in toast	Pass

Functional Testing 4: Voice Cloning

Table 67: Functional Test Cases for Voice Cloning

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Upload valid voice sample	File: user_voice.wav	Voice cloned successfully, stored for reuse	“Voice cloned” message shown	Pass
2	Upload unsupported format	File: user_voice.mp3	Error: “Only WAV files are supported”	Validation error appeared	Pass

Functional Testing 5: Transcript Summarization

Table 68: Functional Test Cases for Transcript Summarization

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Generate summary	Transcript: 1000+ words	Summary displayed (≤150 words)	Short summary shown in summary tab	Pass
2	Click summarize with empty input	Transcript: (empty)	Error: "Transcript required"	Validation toast: “Transcript empty”	Pass

Functional Testing 6: AI-User Conversation

Table 69: Functional Test Cases for AI-User Conversation

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Upload mic prompt	File: user_prompt.wav	AI reply generated and shown	Reply displayed below waveform	Pass
2	Submit empty prompt	File: silence.wav	Error: “Please speak to continue”	“Please speak” toast shown	Pass

Functional Testing 7: Podcast Audio Recording & Upload

Table 70: Functional Test Cases for Podcast Audio Recording & Upload

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Start/Stop recording	Record → Stop clicked	Recording preview generated	Waveform rendered	Pass
2	Upload to Firebase	File: recorded_audio.wav	File stored, Firebase URL returned	Link received and saved	Pass
3	Save metadata	Title: “Ep1”, Tags: [AI, Tech]	Podcast entry created in MongoDB	Metadata saved, confirmation shown	Pass

Functional Testing 8: Podcast Library Access

Table 71: Functional Test Cases for Podcast Library Access

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Load personal podcast list	User ID: rabeeachughtai1@gmail.com	User podcasts listed	Podcast cards shown on screen	Pass
2	Play uploaded podcast	File URL from DB	Audio plays in embedded player	Media player started with audio	Pass

3	Show sample podcasts only	Filter: sample=true	Sample podcasts displayed	Filtered list of samples shown	Pass

Functional Testing 9: AI Speaker Customization

Table 72: Functional Test Cases for AI Speaker Customization

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Upload avatar image	File: avatar.png	Avatar displayed and saved	Image shown on profile	Pass
2	Choose speaker role	Role: AI Host	Role stored successfully	Display updated with AI Host tag	Pass
3	Upload valid knowledge base	File: kb_notes.pdf	Text extracted and attached to AI speaker	Content linked and displayed	Pass
4	Upload unsupported KB	File: kb.exe	Error: “Invalid file format”	Error toast: "Only PDF/TXT allowed"	Pass

6.3 Business Rules Testing

This section contains various decisions and actions according to the business rules.

Business Rules Testing 1: Only one account can be registered for a given email address

Table 73: Business Rules Testing 1

Conditions	Rule 1	Rule 2	Rule 3
Email already registered	Yes	No	No
Password valid	Yes	Yes	No
Actions			
Account created	No	Yes	No

Business Rules Testing 2: Password must meet security criteria

Table 74: Business Rules Testing 2

Conditions	Rule 1	Rule 2	Rule 3
Password contains digits	Yes	No	No
Password contains letters	Yes	Yes	No
Actions			
Account created	Yes	No	No

Business Rules Testing 3: Voice Cloning – Audio Upload Constraints

Table 75: Business Rules Testing 3

Conditions	Rule 1	Rule 2
File format supported	Yes	No
Audio meets quality standards	Yes	Yes
Audio accepted	Yes	No
File format supported	Yes	No

Business Rules Testing 4: Knowledge Upload Format

Table 76: Business Rules Testing 4

Conditions	Rule 1	Rule 2
File format supported	Yes	No
Data is structured	Yes	Yes
File accepted	Yes	No

6.4 Integration Testing

This section has the integration test cases for the modules that we have developed. They make sure that the connections and interfaces between various components of the system function correctly.

Integration Testing 1: User Registration & Email Verification

Table 77: Integration Testing 1 for User Registration & Email Verification

No.	Test case/ Test script	Attribute and value	Expected result	Actual result	Result
1	Valid Registration	Email, Password, Role = "Applicant"	"Registration successful. Please verify your email."	Same	Pass
2	Duplicate Email	Already existing email	"Email already exists"	Same	Pass
3	Invalid Role	Role = "Manager"	"Invalid role"	Same	Pass

Integration Testing 2: Login with Verification Flow

Table 78: Integration Testing 2 for Login with Verification Flow

No.	Test case/ Test script	Attribute and value	Expected result	Actual result	Result
1	Login with verified user	Email, Password	"Login successful"	Same	Pass
2	Login with unverified user	Valid credentials	"Email not verified"	Same	Pass
3	Incorrect password	Valid email, wrong password	"Invalid password"	Same	Pass

Integration Testing 3: Voice Cloning Flow

Table 79: Integration Testing 3 for Voice Cloning Flow

No.	Test case/ Test script	Attribute and value	Expected result	Actual result	Result
1	Clone voice with valid .wav	UserID, audio_sample	"Voice cloned successfully", path	Same	Pass
2	Upload unsupported format	.mp3 file	Error: "Unsupported format"	Same	Pass

Integration Testing 4: Podcast Transcription

Table 80: Integration Testing 4 for Podcast Transcription

No.	Test case/ Test script	Attribute and value	Expected result	Actual result	Result
1	Transcribe valid podcast	Valid .wav file	Transcript returned and saved	Same	Pass
2	Upload non-WAV file	.mp3	Error: Invalid format or convert	Same	Pass

Integration Testing 5: Transcript Summarization

Table 81: Integration Testing 5 for Transcript Summarization

No.	Test case/ Test script	Attribute and value	Expected result	Actual result	Result
1	Summarize valid transcript	1024 token input	Summary saved in MongoDB	Same	Pass
2	Excessively long input	2000+ tokens	Summarized within token limits	Same	Pass

Integration Testing 6: AI Podcast Generation

Table 82: Integration Testing 6 for AI Podcast Generation

No.	Test case/ Test script	Attribute and value	Expected result	Actual result	Result
1	Record podcast with 2 speakers	conversation_list	Podcast saved in Firebase, metadata stored	Same	Pass
2	Missing speaker embedding	Unknown role	Error or fallback voice	Same	Pass

7. Conclusion and Future Work

7.1 Conclusion

PodVerse successfully demonstrates how AI can simplify and enhance podcast creation for small business owners, entrepreneurs, and content creators. By combining voice cloning, AI-driven conversations, and editing, the system reduces the need for complex tools and technical expertise. It provides a seamless platform where users can produce high-quality, and engaging podcasts efficiently. Overall, PodVerse bridges creativity and technology to make podcasting more accessible and intelligent.

7.2 Future Work

Future enhancements for PodVerse may include:

- Mobile Application: Introducing a mobile version for on-the-go podcast creation and management.
- Advanced Analytics: Providing listener insights, engagement tracking, and performance reports.
- Multi-Voice Collaboration: Allowing multiple AI guests or hosts to participate simultaneously.
- Enhanced Personalization: Offering deeper customization for AI voice tone, style, and emotion.
- Integration with Streaming Platforms: Automating publishing to platforms like Spotify, Apple Podcasts, and YouTube.

8. References

1. P. Nave. “Exploring RAG: Essential Steps to Implementing AI Agents and Deep Learning Strategies.” Internet: https://medium.com/@pratikabnave97/exploring-rag-essential-steps-to-implementing-ai-agents-and-deep-learning-strategies-f41ceda143c4, Mar. 10, 2024 [Nov. 10, 2024].
2. S. Smith. “The Future of Podcasting in 2024 and Beyond: Key Predictions.” Internet: https://www.opus.pro/blog/the-future-of-podcasting-in-2024-and-beyond-key-predictions, Mar. 22, 2024 [Nov. 10, 2024].
3. G. Harris. “AI Knowledge Base: Enhancing Intelligence for Smart Decisions.” Internet: https://www.getguru.com/reference/ai-knowledge-base, Apr. 5, 2024 [Nov. 10, 2024].

9. Plagiarism Report

PodVerse.docx

ORIGINALITY REPORT

4%	1%	0%	2%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Higher Education Commission Pakistan Student Paper	2%
2	www.coursehero.com Internet Source	1%
3	Submitted to PSBAcademy (ACPeSolutions) Student Paper	1%
4	Submitted to Lindenwood University Student Paper	1%
5	Submitted to Stella Maris College Student Paper	1%
6	Submitted to Colorado State University, Global Campus Student Paper	<1%
7	Submitted to Myongji University Graduate School Student Paper	<1%
8	fsagric.fs.gov.za Internet Source	<1%